PolyPublie
Polytechnique Montréal

POLYTECHNIQUE MONTRÉAL
UNIVERSITÉ D'INGÉNIERIE

Titre: Title:	Guidage et planification réactive de trajectoire d'un drone monoculaire contrôlé par intelligence artificielle
Auteur: Author:	Alexandre Duperré
Date:	2020
Type:	Mémoire ou thèse / Dissertation or Thesis
Référence: Citation:	Duperré, A. (2020). Guidage et planification réactive de trajectoire d'un drone monoculaire contrôlé par intelligence artificielle [Mémoire de maîtrise, Polytechnique Montréal]. PolyPublie. https://publications.polymtl.ca/5399/

Document en libre accès dans PolyPublie
Open Access document in PolyPublie

URL de PolyPublie: PolyPublie URL:	https://publications.polymtl.ca/5399/
Directeurs de recherche: Advisors:	Sofiane Achiche, & David Saussié
Programme: Program:	Génie aérospatial

Ce fichier a été téléchargé à partir de PolyPublie, le dépôt institutionnel de Polytechnique Montréal
This file has been downloaded from PolyPublie, the institutional repository of Polytechnique Montréal

https://publications.polymtl.ca

POLYTECHNIQUE MONTRÉAL

affiliée à l'Université de Montréal

Guidage et planification réactive de trajectoire d'un drone monoculaire contrôlé par intelligence artificielle

ALEXANDRE DUPERRÉ

Département de génie mécanique

Mémoire présenté en vue de l'obtention du diplôme de *Maîtrise ès sciences appliquées*

Génie aérospatial

Août 2020

© Alexandre Duperré, 2020.

POLYTECHNIQUE MONTRÉAL

affiliée à l'Université de Montréal

Ce mémoire intitulé :

Guidage et planification réactive de trajectoire d'un drone monoculaire contrôlé par intelligence artificielle

présenté par **Alexandre DUPERRÉ**

en vue de l'obtention du diplôme de *Maîtrise ès sciences appliquées*

a été dûment accepté par le jury d'examen constitué de :

Maxime RAISON, président
Sofiane ACHICHE, membre et directeur de recherche
David SAUSSIÉ, membre et codirecteur de recherche
Giovanni BELTRAME, membre

REMERCIEMENTS

J’aimerais commencer par remercier mes superviseurs, Prof. Sofiane Achiche et Prof. David Saussié, professeurs aux Départements de génie mécanique et de génie électrique de Polytechnique Montréal. Merci de votre soutien, de vos précieux conseils, du temps investi dans vos étudiants et de la merveilleuse opportunité que vous avez su m’offrir. Ce fut un immense plaisir d’entreprendre ce projet académique d’envergure sous votre tutelle.

J’aimerais également remercier le Prof. Reza Hoseinnezhad et Prof. Alireza Bab-Hadiashar pour m’avoir accueilli si chaleureusement au sein du Intelligent Automation Research Group. Mon expérience au RMIT fut définitivement inoubliable. Ce séjour en Australie m’a permis de rencontrer des personnes exceptionnelles, dont mes colocataires Reza Omrani et Colorful Shih, que j’aimerais remercier du fond du cœur pour leur générosité, leur hospitalité et surtout pour les merveilleux moments passés ensemble.

J’aimerais exprimer ma reconnaissance aux membres du jury, constitué des professeurs Giovanni Beltrame et Maxime Raison, pour avoir accepté la tâche d’évaluer le travail effectué dans cette *Maîtrise ès sciences appliquées*. Je tiens à remercier spécialement Gabriel Bernard ainsi que Cédric Leblond Ménard pour leurs précieux conseils et leur volonté à aider leurs pairs. Merci à mon cousin Olivier Brochu-Dufour pour avoir entrepris une maîtrise recherche avec moi; j’ai énormément apprécié le temps passé en ta compagnie. Je te promets que je vais faire de mon mieux pour mieux organiser mes scripts de programmation! J’en profite aussi pour remercier tous les membres du laboratoire CoSIM, pour avoir su rendre l’atmosphère si agréable tout au long de mon passage. Je vais beaucoup m’ennuyer de jouer aux cartes chaque midi avec vous.

Le travail présenté ci-après a nécessité l’utilisation d’une installation particulière du système d’exploitation Linux en concordance avec l’infrastructure informatique du département de génie mécanique. Je remercie donc Alain Robidoux et Richard Mottet pour leur constant support dans la gestion de tous les problèmes informatiques rencontrés au cours de ce projet.

Je souhaite adresser une mention spéciale à ma famille, soit mon père Michel, ma mère Nathalie ainsi que mon frère Samuel, qui malgré la distance, ont toujours été proches de moi et ont su me soutenir et à m’encourager dans mes projets. Votre soutien à travers mes années aux études supérieures m’a toujours fait chaud au cœur.

Merci à mes amis d'avoir contribué à tous les beaux souvenirs qui ont découlé de ces deux dernières années. Une mention spéciale à mes colocataires Mathieu Ashby et Manuel Génier qui ont toujours su montrer leur intérêt pour mon projet et ont partagé leur expertise. Vos conseils m'ont énormément aidé à l'atteinte de mes objectifs. Sans vous mon confinement dû au COVID-19 aurait été beaucoup moins agréable…

Enfin, j'aimerais remercier les organismes subventionnaires du Fonds de Recherche du Québec - Nature et Technologies (FRQNT), du Conseil de Recherches en Sciences Naturelles et en Génie du Canada (CRSNG), du Programme de bourse du Ministère de l'Éducation et de l'Enseignement Supérieur (MEES), Mitacs et finalement la Fondation et alumni de Poytechnique Montréal. Votre soutien financier a rendu possible l'obtention des résultats présentés dans ce mémoire et je vous en suis infiniment reconnaissant.

RÉSUMÉ

Le problème de guidage autonome est un domaine de recherche en constante évolution. La popularisation des drones a étendu ce domaine de recherche au cours des dernières années. La nature de ce type d'engins amène plusieurs nouveaux défis à surmonter, notamment en lien avec la variété d'environnements auxquels ils peuvent être confrontés. Contrairement aux voitures autonomes, les drones se retrouvent souvent dans des milieux inconnus non cartographiés et dépourvus de signal GPS. De nouvelles méthodes ont donc été développées pour mitiger ces défis.

Les solutions au problème de guidage autonome dans la littérature peuvent dans ce mémoire de maîtrise être classées dans deux catégories : le guidage réactif localement à des fins d'exploration et le guidage orienté. La première catégorie regroupe les solutions de guidage local d'engins naviguant sans destination précise alors que la seconde regroupe celles de guidage tentant d'atteindre une destination. Les deux catégories de guidage en milieu inconnu utilisent majoritairement des approches incluant l'apprentissage par renforcement ainsi que l'apprentissage par imitation. Cependant, peu d'études abordent le problème de guidage orienté dans des environnements complexes de grandeur nature.

L'objectif de ce projet de recherche est donc de concevoir un agent intelligent capable d'imiter la logique de guidage d'un humain dans un environnement inconnu complexe en se basant sur la vision de profondeur et une estimation de sa destination. Une approche utilisant l'apprentissage par imitation est employée pour minimiser les coûts et les temps de calcul. Un environnement de simulation sophistiqué est donc mis sur place afin de créer un ensemble de données pour l'entraînement par imitation. L'ensemble de données qui a été créé comporte 624 trajectoires parmi 9 environnements différents effectuées par un expert suboptimal pour un total de 296 466 paires d'entraînement. L'attributif suboptimal est employé pour qualifier l'humain à imiter puisque ce dernier devra dresser les trajets au meilleur de ses capacités sans avoir recours à des algorithmes de planification de trajectoire optimale.

Un modèle de classification capable de prédire la prochaine commande de guidage à effectuer compte tenu des observations actuelles et précédentes a été implémenté. Le modèle est entraîné à encoder une représentation de l'image de profondeur obtenue à partir de l'image RGB ainsi qu'une représentation des coordonnées relative à sa destination. Ces représentations sont traitées par un réseau récurrent à mémoire court et long terme («Long Short-Term Memory» ou LSTM) ainsi

qu'un perceptron multicouches («Multilayer Perceptron» ou MLP) afin de prédire la direction à emprunter. Une fonction coût adaptée au problème ainsi que des techniques d'augmentation de l'ensemble de données sont incorporées lors de l'entraînement afin d'améliorer la précision du modèle en validation et en test. Une recherche d'hyperparamètres de type *grid search* a été effectuée afin de sélectionner le meilleur modèle selon la précision obtenue sur l'ensemble de données de test. Des précisions entre 77.10% et 82.59% ont été atteintes indiquant un impact significatif des méthodes d'augmentation de l'ensemble de données.

Les résultats de simulation n'ont cependant pas été corrélés directement avec les performances en entraînement. Il a été confirmé que l'hypothèse selon laquelle les paires d'entraînement sont identiquement indépendamment distribuées est en fait erroné dans un contexte de processus de décision Markovien tel que prédit par la théorie, car chaque action induit un nouvel état dépendant du précédent. Les erreurs effectuées par le modèle peuvent donc s'accumuler et le pousser dans une situation jamais rencontrée par l'expert. Il est dans ce cas très difficile pour le modèle de choisir la bonne action à poser et peut occasionner des accidents. Malgré les techniques d'augmentation introduites, les meilleurs modèles en entraînement prennent ainsi les dernières positions dans le tableau des performances de simulation, laissant les modèles initialement en deuxième position en entraînement au podium en simulation. Les modèles performants en simulation semblent donc avoir appris une représentation plus précise et moins rigide de la politique de guidage de l'expert. Les modèles performants en simulations ont en fait été en mesure d'imiter le comportement de l'expert et atteindre toutes les destinations en obtenant des erreurs entre 1.9 m et 3.2 m par rapport aux trajectoires effectués par l'expert. Plusieurs expérimentations ont été effectuées afin d'évaluer les aptitudes de généralisation du modèle en simulation. Les variations en vitesse, en altitude et en fréquence de rafraîchissement de la mémoire du modèle ont prouvé que l'agent autonome obtenu parvient tout de même à atteindre les destinations. Des analyses supplémentaires justifient l'utilisation d'images de profondeur par opposition aux images RGB ainsi que l'utilisation d'un module récurrent dans la représentation de l'état observé.

Une multitude d'améliorations sont suggérées concernant l'utilisation et la création d'un ensemble de données destiné à l'apprentissage par imitation pour le guidage autonome. Il est également suggéré d'incorporer l'image RGB à l'image de profondeur considérant certains éléments de comportement du modèle entraîné sur les images RGB. Les résultats encourageants obtenus motivent donc à entreprendre les tests hors simulation dans la suite du projet.

ABSTRACT

The autonomous guidance field is a continuously evolving research topic. The popularization of micro aerial vehicles such as quadcopters has contributed to the expansion of this research topic. Because of the wide range of different environments they can navigate into, quadcopters have many challenges on their own. In contrast with autonomous cars, quadcopters will most likely navigate more often in unknown environments with limited or no GPS service. New methods for autonomous guidance were needed for quadcopters.

The literature review reveals two main categories relevant to the autonomous guidance problem: locally passive-reactive guidance and oriented guidance. The former includes all forms of guidance not aiming for a specific target while the latter focuses on reaching a destination. Both categories are considering guidance in unknown environments and use mostly reinforcement learning or imitation learning as a solving method. However, most of the studies on autonomous oriented guidance are not executed in a full size, complex environment setting.

The objective of this research project is to create an intelligent agent capable of imitating a human guidance policy in a complex and unknown environment based on a depth map image and relative goal inputs. Considering the lower cost in development and computation time, the imitation learning approach was chosen. A sophisticated simulation environment was set up to create an imitation learning datasets. A total of 624 suboptimal demonstration paths from 9 different 3D environments were gathered, which represent 296 466 learning pairs. The demonstrations are qualified as suboptimal since the expert is a human trying its best to solve the guidance problem without any optimal planners.

A classification model was introduced for predicting the appropriate guidance command based on the observations over time. The model learned a meaningful representation of its inputs which can be processed by a long short-term memory network (LSTM) followed by a fully connected network. In this way, the depth image obtained from the RGB original image along with the relative coordinates to the destination are converted into a guidance command at each time step. In order to improve the classification accuracy on the test set, a custom loss function and data augmentation techniques were implemented.

A grid search over possible combination of dataset augmentation proportions was conveyed to optimize the hyperparameters. Accuracy ranging between 77.10% and 82.59% were obtained for this experiment, revealing a significant dependency to the augmentation technique.

Simulation results were, however, not directly correlated with the classification accuracy on the test set. We found that the independent and identically distributed assumption of the training examples was in fact no longer valid in an MDP problem as predicted by the theory since each state is influenced by the action taken at the previous state. Errors made by the agent can then add up over time and lead the agent into a state never encountered by the expert during the training. It is in this case extremely difficult fort the agent to choose the proper action and can cause a crash. Despite the augmentation techniques used, the best models according to the test accuracy were in fact the worst in simulation, leaving the second-best models in first place in the simulation. Thus, models performing well in simulation appeared to have learned a better representation and a less rigid formulation of the expert's guidance policy. The simulation's high-performing models were in fact, able to imitate the expert's behavior and reach all destinations with a mean error ranging between 1.9 m and 3.2 m.

The generalization ability of the policy learned by the model was also evaluated in this work. The model has shown its ability to perform through a wide range of speed, altitude and memory refresh rate. Cross-validation was also carried out to test the impact of the selected validation and test sets. Experiments with the model also validate the hypothesis in favor of using depth maps as opposed to RGB images as well as using recurrent models.

Various improvements concerning the use and the creation of an imitation learning dataset are then proposed. Considering certain positive behaviors when using RGB image inputs, it is also proposed to incorporate RGB image to the depth input in future works. In the light of the results obtained in this thesis, it is fair to say that the next step of this work will include real-world testing.

TABLE DES MATIÈRES

REMERCIEMENTS III

RÉSUMÉ V

ABSTRACT VII

TABLE DES MATIÈRES IX

LISTE DES TABLEAUX XII

LISTE DES FIGURES XIV

LISTE DES SIGLES ET ABRÉVIATIONS XVII

LISTE DES ANNEXES XIX

CHAPITRE 1 INTRODUCTION 1

1.1 Problématique 1

1.2 Objectifs 3

1.3 Plan du mémoire 3

CHAPITRE 2 REVUE DE LITTÉRATURE 5

2.1 Notions d'apprentissage profond 5

2.1.1 Méthodes d'apprentissage 5

2.1.2 Types de réseaux communs 7

2.2 Estimation de profondeur 9

2.3 Guidage autonome 12

2.3.1 Exploration non structurée 13

2.3.2 Planification de trajectoire globale 17

CHAPITRE 3 MÉTHODOLOGIE 24

3.1 Interprétation de la profondeur 24

3.2 Sélection du drone 25

3.2.1 Caractéristiques recherchées 25
3.2.2 AR Drone 2.0 27
3.3 Environnement de simulation 27
3.3.1 Robot Operating System 28
3.3.2 Gazebo 28
3.3.3 Modèle du drone 28
3.3.4 Communication interprogrammes 30
3.4 Ensemble de données 31
3.4.1 Sélection des environnements 32
3.4.2 Sélection des classes du modèle 34
3.4.3 Collecte de l'ensemble de données 36
3.4.4 Variantes de l'ensemble de données 43
3.5 Modèle de guidage autonome pour drone 45
3.5.1 Traitement d'image 47
3.5.2 Représentation des entrées scalaires 50
3.5.3 Traitement temporel des entrées 51
3.5.4 Interprétation de la classe 51
3.6 Fonction de coût 52
3.6.1 Pondération du contexte $\boldsymbol{wcontexte}$ 53
3.6.2 Coût de distance $\boldsymbol{Cd}$ 54
3.6.3 Coût d'orientation $\boldsymbol{Co}$ 54
3.6.4 Coût des fins de trajectoires $\boldsymbol{Cft}$ 55
3.7 Entraînement 56
3.7.1 Utilisation de l'ensemble de données 56

3.7.2 Augmentations de l'ensemble de données 58
3.7.3 Optimisation des hyperparamètres 58
CHAPITRE 4 RÉSULTATS ET DICUSSION 61
4.1 Résultats de l'entraînement 61
4.1.1 Analyse des tendances 61
4.1.2 Mitigation du décalage du modèle avec l'expert 66
4.1.3 Analyse des modèles obtenus 68
4.2 Résultats de simulation 70
4.2.1 Comparaison des modèles obtenus 72
4.2.2 Étude d'un modèle performant 77
4.2.3 Impact de l'ensemble de validation et de test 84
4.2.4 Nécessité du module récurrent 87
4.2.5 Étude de l'effet de la profondeur 89
4.3 Discussion et pistes de travaux futurs 91
CHAPITRE 5 CONCLUSION 95
RÉFÉRENCES 97
ANNEXES 108

LISTE DES TABLEAUX

Tableau 3.1 Modèles de prédiction de profondeur considérés pour le projet 24

Tableau 3.2 Caractéristiques recherchées pour la sélection du drone.....26

Tableau 3.3 Caractéristiques du modèle AR.Drone 2.0.....29

Tableau 3.4 Description des modèles 3D sélectionnés [123].....34

Tableau 3.5 Commandes utilisées pour la création de l'ensemble de données 36

Tableau 3.6 Éléments enregistrés lors de la création de l'ensemble de données 41

Tableau 3.7 Tableau récapitulatif des différentes itérations de l'ensemble de données DGD.....43

Tableau 3.8 Description des données finales constituant l'ensemble de donné DGD 44

Tableau 3.9 *Learning rates* des différents modules du modèle ADG.....56

Tableau 4.1 Statistiques sur les erreurs obtenues pour la recherche d'hyperparamètres.....62

Tableau 4.2 Précision de classification sur l'ensemble de validation.....69

Tableau 4.3 Précision de classification sur l'ensemble de test 69

Tableau 4.4 Performances moyennes par trajectoire par rapport à l'expert 72

Tableau 4.5 Erreur moyenne en mètres entre les agents autonomes et l'expert.....75

Tableau 4.6 Résumé des facteurs étudiés et leurs objectifs 77

Tableau 4.7 Erreur entre l'agent autonome et l'expert en mètres pour chaque trajectoire en fonction de la vitesse du drone 78

Tableau 4.8 Écart entre l'agent autonome et l'expert en mètres pour chaque trajectoire en fonction du scénario d'altitude 80

Tableau 4.9 Effet de la fréquence de rafraîchissement de la mémoire sur l'erreur entre l'agent autonome et l'expert en mètres pour chaque trajectoire 83

Tableau 4.10 Ensembles utilisés pour la pseudo-validation croisée à *k* blocs. 85

Tableau 4.11 Performances obtenues en fonction de l'ensemble de données de validation et de test 86

Tableau 4.12 Effet du module récurrent sur l'erreur relative entre l'agent autonome et l'expert en mètres pour chaque trajectoire 89

Tableau 4.13 Effet du type de vision sur l'erreur relative entre l'agent autonome et l'expert en mètres pour chaque trajectoire 90

LISTE DES FIGURES

Figure 2.1 Représentation schématique d'un «octomap» [34, Fig. 3 et 4] 10

Figure 2.2 Architecture multiéchelle proposée par [43, Fig. 2] 11

Figure 2.3 Image de profondeur discrétisée pour le calcul des vélocités [87, Fig. 2].....17

Figure 2.4 Guidage vers une image de référence [92, Fig. 1] 19

Figure 3.1 AR Drone 2.0 de la compagnie Parrot [124] 27

Figure 3.2 Diagramme de communication des nœuds ROS 30

Figure 3.3 Diagramme de communication de l'interface ROS et du programme externe 31

Figure 3.4 Vue isométrique (gauche) et intérieure (droite) du modèle 3D *Resto-Bar* [130] 33

Figure 3.5 Trajectoires typiques obtenues avec les trois commandes de base (gauche) versus avec les commandes combinées (droite) 35

Figure 3.6 Annotation des points d'intérêts dans l'environnement *Maison 2*.....37

Figure 3.7 Fenêtre de référence pour le guidage et la création de l'ensemble de données. Contient la carte de profondeur, la distance euclidienne et l'orientation entre la position actuelle et l'objectif. 38

Figure 3.8 Diagramme décisionnel pour l'expert 40

Figure 3.9 Quantités scalaires mesurées lors de la réalisation d'une trajectoire 42

Figure 3.10 Répartition des classes de l'ensemble de données DGD.....45

Figure 3.11 Architecture déroulée dans le temps du réseau de guidage autonome pour drones (ADG).....46

Figure 3.12 Vue détaillée du CNN REP1 de l'*Encodeur*. 48

Figure 3.13 Exemple de reconstruction des images de profondeur. 49

Figure 3.14 Architecture du réseau de guidage autonome pour drones (ADG).....50

Figure 3.15 Vue isométrique (gauche) et orthogonale supérieure (droite) du *Bar à Billard* [144]. 60

Figure 4.1 Courbes d'apprentissage de la recherche d'hyperparamètres. 63

Figure 4.2 Exemple de matrice de confusion typique 65

Figure 4.3 Tracé d'une trajectoire de l'expert comparée à celle de l'agent autonome 67

Figure 4.4 Comparaison et évaluation de la qualité des trajectoires (2 à 6) de 3 modèles avec l'expert 73

Figure 4.5 Comparaison des trajectoires 1 à 6 par l'expert (gauche) et le modèle 60% DA et 40% SM (droite) 74

Figure 4.6 Comparaison de la trajectoire 10 à 4 de l'expert (haut) et de l'agent autonome (bas) ..76

Figure 4.7 Comparaison des trajectoires 1 à 6 pour une vitesse de 0.3 m/s (gauche) et 0.5 m/s (droite) 79

Figure 4.8 Scénario encourageant l'agent à s'approcher dangereusement d'un obstacle. 81

Figure 4.9 Avant et après le cloisonnement complet des tables de billard. 81

Figure 4.10 Trajectoire typique lorsque le modèle est incertain. 82

Figure 4.11 Trajectoire (10 à 1) où le drone reste pris dans le coin en raison du non-rafraîchissement de la mémoire. 84

Figure 4.12 Exemples de trajectoires dans des environnements plus grands (bureau : droite) et plus complexes (salle de machine : gauche). 87

Figure 4.13 Exemple de trajectoire exécutée par le modèle dépourvu de module récurrent (trajet 4 à 10) 88

Figure 4.14 Comportements typiques du modèle RGB 91

Figure A.1 Vue intérieur d'une des pièces du *Bar à Billard* [144]. 108

Figure A.2 Vue isométrique (gauche) et orthogonale supérieure (droite) du *Bar à Billard* [144]. 108

Figure A.3 Numérotation des points d'intérêts du B*ar à Billard*.....109

Figure A.4 Vue isométrique du premier étage de la *Maison 1* [148]. 109

Figure A.5 Vue orthogonale du premier étage de la *Maison 1* [148]. 110

Figure A.6 Vue isométrique du deuxième étage de la *Maison 1* [148]. 110

Figure A.7 Vues intérieures du salon, d'un couloir et de la cuisine de la *Maison 1* [148]. 111

Figure A.8 Numérotation des points d'intérêt de la *Maison 1*, vue de côté en perspective. 112

Figure A.9 Numérotation des points d'intérêt de la *Maison 1*, vue orthogonale. 113

Figure A.10 Numérotation des points d'intérêt de la *Maison 1*, vue isométrique. 114

Figure A.11 Vue intérieure 1 de la *Salle de machines* 115

Figure A.12 Vue intérieure 2 de la *Salle de machines* 115

Figure A.13 Numérotation des points d'intérêt de la *Salle de machines.* 116

Figure A.14 Numérotation des points d'intérêt de la *Salle de serveurs.* 117

Figure A.15 Vue isométrique du *Bureau* [149]. 118

Figure A.16 Vues intérieure du *Bureau* 118

Figure A.17 Numérotation des points d'intérêt du *Bureau.* 119

Figure A.18 Vue isométrique du *Resto-Bar* [130]. 120

Figure A.19 Vue intérieure du *Resto-Bar* [130].. 120

Figure A.20 Numérotation des points d'intérêt du *Resto-Bar.* 121

Figure A.21 Vue orthogonale du premier étage de la *Maison 2* [150]. 122

Figure A.22 Vues intérieur du salon et de la cuisine de la *Maison 2* [150]. 122

Figure A.23 Numérotation des points d'intérêt de la *Maison 2*, vue avant. 123

Figure A.24 Numérotation des points d'intérêt de la *Maison 2*, vue orthogonale supérieure. 123

Figure A.25 Numérotation des points d'intérêt de la *Maison 2*, vue arrière. 124

LISTE DES SIGLES ET ABRÉVIATIONS

ADG	Autonomous Drone Guidance
DGD	Drone Guidance Dataset
IA	Intelligence Artificielle
i.i.d.	Identiquement indépendamment distribué
MLP	Multilayer Perceptron
CNN	Convolutional Neural Network
RNN	Recurrent Neural Networks
NLP	Natural Language Processing
GRU	Gated Recurrent Unit
LSTM	Long Short-Term Memory
MRF	Markov Random Field
MDP	Markov Decision Process
CRF	Conditional Random Fields
SLAM	Simultaneous Localisation and Mapping
VI	Value Iteration
GAN	Generative Adversarial Network
ICM	Intrinsic Curiosity Module
API	Application Programming Interface
SfM	Structure from Movements
SDK	Software Development Kit
ROS	Robot Operating System
FCRN	Fully Convolutional Residual Networks
DA	Destination Artificielle

SM	Séquence Miroir
KLD	Kullback Leibler Divergence
MACN	Memory Augmented Control Network
DARPA	Defense Advanced Research Project Agency
DAGGER	Dataset Aggregation
CoSIM	Conception de Systèmes Intelligents et Mécatroniques

LISTE DES ANNEXES

Annexe A : Caractéristiques des modèles 3D des environnements 108

CHAPITRE 1 INTRODUCTION

1.1 Problématique

Depuis quelques années, les drones de type quadricoptère ou multicoptère ont connu une maturation intense au niveau technologique qui les a rendus à la fois très populaires et accessibles au public. Leur utilité est maintenant très variée et comprend notamment des tâches de surveillance, de transport et aussi de recherche et sauvetage [1]–[3]. Certaines compagnies comme Amazon, ont même déjà commencé à utiliser des drones pour la livraison de colis à domicile [2]. En revanche, de ces nouvelles applications découlent aussi de nouveaux défis. L'un de ces défis provient du fait que les vols à basse altitude sont souvent synonymes d'un environnement non cartographié et dépourvu d'assistance GPS, notamment dans le cas de vols à l'intérieur ou dans des environnements urbains achalandés. Les drones doivent donc se baser sur de nouveaux outils pour naviguer dans ces nouveaux espaces.

Malgré la présence de plusieurs technologies dédiées à la localisation indépendante des GPS, incluant l'odométrie visuelle et la fusion sensorielle utilisant les filtres de Kalman [4], peu d'études s'attaquent au problème de guidage orienté en milieu inconnu. La possibilité d'utiliser une carte de l'environnement pour la planification de trajectoire et le guidage étant pour la plupart du temps non envisageable, il devient nécessaire de créer une perception de l'environnement sur laquelle se baser. Plusieurs études ont donc envisagé d'utiliser différents types de technologie tel que le LIDAR 3D ou les dispositifs de stéréovision afin d'acquérir une perception de la profondeur des environs. Le problème de ces technologies est cependant en lien avec la puissance de calcul sur les drones [5] ainsi que le coût. Il est donc devenu intéressant d'utiliser une seule caméra pour interpréter l'environnement de navigation [6].

L'intelligence artificielle ayant atteint un niveau de performance inégalé au cours des dernières années, elle est maintenant mieux adaptée que jamais à l'interprétation d'images [7]. La majorité des études s'intéressant au problème de guidage autonome se classent selon deux catégories : le guidage réactif localement pour exploration et le guidage orienté. Le guidage réactif localement permet aux engins de naviguer et d'éviter des obstacles en réagissant à l'environnement sans destination précise. Le guidage orienté reprend quant à lui les caractéristiques de guidage réactif mais, comme son nom l'indique, permet de se diriger vers un objectif quelconque. Les deux

catégories utilisent majoritairement l'apprentissage par imitation ou l'apprentissage par renforcement afin de résoudre leur problématique.

Même si les deux catégories n'utilisent pas de cartes de l'environnement à des fins de guidage, la deuxième catégorie nécessite une estimation de la position relative à son objectif. La source responsable de l'acquisition de la position peut varier parmi celles mentionnées plus haut ainsi que selon les domaines d'application. Dans le cadre de cette étude, l'estimation de la position peut tolérer une incertitude comparable à la taille de la zone de destination sans diminution de performances.

Considérant que la majorité des études sur le guidage autonome orienté en milieux inconnus s'applique à des robots terrestres et que bon nombre d'entre elles se limitent à des simulations de guidage dans des environnements simples et sur de courtes distances, concevoir un agent intelligent de guidage pour drones représenterait une contribution aux connaissances. Considérant aussi que cette étude constitue une preuve de concept à planification de trajectoire dans des environnements inconnus complexes, et qu'il devient très compliqué d'établir une fonction de récompense dans des problèmes complexes tels que celui-ci ou celui abordé dans [8], l'apprentissage par imitation est préférable pour résoudre le problème de guidage autonome. Ce type d'apprentissage permet d'ailleurs d'utiliser un humain comme expert, ce qui répond directement à la question de recherche en plus d'éliminer le besoin des ressources de calcul pour l'apprentissage par renforcement. L'utilisation d'un expert humain pour une tâche d'apprentissage par imitation permettra d'obtenir des démonstrations dans lesquelles l'expert effectuerait des trajectoires en explorant véritablement les environnements, contrairement à si l'expert humain était remplacé par un algorithme de planification optimal. Il serait en effet beaucoup plus complexe d'implémenter un système de guidage contrôlé par un algorithme de planification optimale, tel que A* ou RTT*, agissant comme un humain dans une tâche de guidage orienté en se basant sur la vision en plus d'une distance et orientation relative à sa destination. Dans des travaux futurs ayant pour objectif de surpasser les performances de cette preuve de concept, les avantages de l'apprentissage par renforcement pourraient être mis à profit en utilisant le modèle obtenu par apprentissage par imitation à des fins d'initialisation pour la procédure d'apprentissage par renforcement tel que dans [9].

1.2 Objectifs

L'objectif principal de ce projet de recherche est donc de :

Concevoir un agent intelligent capable d'imiter la logique de guidage d'un humain dans un environnement inconnu complexe.

La réussite de ce projet repose sur l'atteinte de trois sous objectifs :

SO1. Créer la logique de guidage nécessaire à la résolution du problème d'apprentissage par imitation.

SO2. Développer une architecture d'apprentissage profond capable d'extraire la logique de guidage.

SO3. Caractériser les modèles d'apprentissage profond obtenus dans un environnement de simulation virtuel.

Quatre hypothèses reliées à la méthodologie employée pour atteindre ces objectifs devront être vérifiées. La première hypothèse concerne l'aspect de la vision du problème et suggère que l'utilisation de la profondeur d'image permettra d'établir la logique plus facilement. L'indépendance des couleurs et des textures dans l'obtention d'une image de profondeur améliorera ainsi la capacité de généralisation à de nouveaux environnements. La seconde hypothèse suggère que les démonstrations d'un expert humain constituant l'ensemble de données sont suffisamment bien exécutées de façon à ce que la logique de guidage puisse être extraite. Ainsi, les images de profondeur de même que les données de position relatives à la destination à atteindre qui sont contenues dans les démonstrations seront nécessaires à l'extraction de la logique de guidage. La troisième hypothèse stipule donc que le modèle d'apprentissage par imitation sera en mesure d'extraire cette logique. Enfin, la quatrième hypothèse veut que les environnements virtuels choisis pour la génération de l'ensemble de données et des tests en simulation soient suffisamment riches en détail pour s'apparenter à un contexte hors simulation et permettre une caractérisation satisfaisante des performances de l'agent autonome.

1.3 Plan du mémoire

Une description détaillée des approches employées dans la littérature pour résoudre le problème de guidage autonome sera premièrement présentée dans la Revue de littérature (Chapitre 2). Suivra

ensuite le Chapitre 3, portant sur la méthodologie expliquant le processus qui permet d'atteindre les objectifs du projet. Chaque section de ce chapitre comportera une justification motivant les choix qui ont été faits. Les performances du modèle de guidage autonome pour drones (Autonomous Drone guidance, ADG) seront ensuite présentées et analysées au Chapitre 4, allant de l'analyse des performances de l'entraînement du modèle jusqu'aux performances obtenues en simulation. Ce chapitre fera également état des limitations et des améliorations potentielles à apporter au modèle via une discussion des résultats. Enfin, le Chapitre 5 comprendra un retour sur les principaux résultats avant de conclure le mémoire.

CHAPITRE 2 REVUE DE LITTÉRATURE

2.1 Notions d'apprentissage profond

L'intelligence artificielle (IA) est parmi les disciplines ayant le plus évolué au cours des dernières années. Bien que l'apprentissage machine soit connu et utilisé depuis les années 50, l'apprentissage profond fait quant à lui son arrivée dans les années 80. Les algorithmes d'apprentissage profond sont développés en profondeur dans ces années, mais ce n'est que tout récemment que ce type d'algorithmes ont vu leur popularité augmenter. La période suivant leur découverte et leur gain en popularité est référée comme étant l'hiver de l'IA, une période durant laquelle l'intérêt envers cette science a été moins marqué en raison d'un découragement face aux performances de l'IA [10]. Suivant la loi de Moore, la puissance de calcul des processeurs doubla environ chaque deux ans jusqu'à aujourd'hui, accordant ainsi une plus grande capacité de calculs pour la résolution des problèmes d'IA [11]. L es plateformes d'apprentissage profond facilitant l'implémentation de ces algorithmes ont finalement vu le jour [12]–[16], rendant plus accessible son utilisation.

2.1.1 Méthodes d'apprentissage

La plupart des algorithmes d'intelligence artificielle peuvent généralement être catégorisés comme étant supervisés ou non supervisés. Les deux types d'algorithmes effectuent leur apprentissage à partir d'un ensemble de données. Dans le contexte de l'apprentissage supervisé, l'algorithme aura comme objectif de prédire une sortie $\boldsymbol{y}$ en fonction d'une entrée $\boldsymbol{x}$. L'ensemble de données est dans ce cas constitué de paires d'exemples de la forme $(\boldsymbol{x}, \boldsymbol{y})$. Pour ce qui est de l'apprentissage non supervisé, l'algorithme peut avoir plusieurs types d'objectifs, soit la prédiction de densités à des fins de synthèse ou de suppression de bruit («denoising»), l'agrégation de caractéristiques communes, etc. L'ensemble de données pour ce type de problème est constitué d'exemples de la forme $\boldsymbol{x}$ seulement. La plupart des algorithmes abordés dans le contexte de ce mémoire utilisent l'apprentissage supervisé pour des tâches de classification ou de régression [17]. Des ensembles de données pour la classification et la régression sont maintenant souvent disponibles en ligne pour permettre la comparaison des nouveaux algorithmes [18], [19].

2.1.1.1 Apprentissage par renforcement

L'apprentissage par renforcement fait exception aux deux catégories mentionnées ci-dessus. Contrairement aux deux approches, l'apprentissage par renforcement ne repose pas sur l'utilisation d'un ensemble de données obtenu au préalable. Ces algorithmes, appelés agents ou machines, sont libres d'interagir avec leur environnement et ont pour objectif de trouver un comportement désiré maximisant les récompenses à long terme. Les récompenses sont attribuées à chaque action grâce à un système de récompense préétabli [20].

Ces algorithmes s'entraînent généralement en simulation pour des fins pratiques, mais peuvent être déployés sur de véritables plateformes robotiques [21], ce qui en fait une avenue intéressante d'apprentissage. Ils ont d'ailleurs eu beaucoup de succès en 2013 après avoir vaincu un joueur professionnel sur plusieurs jeux de la célèbre console Atari 2600 [22].

Ce type de solutions peut cependant être limité par les ressources informatiques disponibles pour la simulation et est reconnu pour être sujet au problème de récompense intermittent («sparse reward problem»). Ce problème nuit à la convergence de ces modèles et survient lorsque la fonction de récompense est mal adaptée au problème [23].

2.1.1.2 Apprentissage par imitation

L'apprentissage par imitation fait quant à lui partie des méthodes supervisées. Ces algorithmes tentent d'imiter un expert en apprenant une logique de décision extraite d'une large quantité de démonstrations provenant de ce dernier [24]. Cette caractéristique particulière évite le besoin de manuellement développer une fonction de récompense satisfaisant le comportement recherché comme c'est le cas avec l'apprentissage par renforcement. La résolution du problème d'imitation repose sur la construction d'un ensemble de données, spécifique à la tâche, par un expert. La collecte de données peut se faire de façon expérimentale ou par simulation. Dans le second cas, il est possible de s'assurer que l'expert agit de façon optimale. Dans le cas du guidage, il est par exemple possible d'automatiser l'expert pour qu'il suive une trajectoire optimisée par une approche déterministe. Dans les deux scénarios, il est reconnu que la logique apprise par l'agent intelligent sera au mieux quasi-optimale [25].

Autre que l'aspect fastidieux de la collecte de données d'entraînement, les agents entraînés par imitation ne respectent typiquement pas l'hypothèse selon laquelle les éléments de l'ensemble de

données sont identiquement indépendamment distribués (i.i.d.). Le non-respect de cette hypothèse statistique peut mener à de mauvaises performances au moment de tester le modèle puisque la plupart des observations dépendent des prédictions/observations antérieures [24]. Il a été démontré que les techniques d'agrégation d'ensembles de données («Dataset Aggregation» ou DAGGER) améliorent considérablement les performances de ce type d'apprentissage [25].

Bien que la solution mentionnée ne rétablisse pas le respect de l'hypothèse i.i.d., l'amélioration est due au fait que ces techniques d'entraînement ralentissent le rythme avec lequel la politique d'action est modifiée [26]. Elles permettent d'évaluer même au début de l'entraînement le modèle en simulation et de fournir une rétroaction sur les actions posées en ajoutant les scénarios rencontrés à l'ensemble de données initial. La politique d'action finale du modèle aura donc la particularité d'être robuste à une plus grande quantité de scénarios différents, tels que des erreurs de parcours non rencontrées par l'expert. Une approche similaire pour minimiser le rythme de modification de la politique de guidage est proposée dans [27]. L'utilisation de réseaux de neurones récurrents pourrait potentiellement aussi augmenter les performances puisque ces modèles capturent l'aspect temporel liant les observations et les prédictions [28].

Pour terminer, e travail présenté par Bansal et al. (2018) prouve qu'il est possible d'entraîner un agent performant pour la conduite autonome lorsqu'une attention particulière est portée à la causalité des exemples et de la différence de distribution statistique de l'ensemble de données [29], [30].

2.1.2 Types de réseaux communs

Il existe une multitude de réseaux de neurones qui ont chacun leurs particularités. Ces réseaux peuvent être jumelés ensemble afin de construire un réseau complexe ayant plusieurs fonctionnalités. Les prochaines sous-sections analyseront les types les plus communs, soit les perceptrons multicouches, les réseaux convolutifs et les réseaux récurrents.

2.1.2.1 Perceptron multicouches

Le perceptron est l'unité de base d'une multitude de réseaux de neurones. Il s'agit d'une fonction permettant de transformer un vecteur d'entrée en une valeur de sortie ayant une valeur concise par une fonction d'activation quelconque [17], [31], [32]. La fonction peut être représentée par l'équation (2.1).

$$f(\boldsymbol{x}) = \sigma(\boldsymbol{w^T x} + \boldsymbol{b}) \quad (2.1)$$

$\boldsymbol{x}$: Vecteur d'entrée du perception

$\boldsymbol{w^T}$: Vecteur de pondération de l'entrée

$\boldsymbol{b}$: Vecteur de biais (ordonnée à l'origine)

σ : Fonction d'activation

Plusieurs perceptrons peuvent être employés sur un même vecteur d'entrée. Cet arrangement sera appelé perceptron multicouches («Multilayer Perceptron» ou MLP). Une succession de plusieurs couches connectées ensemble donne alors naissance à réseau d'IA profond dense souvent appelé «fully connected network» en anglais. Dans les problèmes de classification, ces réseaux sont généralement utilisés à la toute fin comme classificateurs («classifier») pour transformer une représentation interne de l'entrée initiale en classe.

2.1.2.2 Réseaux convolutifs

Les réseaux de neurones convolutifs (CNN) sont désormais l'outil de choix pour toute tâche impliquant la vision par ordinateur. Ce type de réseaux se spécialise dans l'exploitation de la topologie des entrées (1D pour les entrées temporelles et 2D pour les images) qui ne serait pas possible à l'aide d'un MLP. L'opération de convolution permet cet exploit et assure aussi une réduction significative du nombre de paramètres à apprendre par le réseau en ayant un seul ensemble de paramètres par filtre de convolution. Les filtres convolutifs sont appliqués sur toute l'entièreté de l'entrée et assurent dans le cas d'une image, qu'une caractéristique peut être capturée, peu importe où elle se trouve dans l'image. Des couches de «pooling» sont également employées dans les CNN et permettent davantage la diminution de paramètres tout en rendant le réseau invariant à l'échelle des caractéristiques recherchées [17]. Ces réseaux sont en constante évolution et ont même récemment bénéficié de nouvelles techniques de régularisation employant la fonction de Green, ce qui leur permet d'apprendre plus rapidement tout en améliorant la capacité de généralisation [33]. Leur utilisation s'est notamment avérée utile dans des tâches de détection de saillance visuelle («saliency map») [34], [35].

Les CNN ont révolutionné la capacité à détecter et à classifier des objets présents dans une image. Il est désormais possible d'utiliser ces outils pour détecter pas moins de mille objets différents de la vie quotidienne [36], [37], ce qui est un excellent début dans l'automatisation de tâches robotisées. Leur potentiel saura sans doute contribuer à la réalisation de ce projet.

2.1.2.3 Réseaux de neurones récurrents

Les réseaux de neurones récurrents (RNN) sont une famille de réseaux d'apprentissage profond se spécialisant dans le traitement d'entrées sous forme séquentielle tels que les mots d'une phrase, des notes de musique ou toute autre séquence dont les éléments ont une dépendance chronologique. La particularité des RNN est qu'ils apprennent à sélectionner et à retenir ou oublier de l'information temporelle leur permettant d'accomplir leur tâche. Le mécanisme responsable de cette particularité se base, dans le cas des réseaux les plus performants, sur l'utilisation de portes dédiées à un comportement (retenir ou oublier par exemple) qui sont gérées par des paramètres appris lors de la phase d'entraînement [8][25]. Le réseau récurrent à mémoire court et long terme («Long Short-Term Memory» ou LSTM) est un excellent exemple de ce type de réseaux.

Les RNN sont utiles dans plusieurs domaines tels que celui du traitement automatique du langage naturel («Natural Language Processing» ou NLP) [39] mais aussi dans le domaine du guidage autonome [40].

2.2 Estimation de profondeur

L'estimation de la profondeur dans une image est un sujet de recherche en constante évolution. Comme l'hypothèse de ce projet de maîtrise se fonde sur l'utilisation d'un flux d'image de profondeur, il est impératif de bien analyser et comprendre les différentes options accessibles dans la littérature. Il existe des solutions telles que celle employée dans l'étude de préhension de [41], permettant d'obtenir de façon rapide et efficace un aperçu de la profondeur spatiale. Cependant, ces solutions impliquent souvent l'utilisation de lasers coûteux et ne fournissent qu'une représentation clairsemée de la profondeur sous la forme d'un nuage de points 3D ou d'une image 2.5D à basse résolution. Ces points 3D peuvent également servir à la création d'«octomaps», une représentation multirésolution des points par des cubes d'occupation [42] telle que représentée à la Figure 2.1.

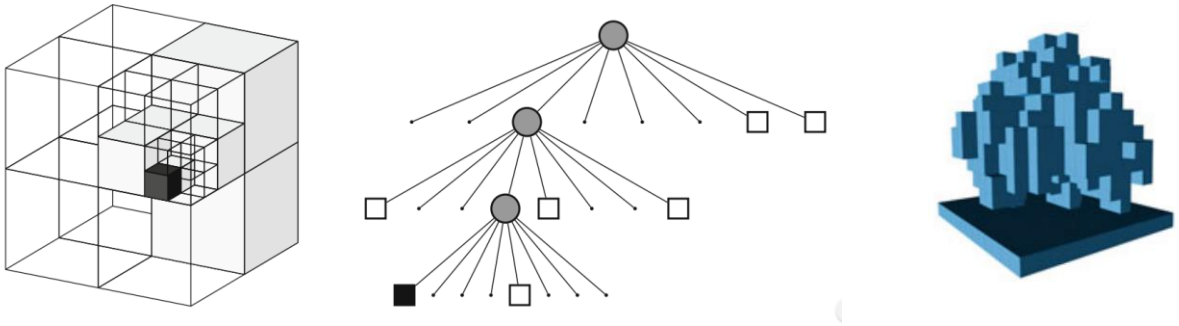


Figure 2.1 Représentation schématique d'un «octomap» [34, Fig. 3 et 4]

De telles représentations nécessitent souvent plusieurs mesures avant de générer un «octree» précis et représentatif de l'environnement. C'est pourquoi les méthodes utilisant la vision stéréoscopique peuvent s'avérer intéressantes. La vision stéréoscopique utilise deux caméras afin de calculer la disparité entre deux images et enfin exprimer la profondeur. Le processus ne nécessite pas de capteurs lasers dispendieux et permet de calculer la profondeur immédiatement à partir d'une image stéréo.

Il existe deux types de calculs d'image de profondeur pour les algorithmes utilisant la vision stéréoscopique : le calcul dispersé, s'intéressant aux segments et aux contours, ainsi que les calculs denses, utilisant la totalité des pixels [43]. Le calcul dispersé générera donc généralement une image de profondeur constituée des contours ou de régions alors que l'approche dense générera une image complète [4], [5]. Même si la première méthode est habituellement plus rapide, les deux procèdent initialement à un calcul de concordance suivi d'une agrégation des concordances en régions et du calcul de disparité. Certains algorithmes incluent une phase de raffinement à la fin du calcul. Il existe aussi deux types de méthodes denses : les méthodes locales basées sur des régions [46] ainsi que les méthodes globales, basées sur l'énergie [45].

L'utilisation de l'apprentissage profond a également pris place parmi les techniques de prédiction de la profondeur [47]. Le besoin de deux caméras est en revanche en soi un désavantage pour des raisons de coût et de poids engendrés par le dédoublement des appareils. De surcroît, puisque la majorité des drones commerciaux sont équipés d'une seule caméra, il sera dans le cadre de ce projet plus simple de choisir l'un de ces drones et utiliser un algorithme pour système monoculaire. Les algorithmes d'estimation pour systèmes monoculaires sont de plus en plus précis grâce aux avancées en intelligence artificielle [48]. Quelques méthodes ont auparavant réussi à extraire une

perception de profondeur en mettant à profit des techniques classiques. Citons, entre autres, la méthode de Michels et al. (2005) qui utilise les contours, les transformées de Radon et le théorème de Law sur l'énergie pour enfin effectuer une régression linéaire pouvant lier ces caractéristiques avec la profondeur observable à l'horizon [49]. Ou encore l'approche qui détecte des surfaces via un champ aléatoire de Markov («Markov Random Field» ou MRF) appliquée à de super pixels de [50]. Ces méthodes classiques ont cependant rapidement été surpassées par les méthodes utilisant l'apprentissage profond.

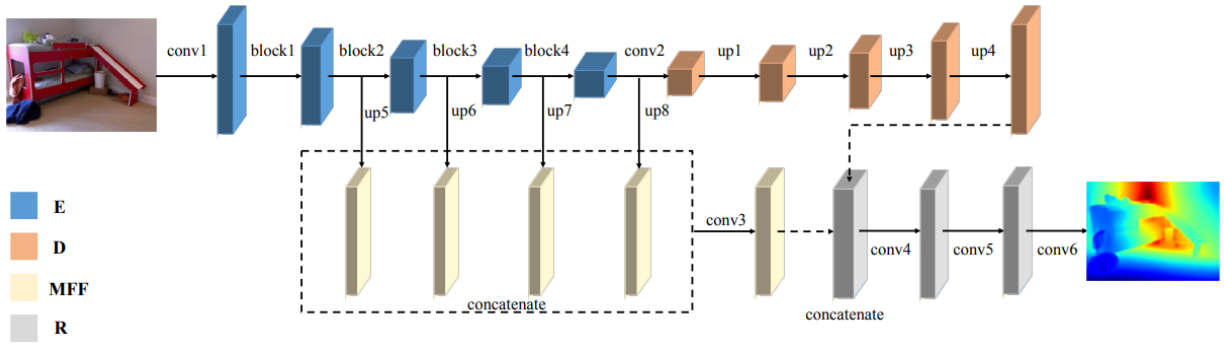


Figure 2.2 Architecture multiéchelle proposée par [43, Fig. 2]

L'une des branches de la prédiction de la profondeur monoculaire s'intéresse aux réseaux multiéchelles, par opposition aux réseaux convolutionnels simples comme dans [52]. Ces architectures constituées de CNNs souvent basées sur les modèles classiques comme ResNet [53], auxquels on a ajouté des sections de convolutions puisant à différents niveaux la représentation obtenue par convolutions tel qu'illustré à la Figure 2.2 [39][40]. Une autre branche se démarque par l'utilisation de champs aléatoires conditionnels («Conditional Random Fields» ou CRF) pour capturer l'information dans les pixels avoisinants de façon continue [41][42]. Cette dernière s'apparente à l'approche adoptée dans [50] puisqu'elle utilise une variante des MRF. D'autres auteurs dans [57] ont combiné l'approche CRF en incorporant des modules d'attention à différentes échelles de leur architecture de convolution qui régulent l'information transférée par chaque échelle de convolution. Les auteurs dans [58] ont pour leur part considéré le problème comme un problème de régression ordinale et ont utilisé des convolutions dilatées afin de discrétiser la profondeur à plusieurs échelles. L'apprentissage non supervisé peut aussi être employé pour la résolution de ce problème comme démontré par [59] dans leur travail sur une architecture et une fonction de coût optimisant la reconstruction d'images de profondeur. Plutôt que de calculer une seule matrice de

disparité pour les deux images obtenues par stéréovision, deux matrices de disparité sont calculées à partir d'une image et sont combinées avec les deux images originales afin de produire une image de profondeur. L'algorithme est donc en mesure en période de test de générer une image de profondeur à partir d'une seule image monoculaire.

Enfin, une dernière catégorie de prédiction de profondeur à partir d'images exploite le mouvement dans une séquence vidéo. Ces méthodes comme dans [60]–[62] utilisent un flux d'image monoculaire afin de prédire la profondeur de l'image ainsi que le mouvement de la caméra. Les auteurs dans [62] iront jusqu'à intégrer un module estimant le flux optique avec une cote de confiance y étant relié afin de mieux prédire le mouvement de la caméra et de la profondeur d'image à l'intérieur d'un processus itératif. Cette approche peut cependant poser un problème dans une application en temps réel en raison de sa nature itérative. L'apprentissage non supervisé est également mis à profit par Zhou et al. (2017) qui prédit aussi la profondeur de l'image ainsi que le mouvement de la caméra [63]. L'utilisation du mouvement de la caméra est d'ailleurs enviable pour une utilisation telle que le guidage. Le mouvement pourrait techniquement remplacer le besoin d'un signal GPS par exemple, où le positionnement s'effectuerait par odométrie visuelle tel que dans [64].

Comme mentionné précédemment, une solution reposant sur le besoin d'une seule caméra sera préférée à une approche nécessitant de l'équipement laser ou de plusieurs caméras. La solution qui sera retenue devra être en mesure de fournir une image de profondeur en temps réel de qualité suffisante pour permettre à un expert de naviguer dans un environnement inconnu en s'y référant. Elle devra également être facilement implémentable ou être disponible pour utilisation.

2.3 Guidage autonome

Le guidage autonome de robots intelligents a longtemps été un sujet d'intérêt dans la communauté scientifique. Le fruit de ces recherches se base principalement sur des techniques de vision par ordinateur. Alors que de nombreuses études antérieures utilisent des techniques conventionnelles d'extraction de formes et de contours pour le guidage d'une voiture autonome [2] [3], d'autres avaient dès la fin des années 1980 déjà adopté l'utilisation de réseaux de neurones dans un modèle perception-action pour prédire la rotation du volant appropriée à partir d'images [8]. Bon nombre d'études se sont ensuite intéressées au problème de guidage de drones dans un environnement

connu ou partiellement connu. Des méthodes utilisant des algorithmes de planification de trajectoire assistés de modules de planification locale pour éviter les obstacles ont ainsi vu le jour comme dans [67]. Le guidage autonome pour drone repose également souvent sur la disponibilité d'un signal GPS [68]. Réalisant que la disponibilité d'une carte de l'environnement ou d'un signal GPS n'est pas toujours disponible, d'autres travaux comme [69]–[71] exploitent les algorithmes de localisation et cartographie simultanées («Simultaneous Localisation And Mapping» ou SLAM) pour générer des cartes locales pouvant servir à planifier une trajectoire utilisant des algorithmes traditionnels.

Comme les algorithmes de cartographie SLAM sont coûteux en termes de calculs [72], ils créent un délai significatif entre la perception du robot et l'action à effectuer. Il serait aussi ultimement souhaitable que ces robots aériens soient indépendants d'une connexion à une station calcul. La communauté scientifique s'est donc intéressée aux solutions sans cartographie par SLAM. Une des solutions alternatives inclut l'utilisation de modèles de type perception-action tel que mentionné plus haut. Ces modèles se fondent souvent sur le Processus de Décision Markovien (MDP) [73], [74], qui établit la relation entre divers éléments, dont les états d'un système, les actions possibles sur ce système, la probabilité de transition d'état et la fonction de coût (récompense). Ce théorème s'applique lorsque les états du système répondent à la propriété de Markov voulant que chaque état du système ne soit dépendant que de l'état précédent. Dans plusieurs contextes requérant plus d'informations précédentes tels que le guidage, il est possible d'utiliser un réseau de neurones récurrent afin de représenter l'état du système actuel [28].

La section de la revue de littérature qui suit fait état des deux principaux types de catégories de solutions de guidage autonome connexes au problème abordé dans ce mémoire. Elle traitera des solutions de guidage à des fins d'exploration non structurée, suivi des solutions de guidage utilisant un planificateur global dépourvu de carte ou de signal GPS.

2.3.1 Exploration non structurée

L'exploration non structurée réfère ici à la planification à court terme des actions à effectuer à des fins d'exploration. Ce type de guidage permet aux robots qui l'utilisent de se déplacer librement en respectant certaines contraintes telles que le suivi des lignes sur les routes, ou encore le suivi d'un sentier pédestre en forêt, tout en évitant d'entrer en collision avec les obstacles environnants. Les algorithmes de planification à court terme qui sont utiles à ce projet de recherche se classent

majoritairement parmi les techniques utilisant l'apprentissage par imitation, l'apprentissage par renforcement ou encore d'autres techniques variées.

2.3.1.1 Apprentissage par imitation

Les techniques d'apprentissage par imitation appliquées au problème de guidage basé sur la vision comme ALVINN [8], une voiture autonome, ont été utilisées il y a plusieurs années. Ceci s'explique que malgré le fait d'être laborieux, la tâche de recueillir des données en grande quantité pour la création d'un ensemble de données d'entraînement est souvent possible dans le cadre du guidage automobile. Il s'agit simplement dans le cas de ALVINN, d'associer chaque image à une rotation du volant. D'autres travaux introduiront de nouveaux éléments à l'ensemble de données afin de pouvoir ajouter une prédiction pour la vitesse de guidage basée sur la probabilité de collision comme dans [75].

Un bon nombre d'études sur les voitures autonomes se concentrent cependant seulement sur la prédiction de l'angle de rotation du volant. Tel est le cas avec [76] où l'approche envisagée est de générer un ensemble de données constitué de trois prises de vue de la chaussée : latérale gauche, centrée, latérale droite. Ces prises de vue sont décalées de quelques degrés par rapport à la prise de vue centrale, mais ont le même horizon de façon à simuler une mauvaise orientation de la direction du véhicule. Les prises de vue latérales sont ainsi jumelées à des commandes d'angle de rotation du volant qui corrigeraient l'orientation du véhicule, générant ainsi un modèle robuste aux erreurs de guidage. Van et al. (2018) ont adopté une technique différente pour le guidage autonome robuste utilisant un module de logique floue incorporant la prédiction de l'angle du volant et la vitesse précédente du véhicule pour déterminer un nouvel angle de rotation et une vitesse appropriée pour cette rotation [77].

L'apprentissage par imitation est aussi utilisé pour les problèmes de guidage autonome aériens. Loquercio et al. (2018) de l'Université ETH Zurich ont développé DroNet [75], un modèle de prédiction de l'angle de rotation en lacet et de la probabilité de collision avec l'environnement. La partie du modèle de prédiction de l'angle de rotation est entraînée avec le populaire ensemble de données Udacity [16] [17], constitué de séquences d'images prises par une voiture équipée de plusieurs détecteurs alors que la partie de prédiction de la probabilité de collision se base sur un ensemble de données sur mesure constitué de segments vidéos pris à vélo où le conducteur s'approche d'un obstacle de façon linéaire à chaque séquence. La probabilité d'entrer en collision

avec un obstacle est donc répartie linéairement tout au long de cette séquence. Ce projet novateur aura permis de prouver que le guidage de drones autonomes peut se faire en imitant le comportement de voitures ou de vélos. Ceci encouragera [80] à implémenter une version embarquée de DroNet dans le drone Crazyflie développé par Bitcraze [81].

Le besoin de générer un ensemble de données sur mesure s'est également fait ressentir rapidement alors que [82] et [83] entreprennent le guidage autonome en forêt. Le travail subséquent pousse encore plus loin l'élaboration du modèle final en ajoutant un modèle de classification détectant une variété d'obstacles incluant des personnes ainsi qu'un léger modèle de perception de profondeur. Les travaux dans [22] et [23] entreprennent ensuite le guidage à l'intérieur nécessitant encore la création d'un ensemble de données sur mesure. Kouris et al. (2018) dans [85] décident pour leur part de mettre en place un système de cueillette de données autonome établissant des mesures de profondeur laser dans plusieurs directions pour chaque image. Ils démontrent que les modèles de régression performent mieux que les modèles de classification dans leur problème de guidage utilisant le AR Drone 2.0 développé par Parrot [86].

Les algorithmes d'apprentissage par imitation sont comme mentionné dans la section 2.1.1.2 susceptibles de violer l'hypothèse selon laquelle les exemples sont i.i.d.. L'application de structure d'apprentissage DAGGER a donc été implémentée avec succès dans [82] et [87]. Xu et al. (2019) proposent en surplus une méthode permettant de classifier au fur et à mesure de nouveaux scénarios inconnus automatiquement sans supervision humaine [87]. L'approche dans [76] consiste également à une solution simplifiée pour ce problème.

Très peu des approches présentées jusqu'à maintenant exploitent le potentiel de l'utilisation d'images de profondeur par opposition aux images RGB traditionnelles. Seule l'approche de Smolyanskiy et al. (2017) propose une approximation clairsemée de la profondeur [83]. Il est pourtant connu que la perception de la profondeur chez les êtres vivants est associée à la conscience de par sa capacité à établir la relation entre l'espace et les mouvements [88]. Une approche intéressante utilisant une prédiction de la profondeur et une prédiction des normales des surfaces sous forme de deux images a été employée dans l'ouvrage de Yang et al. (2017) [89]. Leur méthode combine les deux images dans un réseau de convolution profond afin de résoudre un problème de classification. Les cinq classes établies pour ce problème de guidage par imitation prennent la forme des directions suivantes : gauche, avant-gauche, avant, avant-droite et droite. Ces classes

sont suffisamment raffinées pour permettre le guidage d'un quadcoptère dans un environnement intérieur virtuel et expérimental.

2.3.1.2 Apprentissage par renforcement

L'apprentissage par renforcement a aussi été utilisé pour résoudre le problème de guidage autonome. À condition de résoudre le problème de récompense intermittente ainsi que le problème de généralisation dans des environnements hors simulation [23], il devient possible de réaliser un modèle de guidage intelligent pour robot tel que dans [90]. Greatwood et al. (2010) ont réalisé un modèle basé sur la commande prédictive [91] permettant de sélectionner grâce à l'apprentissage par renforcement vers quel nœud avoisinant il faut se déplacer pour explorer un environnement préalablement virtuellement quadrillé. Il devient ainsi possible de facilement rebrousser chemin lors que le drone détecte qu'il est dans un cul-de-sac. Kahn et al. (2018) ont dans [92] procédé à l'implémentation d'un algorithme d'apprentissage par renforcement qui a la capacité de s'améliorer dans le temps une fois déployé. La méthode se base sur l'itération sur valeur (VI) sans modèle afin de générer un graphe de calcul général pouvant apprendre à partir d'images. Cependant, l'algorithme peut prendre jusqu'à plusieurs heures afin de réussir à naviguer de façon autonome dans un nouvel environnement. Ce type d'apprentissage est de façon générale beaucoup moins rapide à entraîner.

2.3.1.3 Autres approches

D'autres approches n'utilisant pas l'apprentissage profond ont aussi été testées pour le guidage autonome. La vision par ordinateur d'aujourd'hui n'étant pas ce qu'elle était dans les années 90, plusieurs ont exploré l'utilisation de méthodes classiques pour extraire des caractéristiques essentielles d'images prises par une caméra déployée sur un véhicule. Franke et al. (1998) ont utilisé un nouvel algorithme utilisant les moindres carrés ainsi que des filtres de Kalman afin de minimiser la possibilité de se trouver à l'extérieur du chemin [66]. Schneiderman et al. (1994) ont quant à eux décidé d'utiliser une technique permettant de superposer des lignes imaginaires sur la chaussée afin de remédier aux cas où celles-ci ne seraient pas visibles, permettant d'utiliser plus robustement une technique de guidage pour le suivi des lignes [65]. Une autre technique intéressante proposée par Ghosh et al. (2016) a été d'entraîner un réseau antagoniste génératif («Generative Adversarial Network» ou GAN) afin de générer une image représentant le résultat de chaque option de commande afin de classifier ces images et donc indirectement, choisir la

commande qui est la plus appropriée [93]. Bien qu'attrayante, cette solution est fondamentalement similaire à l'utilisation de techniques par imitation, mais ajoute la complexité reliée à l'utilisation d'un GAN. Enfin, Chakravarty et al. (2017) ont récupéré l'idée d'utiliser une perception de la profondeur pour entraîner un CNN à produire une image de profondeur et discrétiser cette image (voir Figure 2.3), en plusieurs éléments sur l'axe horizontal (Ψ_{horz}: Orientation horizontale (rad) , d_{horz} : Profondeur horizontale (m)) et vertical (Ψ_{vert}: Orientation verticale (rad), d_{vert} : Profondeur verticale (m)) [94]. Les profondeurs de l'axe horizontal sont utilisées pour le calcul d'une vitesse angulaire en lacet selon une formule de pondération sur mesure.

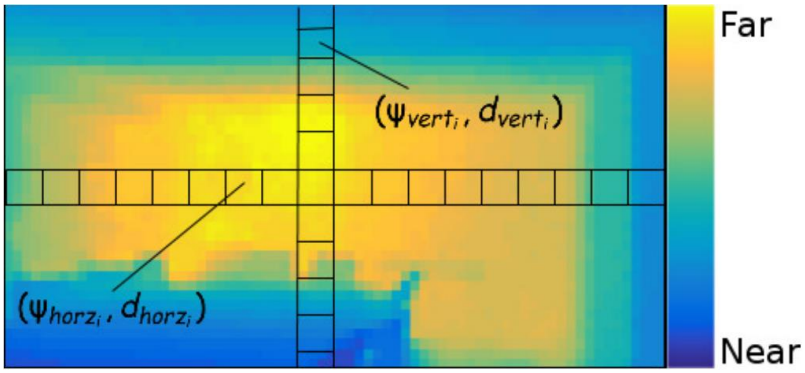


Figure 2.3 Image de profondeur discrétisée pour le calcul des vélocités [87, Fig. 2]

2.3.2 Planification de trajectoire globale

La planification de trajectoire globale réfère ici à la planification des actions à effectuer pour atteindre un objectif défini par des coordonnées cartésiennes sans l'utilisation de carte. Ce type de guidage permet aux robots qui l'utilisent de naviguer librement en respectant une multitude de contraintes locales telles que l'évitement d'obstacles et le suivi de marqueurs tout en gardant en mémoire l'objectif de destination finale. Les algorithmes qui sont utiles à ce projet de recherche se classent aussi majoritairement parmi les techniques utilisant l'apprentissage par imitation, l'apprentissage par renforcement.

2.3.2.1 Apprentissage par imitation

Des concepts utilisant l'apprentissage profond par imitation à des fins de guidage orienté ont aussi vu le jour il y a une dizaine d'années lorsque Farooq et al. (2010) ont entrepris d'automatiser un véhicule miniature [95]. Leur approche initiale consistait à entraîner un réseau de neurones simple utilisant plusieurs couches de perceptrons afin de prédire la direction à emprunter pour éviter un obstacle. Un algorithme procédural a été mis en place pour pondérer la direction fournie par le réseau de neurones avec la direction optimale à emprunter pour atteindre l'objectif. Un travail subséquent fourni par le même auteur remplace l'algorithme procédural avec un nouveau réseau de neurones qui est encore une fois entraîné par imitation basé sur des démonstrations effectuées avec le vrai prototype [96]. Pfeiffer et al. (2017) ont par la suite proposé de combiner ces réseaux ensemble et en utilisant une grille de profondeur obtenue par laser [97]. La grille de profondeur est ensuite traitée comme une image dans un CNN afin de combiner la représentation de cette grille avec l'information de l'objectif. Le tout est traité par un réseau de neurones traditionnel et transformé en commandes à envoyer à un robot. Toujours dans l'optique de l'apprentissage par imitation, Chen et al. (2017) ont exploité l'utilisation du concept de mémoire temporelle, un aspect inexploré de cette technique appliquée au guidage [40]. Ils proposent initialement d'ajouter un réseau récurrent LSTM et vont même jusqu'à tester l'impact de l'ajout d'une matrice de mémoire externe. Cet ajout s'avère efficace pour les obstacles de types cul-de-sac de grande profondeur puisqu'il permet d'augmenter la quantité d'information contenue dans le LSTM.

Bansal et al. (2019) ont pour leur part abordé le problème différemment en employant module de perception similaire à celui dans [97] qui prédit une balise pour la prochaine destination intermédiaire sous la forme de coordonnées relatives au robot. Un module de planification de trajectoire optimale est ensuite utilisé pour atteindre la prochaine balise [98]. Ce travail a l'avantage de générer des trajectoires plus raffinées entre les balises et ce, jusqu'à la destination finale. Le modèle apprend à générer les balises à l'aide de démonstrations reliant chaque image à une balise le long d'une trajectoire optimale entre une paire constituée d'un point de départ et d'un point d'arrivée.

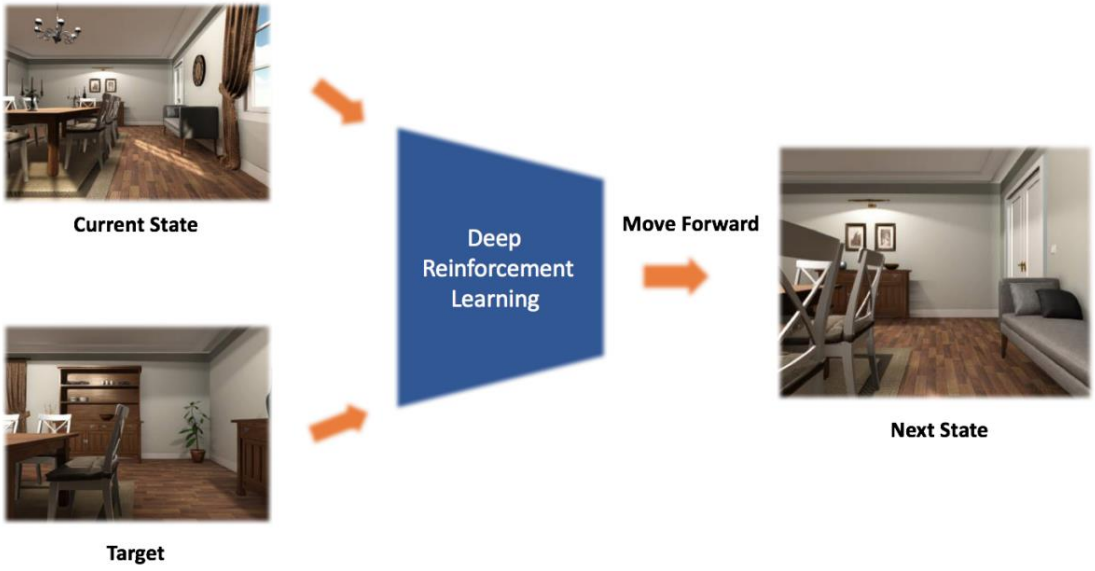


Figure 2.4 Guidage vers une image de référence [92, Fig. 1]

D'autres travaux comme celui de Kim et Jeong [99] entreprennent la tâche de naviguer de façon autonome vers une destination représentée par une image de référence, voir Figure 2.4. Bien que cette tâche se veuille utile dans certains contextes, la stratégie employée requiert un entraînement dans chacun des milieux de déploiement, ce qui constitue une lacune concernant la capacité de généralisation de ce genre d'approche. Hussein et al. (2018) ont eux aussi considéré le problème de guidage basé sur une image de référence à atteindre. Leur méthodologie inclut en revanche une composante active de la méthode DAGGER [100]. L'apprentissage actif se poursuit après l'obtention d'une politique de guidage de base obtenue par imitation et s'exécute automatiquement à l'aide d'un score de confiance attribué à chaque commande prédite par le modèle. Les instances ayant une faible confiance sont souvent reliées à des états non rencontrés pendant l'entraînement. Ils sont donc enregistrés dans l'ensemble de données avec l'action qui a été posée et permettront de réentraîner le modèle en tenant compte de ces séquences. Leur modèle est entraîné pour plusieurs tâches telles que le suivi de lignes au sol, la recherche d'un drapeau ou d'un objet et le suivi de disques flottants. Les résultats sont excellents comparativement aux performances des algorithmes traditionnels entraînés pour les mêmes tâches. Les auteurs mentionnent également le gain en performance pouvant être atteint en sautant à un pas régulier quelques images («frame skipping») dans l'ensemble de données durant l'entraînement.

Similairement à [98], Gupta et al. (2017) utilisent des balises pour le guidage vers un objectif [101]. Les balises sont cependant créées suite à la génération d'une carte locale des environs du robot. La carte est créée à partir d'une multitude de points de vue permettant la création d'une trajectoire qui est ensuite discrétisée en balises intermédiaires. Cette avenue intéressante est approfondie par les mêmes auteurs, mais cette fois couplée à un réseau d'itérations sur la valeur (VI) [102]. Les cartes sont par ailleurs générées à partir d'images via une nouvelle architecture ressemblant à un réseau LSTM avant d'être traitées par le module VI. Les deux modèles de guidage élaborés par ces auteurs se basent aussi sur l'apprentissage par imitation augmentée par la technique DAGGER. Des techniques avancées en construction de cartes de l'environnement sont employées par [103] en partenariat avec Facebook [104]. Leur architecture multimodules leur permet d'atteindre une destination en représentant l'environnement et en utilisant une règle de guidage locale obtenue par apprentissage par imitation et une règle de guidage globale obtenue par apprentissage par renforcement.

2.3.2.2 Apprentissage par renforcement

Pouvant facilement inclure un incitatif pour atteindre une destination dans la récompense, les méthodes suivantes ont quant à elles employé l'apprentissage par renforcement pour surmonter le problème de guidage autonome orienté vers une destination.

Zhou et al. (2019) [105] ont décidé d'affronter le problème de façon similaire à [99] au niveau de la tâche et similaire à [97] au niveau de l'architecture de leur modèle. La tâche à effectuer est le guidage vers une pièce dans un milieu intérieur à partir d'une autre pièce. L'architecture du modèle utilise «Deep-Q», une alternative aux algorithmes d'itération sur la valeur (VI) constituée d'un modèle similaire à [97] pour une tâche dans un contexte Markovien. Bien que les auteurs aient réalisé avec succès des tests hors simulation, leur modèle doit être réentraîné pour chaque nouveau milieu.

L'approche motivée par l'exploration adoptée par Zhelo et al. (2018) dans [106] aussi basée sur un contexte Markovien met pour sa part à profit la création d'une récompense intrinsèque en plus de la récompense extrinsèque habituellement utilisée dans un problème d'apprentissage par renforcement. La récompense intrinsèque vise à récompenser le robot simulé lorsqu'il fait preuve de curiosité et explore les environs. Le module intitulé «Intrinsic Curiosity Module» (ICM) ressemble structurellement à une architecture LSTM et est même complémentée d'un module

LSTM pour améliorer les résultats. Toujours basé sur les architectures LSTM, Khan et al. (2018) incorporent plusieurs concepts afin de mettre en place un modèle complet efficace pour le guidage orienté autonome [107]. Leur modèle encode un état constitué d'une numérisation de profondeur utilisant la technologie LIDAR et génère des prédictions sur les récompenses ainsi que les transitions d'états futurs. L'information encodée est traité par un module VI suivi d'un module de mémoire LSTM augmentée d'une mémoire externe comme dans [40] et [108]. Le tout porte le nom d'un réseau du type contrôle à mémoire augmentée MACN («Memory Augmented Control Network») et obtient d'excellents résultats expérimentaux. Il faut souligner ici que bon nombre de ces travaux se sont arrêtés à l'étape de simulation et qu'il est important de pouvoir discuter de leurs performances hors simulation pour bien les évaluer.

Les LSTM sont encore une fois utilisés pour augmenter les capacités de guidage orienté dans un contexte d'apprentissage par renforcement dans [21]. Cependant, leur travail inclut deux modules LSTM dont le second sert à traiter l'information du premier en plus d'une information supplémentaire ajoutée au réseau. L'information supplémentaire est constituée de la récompense ainsi que de la vélocité actuelle, ce qui permet au premier LSTM de traiter uniquement la représentation de l'image et de l'objectif à atteindre dans un premier temps et de traiter toute l'information dans le deuxième module. Cette technique unique semble porter fruit lorsqu'on se réfère aux résultats expérimentaux obtenus, mais semble également s'appliquer à des problèmes de guidage sur de courtes distances tels qu'à l'intérieur d'une pièce avec quelques obstacles.

Bien que le problème de guidage autonome orienté soit vastement exploré dans le cadre de robots miniatures, certains ouvrages ont préféré s'attarder aux voitures autonomes. Certains ont développé des modèles complexes multifonctions utilisant plusieurs techniques d'intelligence artificielle. Chen et al. (2019) ont élaboré un système pouvant détecter à partir des images prises par la voiture différentes informations sur le guidage tel que les lignes, le trafic routier, la signalisation routière [109]. Un module supplémentaire entraîné dans un GAN génère des scénarios d'urgence plausibles basés sur une base de données regroupant différentes urgences sur la route. Ces informations sont ensuite traitées par un algorithme entraîné par apprentissage par renforcement et décide de l'action à effectuer basé sur les risques exposés par le module génératif du GAN. Une version de ce modèle est aussi augmentée avec une perception de la profondeur de l'image acquise et améliore davantage les performances. Ce type de système complet gagnerait probablement à être testé dans un environnement similaire à [110], où leur algorithme est entraîné

et testé via une interface de programmation (API) développée par Google permettant l'utilisation du «Street View» comme environnement de simulation.

2.3.2.3 Autres approches

Il est possible de remarquer que les ouvrages dans cette section de revue de littérature traitant du problème de guidage global orienté portent principalement sur l'utilisation de robots non holonomes. En comparaison avec les solutions pour la planification de trajectoires locales non orientées, seules quelques études mentionnent l'utilisation de drones comme un système holonome. Il existe plusieurs compétitions pour le guidage autonome pour drones, dont celle menée par «Defense Advanced Research Project Agency» (DARPA) du programme de défense des États-Unis. Un résumé de leur projet est paru dans la littérature scientifique et mentionne les exploits d'un drone navigant à haute vitesse vers une destination inconnue pour ensuite revenir à son point de départ [111]. Leur drone réalise ces exploits en utilisant une combinaison d'approches déterministes et de modèles dynamiques pour la planification de trajectoires plutôt que des méthodes basées sur l'apprentissage. Toujours dans l'approche déterministique, [112] parvient à guider un drone dans un environnement inconnu en conservant une vitesse élevée comparativement aux autres méthodes de la littérature. La vitesse étant importante, des travaux comme [113] tentent d'effectuer des atterrissages sur des cibles en mouvements à grande vitesse. Certains ouvrages, tels que [114] emploient des techniques d'apprentissage pour la prédiction de la posture d'un humain afin de prédire les prochains mouvements d'un drone. Les prédictions permettent enfin de suivre un opérateur plus gracieusement. Le suivi d'une cible avec un ou des drones a aussi été abordé dans [105] et [106], où une approche basée sur la logique floue permet de concevoir un système optimisé pour la tâche. Il est aussi intéressant d'observer que l'optimisation de ces systèmes volants semble toujours tendre vers l'utilisation de systèmes de stéréovision. L'environnement étant connu lors de la réalisation des ensembles de données d'entraînement, il devient intéressant d'utiliser des experts optimaux basés sur des algorithmes tels que A* [117] et RRT* [113][114]. Cependant, ces planificateurs optimaux planifient la trajectoire optimale sans incorporer la composante d'exploration orientée dans leur comportement. En d'autres termes, ils effectuent une trajectoire comme si l'environnement était déjà connu de l'expert, ce qui nuit au comportement d'exploration. Il serait possible de tenter de reproduire ce comportement en ne donnant qu'une partie locale de l'environnement dans lequel l'oracle optimal se trouve. Cette représentation locale de

l’environnement peut alors être soupçonnée de ne pas être équivalente à l’information utilisée par le modèle de guidage, soit la vision de profondeur et les coordonnées relatives à la destination.

Cependant, le travail de recherche présenté dans ce mémoire de maîtrise se concentre sur le guidage et la planification de trajectoires locales orientées vers un objectif global pour un drone dans un environnement inconnu. Le suivi d’une cible ou d’un opérateur n’est donc pas envisageable. Il sera donc nécessaire d’adapter les forces des différentes approches présentées dans cette section sur le guidage.

CHAPITRE 3 MÉTHODOLOGIE

3.1 Interprétation de la profondeur

L'interprétation de la profondeur est un élément essentiel à ce projet. Elle permettra à l'agent autonome de mieux éviter les obstacles et lui permettra de généraliser ses capacités de guidage pour une grande variété d'environnements. Comme l'objectif final n'est pas de développer un algorithme de détection de profondeur optimisé pour le guidage automatique de drones, il a été nécessaire de sélectionner et d'évaluer des algorithmes existants ayant les particularités recherchées. Bien que l'interprétation de la profondeur par stéréovision soit plus performante que la détection monoculaire, il est commun dans la littérature [81, 87, 92, 111] d'opter pour l'alternative monoculaire afin d'éviter la dépendance à une deuxième caméra et par le fait même, diminuer le poids du drone. Le drone choisi devra donc respecter cette particularité.

De plus, il est maintenant reconnu que les modèles de prédiction de profondeur monoculaire par apprentissage profond ont surpassé les méthodes de prédiction basées sur la détection de traits ou de motifs confectionnés manuellement [48]. À cet effet, trois modèles de prédiction de profondeur présentés à la section 2.2 ont été testés.

Tableau 3.1 Modèles de prédiction de profondeur considérés pour le projet

Modèles		
Fully Convolutional Residual Networks (FCRN) **[121]**	SfM-Learner [63]	Revisiting Single Image Depth Estimation [51]

Les modèles du Tableau 3.1 ont été sélectionnés en raison de la disponibilité du code source, de la plateforme d'apprentissage utilisé et de l'accessibilité aux modèles entraînés. Cependant, seul le modèle FCRN a été en mesure de fournir une perception de profondeur à partir d'un flux vidéo à la hauteur des attentes en temps réel. Ce modèle a de meilleures capacités de généralisation à de nouveaux environnements tels que ceux qui seront utilisés pour la simulation dans Gazebo présentée à la section 3.3. Les modèles de détection de la profondeur à partir du mouvement tel

que SfM-Learner («Structure from Movements») ne sont pas suffisamment performants lorsque le drone est en vol statique.

Le modèle FCRN basé sur le populaire modèle ResNet-50 implémenté sur la plateforme TensorFlow a donc été sélectionné pour transformer le flux vidéo du drone préalable à son traitement par le modèle de guidage autonome pour drone. Il sera également utilisé pour générer l'ensemble de données expliqué à la section 3.4.

Puisque ce modèle a été entraîné sur l'ensemble de données « NYU-Depth v2» [122] constitué de 407 024 images prises par une caméra Microsoft Kinect, il effectuera une prédiction relativement précise pour chaque pixel qui varie entre 0.7 et 6 mètres de profondeur [11]. Les prédictions peuvent parfois dépasser la borne supérieure puisqu'il s'agit d'un modèle de régression.

La précision de ce modèle est évaluée selon leur erreur absolue relative sur «NYU-Depth v2» qui est de 0.127, ce qui se traduit par 12.7% d'erreur sur la valeur de chaque pixel de l'image de profondeur. Cette précision nous assure que pour la plage de prédiction actuelle variant de 0.7 à 6 mètres, on peut s'attendre à 76 cm d'erreur pour un objet à 6 mètres. Ceci étant dit, il a été observé que FCRN arrive à repérer la majorité des obstacles pouvant occasionner une collision à un minimum d'un mètre de distance. Les cas limites sont ceux qui comportent des obstacles comme des fils ou des branches de diamètre inférieur à environ 2.54 centimètres. Ce type d'obstacle peut parfois être détecté à environ un mètre de distance, ce qui pourrait justifier la nécessité d'un algorithme de détection pour des obstacles de cette taille à l'avance dans des travaux futurs. Cependant, les scénarios étudiés pour ce mémoire ne nécessitent pas autant de précision. Une estimation de la profondeur sera nettement suffisante pour guider le drone à sa destination. Lors de tests expérimentaux, il sera cependant prudent d'évaluer la sensibilité de l'algorithme au changement de luminosité (passage de l'extérieur ver l'intérieur par exemple) afin d'estimer une vitesse sécuritaire pour le déplacement du drone.

3.2 Sélection du drone

3.2.1 Caractéristiques recherchées

Le drone qui sera utilisé doit respecter plusieurs particularités afin de faciliter la réalisation du projet de recherche. Un résumé de ces particularités est présenté dans le Tableau 3.2.

Tableau 3.2 Caractéristiques recherchées pour la sélection du drone

Caractéristiques
Possède une caméra frontale
Capacités de communication avec un ordinateur
Accès au drone pour expériences futures
Documentation complète accessible

Puisque le projet repose sur le guidage basé sur la vision monoculaire, le drone sélectionné devra posséder une caméra frontale pouvant fournir un flux vidéo sans fil en couleur en temps réel. Il est entendu ici par temps réel que le flux vidéo devra être rafraîchi assez rapidement par rapport à la vitesse de déplacement du drone.

Il est essentiel de pouvoir communiquer avec un ordinateur pour envoyer des commandes de guidage au drone. Il sera aussi nécessaire que la communication sans-fil permette la réception du flux vidéo en temps réel fourni par la caméra frontale. Il n'est pas désiré d'effectuer tous les calculs sur le processeur du drone, car cela engendre un défi technique supplémentaire non négligeable hors de la portée du projet.

La suite de ce projet inclut une partie expérimentale dans laquelle les performances du drone seront testées dans un environnement de test muni d'un système de localisation Vicon [124]. Un accès facile au drone sélectionné est donc à prioriser. On entend par accès facile une facilité d'achat en termes de disponibilité et de coût. Idéalement, il serait question de prouver que cette expérience est réalisable avec un drone récréatif à bas coût disponible dans un magasin à grande surface. Le succès de ce projet exposera le potentiel de cette technologie une fois implémentée dans un drone plus sophistiqué ayant une meilleure représentation de son environnement (ayant plus de caméras par exemple).

Le drone sélectionné aura de préférence une documentation accessible et complète facilitant l'implémentation des divers modules de communication, commande et de simulation.

3.2.2 AR Drone 2.0

Le Parrot AR Drone 2.0, présenté à la Figure 3.1, a été sélectionné en raison de sa conformité aux critères recherchés. Ce drone est d'ailleurs disponible au laboratoire de recherche en Conception de Systèmes Intelligents et Mécatroniques (CoSIM) du Prof. Sofiane Achiche. Le AR Drone 2.0 possède évidemment une caméra frontale ainsi qu'une librairie C++ développée par Parrot permettant la communication avec le drone. La librairie de développement («software development kit» ou SDK) est compatible avec d'autres programmes Python comme expliqué à la section 3.3 et permet facilement la commande du drone expérimental ainsi qu'en simulation numérique. De plus, puisqu'elle est développée par Parrot, une documentation complète est disponible en ligne. La date de sortie du drone combinée à la commodité d'utilisation ont rendu disponible une multitude de tutoriels portant sur l'utilisation et la simulation du drone.



Figure 3.1 AR Drone 2.0 de la compagnie Parrot [125]

3.3 Environnement de simulation

Le projet actuel repose essentiellement sur l'utilisation d'une infrastructure de simulation utilisant l'intergiciel Robot Operating System (ROS) [126] conjointement avec Gazebo [127]. Le choix des versions des deux plateformes a été fait de façon à pouvoir utiliser l'outil de développement logiciel officiel du Parrot AR.Drone 2.0. Ainsi, ROS Kinetic et Gazebo 7.15.0 ont été installés sur une machine Linux Ubuntu 16.04 LTS équipée d'une carte graphique NVIDIA 1080Ti. Les

spécifications techniques de l'ordinateur en question sont nécessaires pour opérer ROS et Gazebo avec le AR.Drone 2.0.

L'outil de développement logiciel développé par Parrot pour le AR.Drone 2.0 se nomme *ardrone_autonomy* [29] et permet le contrôle du AR.Drone 2.0 via un ordinateur. Puisque ce projet se limite à l'étude du AR Drone 2.0 en simulation, un simulateur a dû être installé. Un simulateur intitulé *tum_simulator* [129] réalisé par l'Université Technique de Munich a été conçu pour être utilisé sur la même plateforme ROS et Gazebo. Le simulateur est utilisé de pair avec une version modifiée d'un autre nœud de communication ROS destiné au guidage automatique du AR.Drone 2.0. Une description détaillée de ces plateformes et de ces programmes est présentée dans les sections suivantes.

3.3.1 Robot Operating System

Le Robot Operating System communément appelé ROS, est un système d'exploitation *open source* contenant les librairies et les outils permettant de contrôler un robot. Il fonctionne en générant un graphe de dépendance entre les différents éléments et le robot, et communique entre eux via un réseau de communication interne pair à pair («peer-to-peer»). Le système synchrone ou asynchrone peut également être distribué sur plusieurs machines communiquant entre elles. Chaque élément du graphe, ou encore nœud, publient des messages ROS pouvant être lus par d'autres nœuds.

3.3.2 Gazebo

Gazebo est un paquet ROS contenant l'interface permettant la simulation de robots dans un environnement 3D à corps rigides. Il permet d'importer une multitude d'environnements 3D à condition qu'ils soient sauvegardés sous un format standard. L'interface de simulation comprend également les éléments de base de contrôle pour la physique tels que la gravité. Le tout est intégré à ROS et communique à l'aide de messages ROS.

3.3.3 Modèle du drone

Le modèle du AR.Drone 2.0 qui a été utilisé dans ce projet a été développé par l'Université Technique de Munich. Leur simulateur est disponible sur GitHub sous le nom de *tum_simulator* et est documenté sur le site officiel de ROS [129]. Le simulateur implémente les capteurs essentiels présents sur le drone étudié et permet même la communication avec le vrai drone si désiré.

Tableau 3.3 Caractéristiques du modèle AR.Drone 2.0

Caractéristiques implémentées	Caractéristiques non implémentées
Caméra avant et inférieure	Capteur magnétique (magnétomètre)
Capteur d'altitude	Capteur de pression
Capteurs IMU	Estimation des vents

Le fonctionnement du *tum_simulator* repose sur les pilotes officiels du paquet *ardrone_autonomy* développé par Parrot qui sont également disponibles sur GitHub. Il suffit d'installer les deux paquets selon le guide fourni dans la documentation ROS du simulateur pour pouvoir créer une simulation dans Gazebo. Le guide suggère également l'installation d'un package permettant de contrôler le drone avec une manette de jeu de type PlayStation.

Cependant, la précision du contrôle offerte par une manette de ce genre ajoute un degré de difficulté au problème de guidage étudié. En effet, puisque ce projet de maîtrise se concentre sur le développement d'une solution d'apprentissage automatique par imitation, il serait préférable de limiter l'ensemble des commandes possibles à envoyer au drone. Une manette de type PlayStation comportera ainsi un trop grand nombre de combinaisons de commandes possibles puisqu'elle permet d'encoder simultanément différentes vitesses dans deux directions en fonction de l'angle du joystick.

Il a donc été choisi de contrôler le drone avec un clavier puisque chaque touche est attribuée à une vitesse prédéfinie de translation ou de rotation. Le découplage de ces vitesses permet un meilleur contrôle sur le nombre de directions de déplacement du drone et permet de transformer le problème en un problème de classification plutôt qu'un problème de régression. Le problème de classification fait ici référence à un nombre discret de directions de déplacement à vitesse constante alors que le problème de régression permet un déplacement à vitesse variable dans toutes les directions.

Le projet AR.Drone 2.0 Autopilot a développé un paquet ROS permettant entre autres l'utilisation d'un clavier pour le contrôle du AR.Drone 2.0 en simulation ou en contexte expérimental avec un vrai drone [130]. Ce package intitulé *ardrone_autopilot* a initialement été développé pour la

détection et le suivi d'une cible en utilisant la librairie OpenCV ainsi que les données de guidage en temps réel. Il a été développé en langage Python et contient les fonctions de base pour le contrôle avec un clavier en plus des fonctions de détection et de guidage autonome utilisant la caméra frontale du drone. Cependant, seule la partie permettant le contrôle manuel avec un clavier a été modifiée et réutilisée. Les autres fonctions de guidage autonome ne sont d'aucune utilité au projet. Comme mentionné précédemment, ce paquet a aussi l'avantage d'être implémenté en langage Python, ce qui facilitera la modification de son nœud ROS pour le reste de l'environnement de simulation.

En résumé, l'environnement ROS peut être schématisé par la Figure 3.2, où on peut y voir le lien entre le drone ou son modèle et l'interface de contrôle.

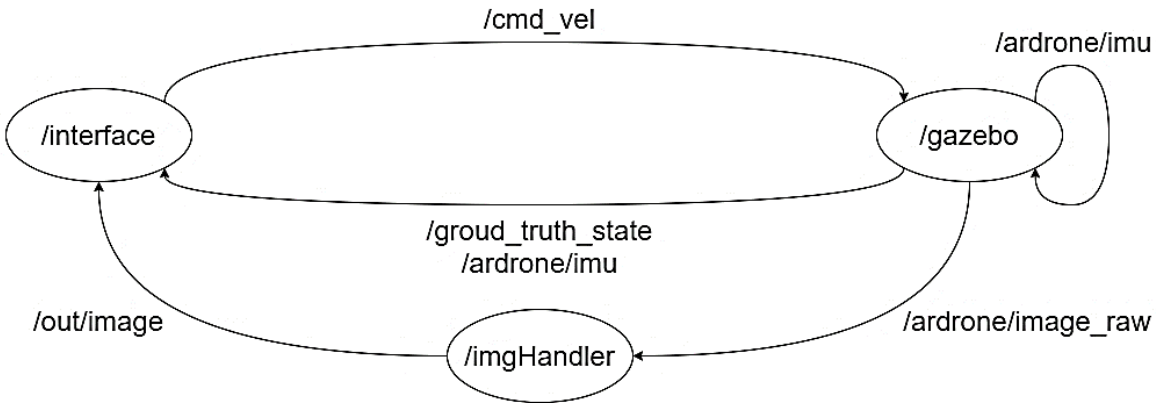


Figure 3.2 Diagramme de communication des nœuds ROS

3.3.4 Communication interprogrammes

Le projet de recherche présenté dans ce mémoire de maîtrise utilise les récents outils d'apprentissage profonds tels que PyTorch et TensorFlow qui s'exécutent sur des versions de Python plus récentes que celle utilisée dans ROS. En effet, les algorithmes d'apprentissage développés et utilisés dans ce projet s'exécutent sur Python 3.X alors que les outils de simulations ROS et Gazebo pour le Parrot AR.Drone 2.0 ont tous été développés pour la version de ROS utilisant Python 2.7.

Il a été impossible de forcer ROS à utiliser Python 3.6 ou à forcer les algorithmes d'apprentissages à utiliser Python 2.7. Il a donc été nécessaire de trouver un moyen de communiquer les informations disponibles dans ROS aux algorithmes d'apprentissage s'exécutant dans un programme à

l'extérieur de l'environnement ROS. La solution qui a été implémentée consiste à utiliser le module OpenCV permettant d'accéder à une caméra IP utilisant un protocole TCP. Le même type de communication entre les deux programmes a été implémenté pour la communication des données de position. Il a été facile d'implémenter un serveur TCP envoyant ces informations en modifiant le nœud Python du paquet *ardrone_autopilot*. Le programme client contenant les programmes d'apprentissage profond en Python 3.6 a au final été en mesure de recevoir et d'afficher les images et les données de position en temps réel à raison d'environ 23 images par seconde.

La Figure 3.3 illustre le schéma de communication entre les deux environnements Python. Puisque le module *ardrone_autopilot* implémente déjà un moyen de communication avec le drone via le clavier, le programme d'apprentissage profond contrôlera le drone en simulant l'exécution de touches sur le clavier avec le module Python *pyautoinput*.

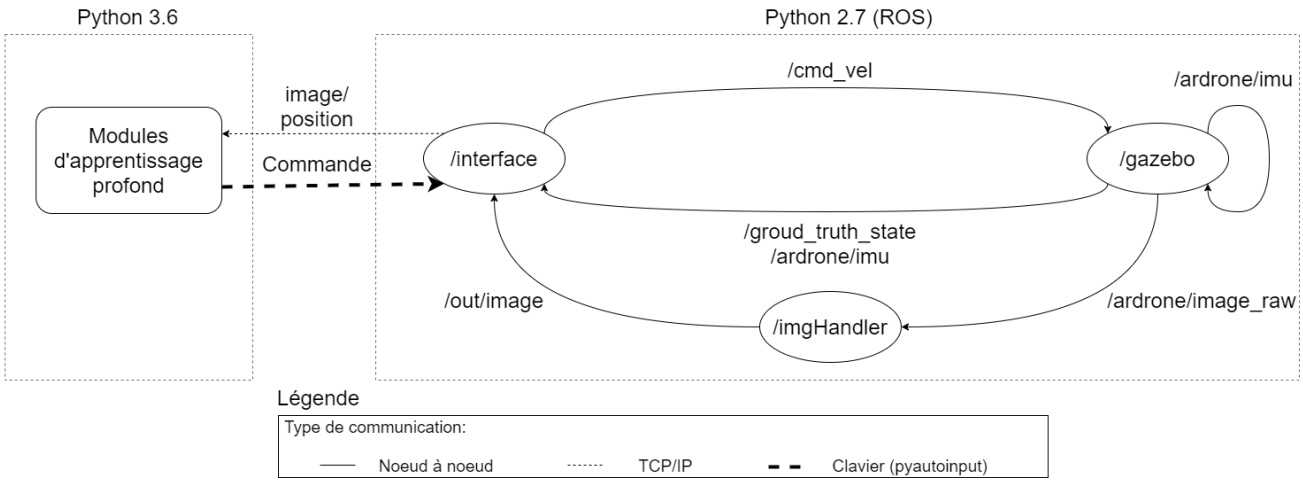


Figure 3.3 Diagramme de communication de l'interface ROS et du programme externe

L'ensemble de ces programmes constitue donc la base de l'environnement de simulation. Cet environnement sera utilisé dans plusieurs étapes subséquentes du projet, particulièrement pour la création de l'ensemble de données d'entraînement et pour la partie de tests de simulations.

3.4 Ensemble de données

Tel qu'expliqué à la section 2.1.1.2, il est nécessaire de bâtir un ensemble de données adapté à un problème d'apprentissage profond par imitation. Ce type d'ensemble de données est créé en

enregistrant une grande quantité de trajectoires réalisées par un expert, donc suboptimal de par la nature humaine. Ces trajectoires permettent d'établir la relation entre les entrées et les sorties du modèle à entraîner. Elles sont constituées de paires de données d'état et de commande de la forme $\tau_i = [(x_0, u_0), (x_1, u_1), \dots, (x_t, u_t)]$. Les données d'état sont représentées par la variable x alors que la commande est représentée par la variable u. Ainsi, chaque trajectoire τ_i comporte un nombre j de paires (x_j, u_j) représentatif de la longueur temporelle de la trajectoire. Le modèle final tentera donc d'imiter l'expert en se basant sur la logique extraite des différents parcours de référence. La section suivante présente les différentes particularités de l'ensemble de données intitulé *Drone Guidance Dataset* (DGD), qui a été créé, ainsi que la procédure employée pour sa création.

3.4.1 Sélection des environnements

L'environnement dans lequel l'ensemble de données sera créé est très important, car il aura plusieurs impacts dans les performances finales du modèle ainsi que d'autres impacts sur la complexité de réalisation.

L'une des idées envisagées était de construire des murs mobiles, qui une fois placés à l'intérieur d'une pièce, serviraient d'obstacles au drone. Différentes configurations de ces murs permettraient d'obtenir différents labyrinthes pour la création d'un ensemble de données varié. Le drone aurait été piloté manuellement dans ces environnements d'essais vers une multitude de cibles marquées par des coordonnées 2D relatives à la position du drone. Les coordonnées du drone auraient été mesurées en temps réel avec un système de caméra Vicon. Ces coordonnées, en plus du flux vidéo et des commandes envoyées au drone, auraient été enregistrées via une interface ROS.

La réalisation de cet environnement est un projet ambitieux ayant un coût élevé en temps et en argent en plus de l'occupation prolongé d'une salle équipée d'un système Vicon ou équivalent. De surcroît, manœuvrer le drone dans cet environnement comporte également son lot de défis à surmonter. L'efficacité de ce procédé est aussi tout au plus médiocre en raison du besoin de composer avec un nombre de batteries limité ayant une courte durée de vie. Sans oublier le stress occasionné au pilote qui doit piloter le drone un nombre incalculable de fois sans erreurs de parcours ni collisions par crainte d'endommager le quadrotor. La collecte de données peut enfin également s'avérer fastidieuse puisqu'elle requiert l'utilisation d'un système informatique complexe qui n'est jamais à l'abri de bogues ou de surcharges.

Pour ces raisons, il a été envisagé de créer un ensemble de données dans un environnement de simulation. Les environnements de simulation sont en constante évolution et offrent désormais une qualité graphique comparable à celle obtenue par caméra. Comme mentionné précédemment, il sera question d'utiliser un flux vidéo constitué d'images de profondeur pour améliorer l'intuition du modèle ainsi que sa généralisation aux différents environnements. Ceci étant dit, il convient d'assumer que tant que les environnements de simulation sont assez riches en textures et en détails pour générer des images de profondeurs comparables à des images réalisées par le même algorithme à partir d'images prises par une caméra, les performances devraient être comparables.

Il a été possible de trouver et d'acheter des modèles 3D de l'intérieur de divers édifices commerciaux et privés réalisés à l'aide d'une technologie de numérisation 3D et d'imagerie de grande qualité. Les modèles 3D comportent une vaste quantité de détails tels que l'ameublement typique relié au type de bâtiment. La Figure 3.4 est un exemple de modèle 3D d'un resto-bar utilisé pour la création de l'ensemble de données. Se référer à l'Annexe A pour les images des autres modèles 3D utilisés.



Figure 3.4 Vue isométrique (gauche) et intérieure (droite) du modèle 3D *Resto-Bar* [131]

Le Tableau 3.4 contient les informations générales sur les modèles qui ont été sélectionnés pour la création de l'ensemble de données.

L'utilisation de ces modèles 3D facilitera donc la création d'un ensemble de données d'entraînement plus varié et plus volumineux que l'option initiale, et ce, à coût moindre en termes de temps et d'argent. Enfin, une plus grande quantité de parcours différents est favorable pour la capacité d'un modèle à généraliser à de nouveaux environnements [17].

Tableau 3.4 Description des modèles 3D sélectionnés [132]

	Nom	Description
1	*Bar à Billard*	Bar à billard antique à aire ouverte
2	*Maison 1*	Énorme maison à 2 étages avec long corridors et une multitude de pièces
3	*Salle de machines*	Salle de machine industrielle complexe avec passages étroits
4	*Salle de serveurs*	Petite salle informatique pour la gestion des serveurs
5	*Bureau*	Bureau commercial à grande surface
6	*Resto-Bar*	Restaurant avec espace bar décontracté
7	*Maison 2*	Maison à 2 étages régulière avec une terrasse

3.4.2 Sélection des classes du modèle

Après s'être familiarisé aux divers environnements de simulation et du AR Drone 2.0, il a été nécessaire de déterminer quelles seraient les commandes qui seraient exécutées par le modèle de guidage autonome. Inspiré par [89], il a été estimé qu'il serait préférable de limiter l'éventail du nombre de commandes possibles par souci de simplicité du modèle.

Il est en effet possible de contrôler la position du drone dans deux dimensions en utilisant seulement trois commandes de base déjà implémentées à même le nœud ROS *ardrone_autopilot*. Les commandes sont en question sont AVANT, GAUCHE, DROITE. Elles ont pour effet de commander une vitesse linéaire constante positive dans le repère du drone pour la commande AVANT et de commander une vitesse de rotation positive ou négative autour de l'axe de lacet du drone pour les commandes GAUCHE et DROITE. Le contrôleur interne du drone s'occupe ensuite de faire respecter la commande envoyée.

Bien que ces trois commandes permettent le guidage en deux dimensions de façon fonctionnelle avec seulement trois commandes, il a été remarqué que ces commandes ne favorisent pas les

déplacements fluides : le drone s'arrête pour corriger son orientation à chaque fois que les commandes GAUCHE ou DROITE sont envoyées. Les accélérations et décélérations des départs et des arrêts ont pour effet de faire osciller le drone, ce qui peut s'avérer nuisible lors de la création de la partie vidéo de l'ensemble de données. Les vibrations nuisent par exemple à la qualité de l'image, car le drone n'est pas équipé de système à cardan comme les drones plus récents.

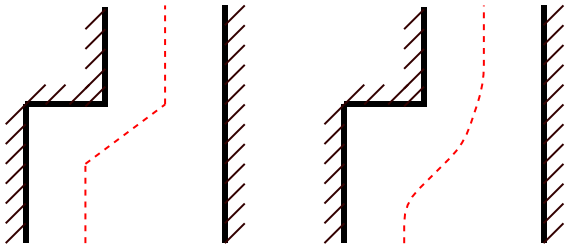


Figure 3.5 Trajectoires typiques obtenues avec les trois commandes de base (gauche) versus avec les commandes combinées (droite)

De ce fait, il a été décidé d'ajouter les commandes AVANT-GAUGHE et AVANT-DROITE. Ces deux nouvelles commandes sont des combinaisons des commandes précédentes favorisant la fluidité des mouvements du drone en lui permettant de corriger sa trajectoire de façon moins saccadée tout en continuant d'avancer. La Figure 3.5 illustre la différence en fluidité des trajectoires possibles. Il est possible d'y observer l'avantage d'un gain en fluidité sur le guidage dans des espaces restreints tels que des corridors.

Il est prévu que l'ajout de ces deux commandes complexifiera l'entraînement ainsi que l'évaluation du modèle, mais que cette complexification est estimée être mineure comparativement à l'avantage qu'elle apporte. La complexification vient d'une part du simple fait que l'augmentation du nombre de classes dans un problème de classification est synonyme d'une plus grande complexité. Elle vient d'une autre part du fait que les couples de commandes AVANT-GAUCHE, GAUCHE ainsi que AVANT-DROITE, DROITE sont équivalents dans une certaine mesure. L'utilisation de l'une ou l'autre des commandes peut alors parfois être justifiée pour une même situation. Il sera donc important de considérer cet aspect sur les performances en classification et observer la fréquence à laquelle ces classes sont confondues.

Les autres commandes telles que celles qui permettent des déplacements latéraux ou vers l'arrière sont dans ce contexte-ci sont peu intéressantes puisqu'elles engendrent des déplacements dans des

directions qui ne sont pas observables par la caméra. Il serait ainsi plus dangereux d'entrer en collision avec l'environnement en utilisant ces commandes dépourvues de rétroaction visuelle.

La dernière commande utile pour le contrôle du drone est la commande NEUTRE. Cette commande est essentielle puisque la personne qui pilotera le drone lors de la création l'ensemble de données devra nécessairement prendre des petites pauses pour évaluer l'image actuelle et prendre une décision quant à la direction à prendre. Les pauses pourront ensuite être retirées de l'ensemble de données en post-traitement puisqu'elles n'ont aucune valeur ajoutée. Au contraire, présenter des états constitués d'images, de positions et d'orientations relatives similaires, mais reliées à différentes actions risque de nuire à l'optimisation du modèle. De plus, il n'est pas souhaitable que le modèle de guidage autonome final prenne des pauses pour analyser la situation actuelle. L'analyse de l'état actuel se fait quasi instantanément à chaque itération, contrairement à un opérateur, qui peut parfois être distrait et prendre un délai supplémentaire pour choisir la bonne direction.

Finalement, les classes qui seront enregistrées lors de la création de l'ensemble de données seront constituées des cinq commandes de vitesses énumérées plus tôt en plus de la commande NEUTRE telles que résumées dans le Tableau 3.5.

Tableau 3.5 Commandes utilisées pour la création de l'ensemble de données

Commandes
NEUTRE
AVANT
GAUCHE
DROITE
AVANT-GAUCHE
AVANT-DROITE

3.4.3 Collecte de l'ensemble de données

L'objectif de la création de l'ensemble de données est de sauvegarder une multitude de trajectoires différentes à l'aide des modèles d'environnements 3D. Pour se faire, chaque environnement 3D a

été exploré de façon à déterminer plusieurs points d'intérêt tels que les différentes pièces d'une maison, etc. Les coordonnées de chacun des points d'intérêt sont notées et numérotées dans un fichier Excel ainsi que dans les plans de l'environnement en question pour utilisation ultérieure. La Figure 3.6 est un exemple d'annotation des lieux d'intérêts pour une maison. Dans le cas où les environnements 3D ont plus d'un étage, chaque étage est utilisé comme un environnement indépendant de façon à faciliter l'enchaînement entre les lieux lors de la création de l'ensemble de données. Les autres environnements 3D utilisés pour l'ensemble de données sont présentés à l'Annexe A.

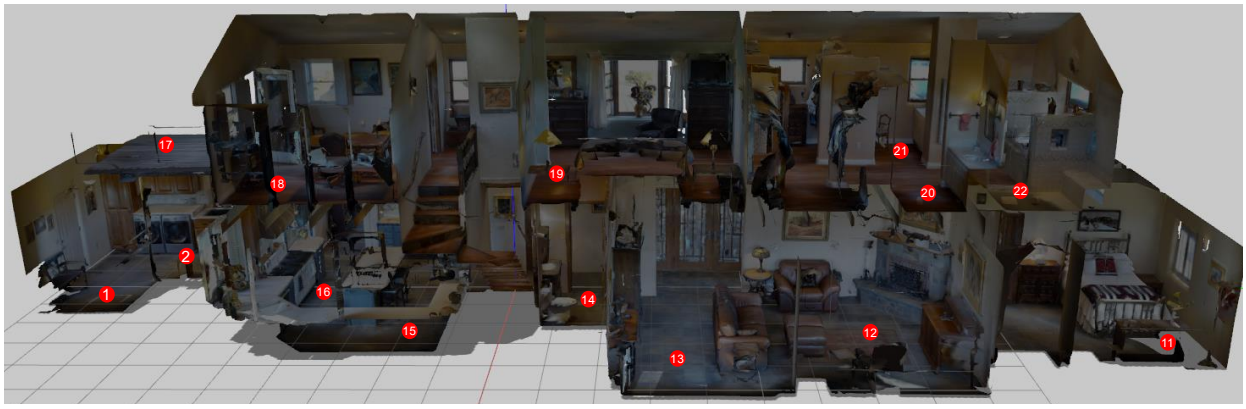


Figure 3.6 Annotation des points d'intérêts dans l'environnement *Maison 2*

Un script Python a été élaboré afin de pouvoir enregistrer des séquences de vol du AR Drone 2.0 dans le simulateur Gazebo. Le script communique avec ROS par un protocole TCP/IP tel que décrit dans la section 3.3.4. L'utilisateur doit communiquer avec le programme avec le clavier pour débuter, annuler ou terminer un enregistrement ou encore pour arrêter le programme.

Au lancement du programme, le point d'intérêt 2 est sélectionné par défaut peu importe l'environnement 3D choisi et doit être atteint à partir du point d'intérêt 1, ce qui implique que le drone doit se situer à l'origine pour débuter. Lorsque l'objectif est atteint, un nouvel objectif est choisi au hasard dans la liste Excel de points d'intérêts. Le drone devra donc parcourir le trajet entre l'objectif précédent et le nouvel objectif. Cette stratégie assurera une bonne variabilité des parcours sans devoir effectuer manuellement tous les trajets possibles. Certains environnements 3D possèdent plus de vingt points d'intérêts en raison de leur grande surface, ce qui permettrait de générer plus de cinq cents trajectoires différentes. Cependant, dû à la configuration de plusieurs de ces modèles 3D intérieurs, plusieurs de ces trajectoires devront emprunter les mêmes corridors ou

espaces communs avant d'atteindre leurs objectifs, ce qui nuit à la richesse (variabilité) de l'ensemble de données. La stratégie aléatoire permettra ainsi de couvrir la majorité des parcours possibles tout en explorant un maximum de points de vue différents.

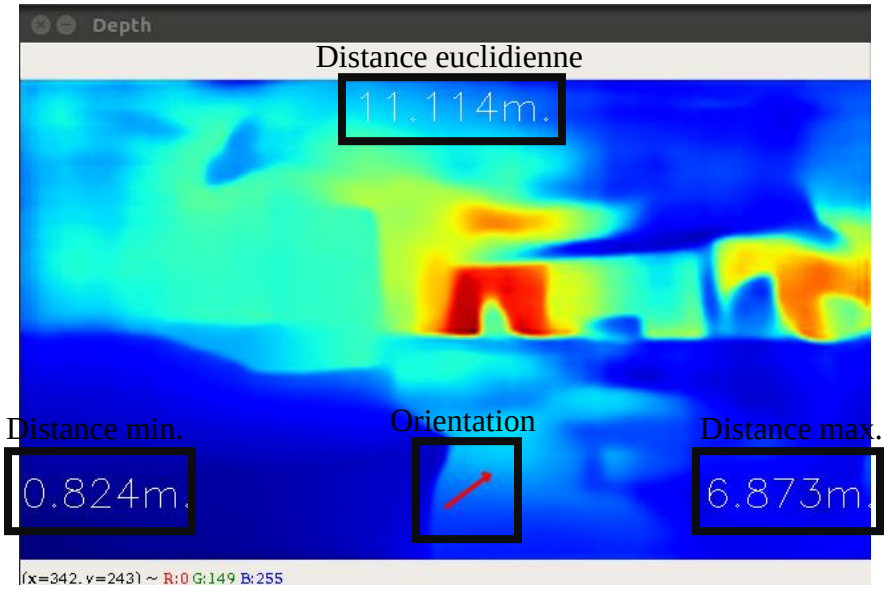


Figure 3.7 Fenêtre de référence pour le guidage et la création de l'ensemble de données. Contient la carte de profondeur, la distance euclidienne et l'orientation entre la position actuelle et l'objectif.

Lors de l'enregistrement des trajectoires, l'expert doit se déplacer dans l'environnement de simulation en en se dirigeant vers un point d'intérêt aléatoire en suivant la flèche indiquant son orientation relative ainsi que la distance euclidienne entre la position actuelle du drone et le point d'intérêt. Pour ce faire, il est impératif que l'expert se serve uniquement de ces informations et du flux vidéo de profondeur tel qu'illustré à la Figure 3.7. Il est à remarquer que la fenêtre affiche également dans les coins inférieurs gauche et droit la distance minimale et maximale détectée par l'algorithme de profondeur respectivement. Les distances sont utiles pour complémenter le code de couleur représentant le champ de profondeur dans l'image. Elles varient entre 0 et 6 mètres comme mentionné à la section 3.1.

Outre les informations limitées sur lesquelles l'expert doit se baser pour réaliser les parcours, il doit s'efforcer de conserver la même logique lors de la prise de décision et de ne jamais se baser sur des informations qui ne sont pas observables ou qui n'ont pas été observées récemment [73]. Ce type d'informations peut être difficile à omettre lors de la prise de décision, car il fait souvent référence à une connaissance préalable de l'environnement de simulation 3D. Idéalement, l'expert

devrait naviguer dans une infinité d'environnements 3D différents qu'il n'a jamais vu afin de minimiser toute forme de familiarisation avec les environnements. Cependant, cette option n'est évidemment pas envisageable simplement puisque le nombre de modèles 3D intérieurs comme ceux utilisés est limité. Pour compenser, il faut s'assurer que l'expert soit constant dans ces actions pour ne pas créer de scénarios conflictuels au sein de l'ensemble de données.

La logique à respecter pour le guidage est toutefois simple. Elle consiste à choisir l'orientation qui permet de minimiser la distance entre l'objectif et la position actuelle. Lorsque l'expert arrive face à un mur, un obstacle, ou plusieurs chemins, il doit emprunter la direction qui minimise l'orientation relative à l'objectif. La décision contraire devra seulement être exécutée dans l'éventualité où l'autre option a déjà été explorée sans succès. Un résumé des actions possibles est présenté à la Figure 3.8.

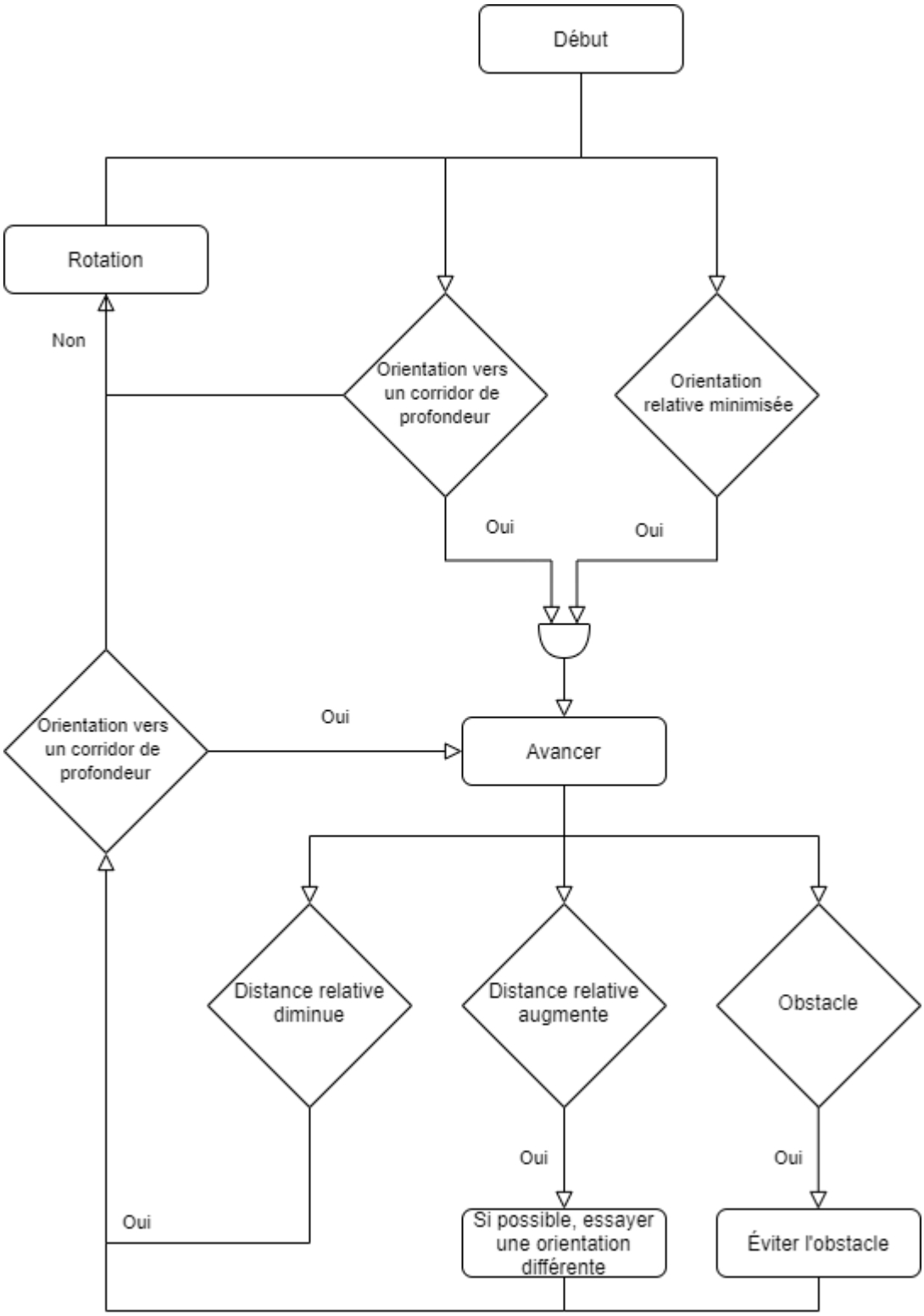


Figure 3.8 Diagramme décisionnel pour l'expert

Le programme responsable de la création de l'ensemble de données enregistre chaque trajectoire dans un fichier de données hiérarchique version 5 (HDF5). Ce type de fichier est reconnu pour son efficacité à sauvegarder des données volumineuses de façon structurée telles que celles recueillies par l'expert [115]. Il permet également de lire le fichier de façon plus optimale en lisant seulement les segments du fichier qui doivent être accédés. Les données enregistrées pour chaque trajectoire sont présentées dans le Tableau 3.6.

Tableau 3.6 Éléments enregistrés lors de la création de l'ensemble de données

Éléments enregistrés	
Flux d'images de profondeur	Distance relative en y
Flux d'images vidéo	Orientation de l'objectif
Orientation relative à l'objectif	Classes
Distance relative en x	

Il va de soi que le flux d'images de profondeur est parmi les données enregistrées. Celui-ci fait partie de l'hypothèse à vérifier. Le flux d'images vidéo est quant à lui nécessaire pour évaluer les performances du modèle final lorsqu'on l'entraîne avec des images RGB plutôt que des images de profondeur. Elles sont aussi utiles s'il devient nécessaire d'utiliser un nouvel algorithme plus performant pour le calcul de la profondeur, ce qui n'a pas été nécessaire dans ce mémoire.

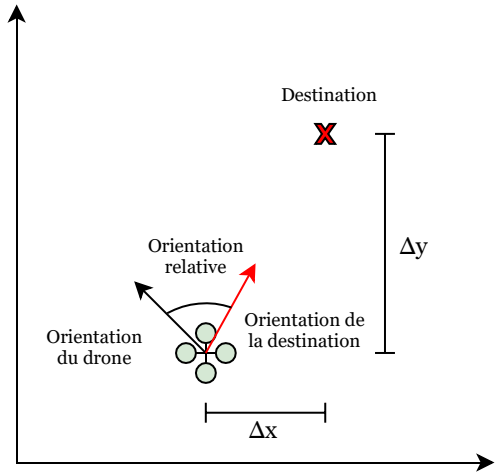


Figure 3.9 Quantités scalaires mesurées lors de la réalisation d'une trajectoire

L'orientation relative ainsi que les distances relatives en x et y à l'objectif sont les informations de base nécessaires à la représentation de l'objectif à atteindre tel qu'illustré à la Figure 3.9. L'orientation relative est la différence entre l'orientation actuelle du drone ainsi que l'orientation de l'objectif. Puisque cette relation mathématique relie les deux orientations, il n'est pas nécessaire de sauvegarder l'orientation actuelle du drone et occuper de la mémoire inutilement. La convention de signe veut que l'objectif soit à gauche lorsque l'orientation relative est positive et vice versa. Les éléments mentionnés précédemment feront partie de l'ensemble de données d'état représenté par la variable x_j dans la trajectoire τ_i.

Le dernier élément crucial à la création de l'ensemble de données est la classe u_j attribuée à chaque instance de donnée d'état x_i. La classe u_j pour ce problème de classification représente la commande qui sera envoyée au drone. Elle peut prendre une seule valeur parmi les classes énumérées dans le Tableau 3.5 de la section 3.4.2.

Une fois les données de la trajectoire recueillies, elles sont enregistrées dans un fichier nommé selon une formulation générique équivalente à *path_X_Y_ZZZ.h5*. Ici, *X* équivaut au numéro du lieu d'intérêt à partir duquel l'expert est parti et *Y* équivaut à la destination finale de la trajectoire. Puisqu'il est possible que la même trajectoire soit empruntée plus d'une fois, le titre du fichier est incrémenté à l'aide des caractères *ZZZ*. Il est à noter que les numéros correspondants aux destinations *(X,Y)* sont des numéros d'index utilisés par le programme Python lorsqu'il lit le fichier Excel comprenant toutes les destinations. Ces numéros d'index commencent donc à zéro, ce qui

signifie que la trajectoire nommée *path_0_3_001.h5* correspond à la trajectoire entre la destination 1 et 4 sur la carte des destinations et à la deuxième itération de cette trajectoire.

Les données brutes sont finalement traitées par un programme de raffinement afin de les préparer à être envoyées au modèle lors de la phase d'entraînement. La taille des images a par exemple été diminuée et les valeurs des pixels normalisées. L'orientation relative a aussi été normalisée entre -1 et 1 pour tenir compte de la direction. Le programme peut également effectuer des transformations sur les classes comme le retrait d'une classe, etc.

Initialement, l'ensemble de données comportait 626 trajectoires réparties sur 9 environnements 3D, ce qui représente 359 706 paires de données d'état et d'entrée de contrôle sous la forme $\tau_i = [(x_0, u_0), (x_1, u_1), \dots, (x_t, u_t),]$ pour un total de 275 GB où τ_i correspond à une des 624 trajectoires comprenant t couples d'état et d'entrée de contrôle répartis à un pas de temps constant.

3.4.4 Variantes de l'ensemble de données

Plusieurs variantes de l'ensemble de données ont été créées au cours de la phase d'entraînement. Telle que représentée dans le Tableau 3.7, la première variante de l'ensemble de données comprenait très peu de modifications autres que les normalisations nécessaires pour faciliter l'entraînement et la suppression de certaines trajectoires incomplètes. Les images étaient de taille 320x192, ce qui limitait considérablement la taille de *batch* servant à l'entraînement sur le GPU. Cette variante possédait aussi la classe NEUTRE avant qu'elle ne soit retirée pour des raisons mentionnées à la section 3.4.2.

Tableau 3.7 Tableau récapitulatif des différentes itérations de l'ensemble de données DGD

Version	Nombre de trajectoires	Nombre de paires (x_t, u_t)	Taille (Gb)
Initiale brute	626	359 706	275
Images recadrées - Sans classe NEUTRE	624	296 466	257
Sans flux d'image RGB	624	296 466	14,8

La seconde itération de l'ensemble de données a été ajustée à la lumière des éléments problématiques mentionnés au sujet de la taille des images. Les images ont vu leur taille diminuer de 320x192 à 160x96 et la classe NEUTRE a été retirée complètement.

Finalement, une dernière variante a été introduite, supprimant le flux d'image vidéo RGB qui n'était présent que pour générer de nouvelles images de profondeur si jamais l'enregistrement de celles-ci devait être corrompu. La version finale de l'ensemble de données DGD comporte 624 trajectoires, soit 296 466 paires de données d'état et d'entrées de contrôle pour un total de 14,8GB. Le Tableau 3.8 présente les caractéristiques des différentes données de l'ensemble DGD. On y remarque que la plupart des éléments sont normalisés pour faciliter le processus d'optimisation lors de l'entraînement du modèle d'apprentissage profond. Seules les distances relatives ont été conservées comme telles puisque les normaliser inhiberait la capacité du modèle à estimer les distances à parcourir.

Tableau 3.8 Description des données finales constituant l'ensemble de donné DGD

Donnée	Normalisation	Unités
Flux d'images de profondeur 160x96 ($\mu = 0.1792, \sigma = 0.1497$)	[0, 1]	N/A
Orientation relative	[-1, 1]	N/A
Distance relative en x	N/A	mètres
Distance relative en y	N/A	mètres
Orientation de l'objectif	[0, 2π]	N/A
Classe	N/A	N/A

La répartition des classes de l'ensemble de données final est présentée dans le Figure 3.10. Le débalancement entre le nombre de classes n'est pas surprenant considérant la nature de la tâche à imiter. Comme remarqué lors de la création de l'ensemble de données, la majorité des déplacements effectués dans des environnements quotidiens (en voiture, à la maison, dans un édifice, etc.) impliquent peu de changements de direction par rapport au temps passé à suivre une trajectoire linéaire tel qu'un corridor ou une route pour un automobile. Un débalancement des classes est donc inévitable. Des solutions seront présentées à la section 3.6 afin de pallier ce débalancement.

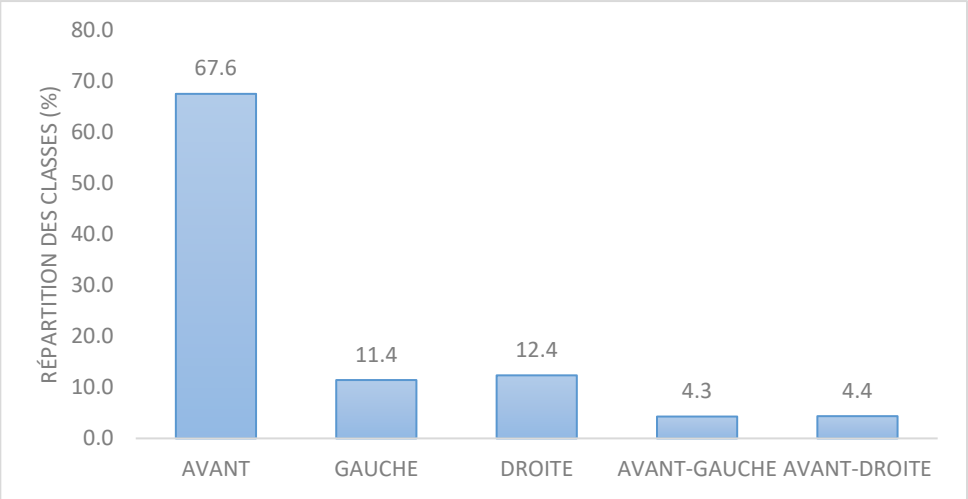


Figure 3.10 Répartition des classes de l'ensemble de données DGD

3.5 Modèle de guidage autonome pour drone

Le modèle d'apprentissage par imitation développé dans ce projet de recherche a été construit sur la plateforme PyTorch développée par Facebook [16]. PyTorch est une plateforme flexible et modulaire facilitant tout travail de recherche dans le domaine [134]. C'est également une plateforme familière pour les membres du laboratoire CoSIM, ce qui facilite la poursuite du projet au sein du laboratoire. Il est à noter que le fait que l'algorithme d'apprentissage profond utilisé pour la prédiction des images de profondeur utilise la plateforme TensorFlow ne pose aucun problème au niveau de l'exécution du modèle de guidage. Les deux librairies peuvent être utilisées simultanément dans un même programme Python sans interférences.

L'ensemble de données créé à la section 3.4 contient pour chaque couple d'entrée et de sortie d'une trajectoire (x_j, u_j), une entrée x_j constituée d'une image de profondeur ainsi que plusieurs valeurs scalaires associées à l'orientation relative et à la distance relative à l'objectif de guidage du drone. Ces valeurs scalaires sont représentées par une cible à la Figure 3.11. Ceci implique que le modèle de prédiction pour la commande du drone devra pouvoir traiter des entrées de deux types différents. En raison des nombreux avantages mentionnés à la section 2.1.2.2, un réseau convolutif a été adopté pour encoder l'image. Les valeurs scalaires sont

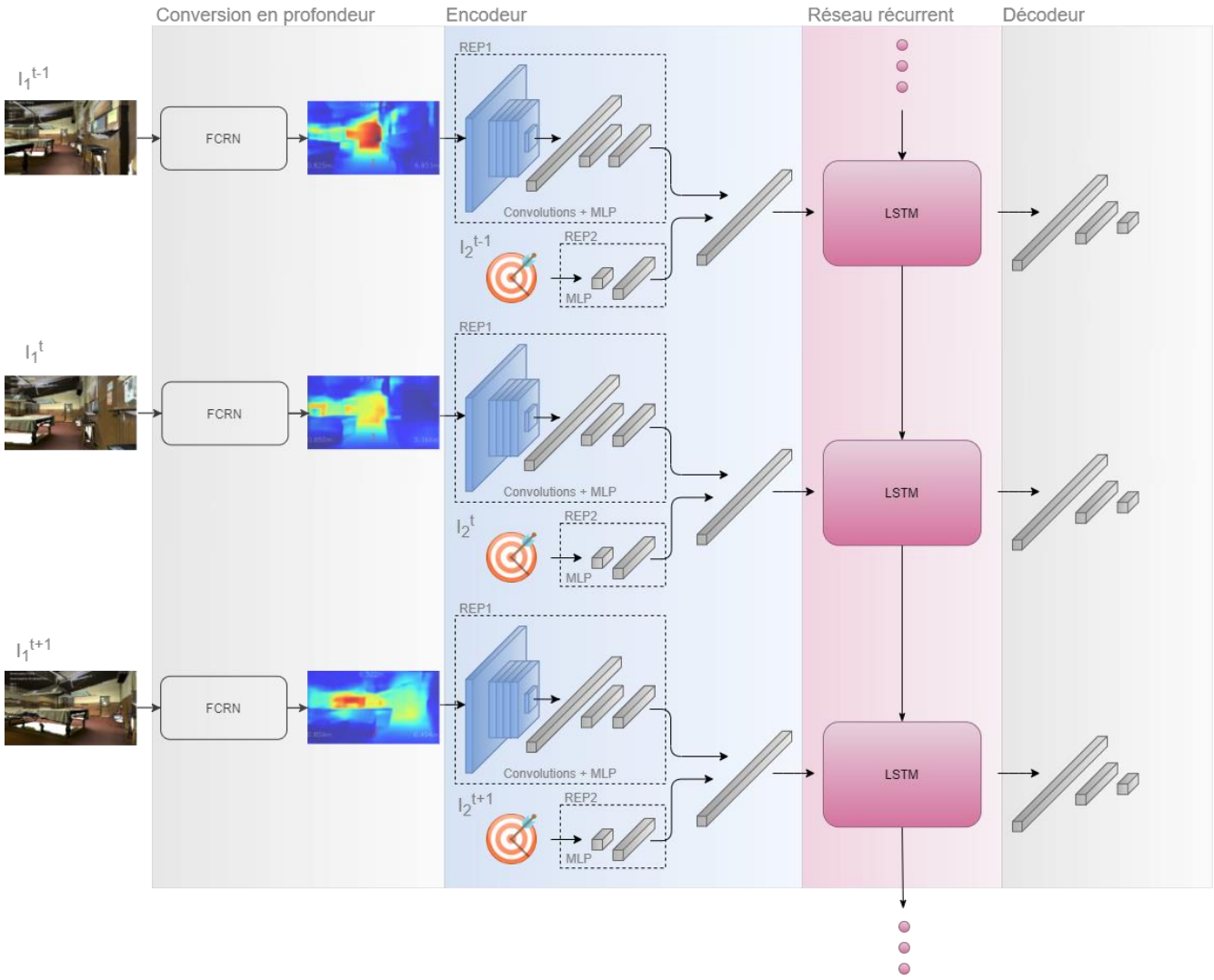


Figure 3.11 Architecture déroulée dans le temps du réseau de guidage autonome pour drones (ADG) Constitué de quatre étapes, le modèle convertit premièrement l'image en image de profondeur. L'*Encodeur* transforme l'image de profondeur et les entrées scalaires en représentation condensée qui est traitée par le *Réseau récurrent* suivi du *Décodeur*.

quant à elles encodées via un simple réseau de type MLP. Les deux réseaux précédents travaillent en parallèle pour encoder une représentation des entrées x_j du modèle ADG. La partie créant la représentation de l'image s'intitule REP1 alors que celle représentant les entrées scalaires s'intitule REP2. La combinaison des deux réseaux constitue l'*Encodeur* du modèle de guidage autonome présenté à la Figure 3.11. Au moment du déploiement du modèle, les images RGB seront converties en images de profondeur via le réseau *FCRN* [52] sélectionné à la section 2.2 et aussi visible à la Figure 3.11.

La nature du problème d'apprentissage par imitation étant Markovienne, il est comme mentionné dans la section 2.1.1.2, que l'hypothèse concernant l'échantillonnage i.i.d. n'est souvent pas respectée [24]. Dans l'optique de minimiser les effets de ce type d'apprentissage, il a été décidé d'utiliser un réseau de neurones récurrent conformément à [28]. Bon nombre de travaux approchent le problème de guidage avec ce type de réseaux dans le cadre d'apprentissage par renforcement [21], [107]–[110], mais seulement [40] l'applique dans un contexte d'apprentissage par imitation. Nous proposons d'utiliser un réseau LSTM et de trouver quelle est la longueur ainsi que le saut d'images («frame skipping») optimal pour les séquences de démonstration. La section comprenant le réseau récurrent LSTM est intitulée *Réseau Récurrent* dans la Figure 3.11. L'architecture du graphe de calcul du LSTM de la Figure 3.11 est déroulée pour montrer les dépendances temporelles entre les entrées et les sorties du LSTM.

Les deux représentations obtenues dans l'*Encodeur* sont concaténées et traitées par le LSTM. La sortie du LSTM doit ensuite être transformée en commande de guidage pour le drone. Une dernière section au réseau appelée *Décodeur* remplit cette tâche en utilisant un MLP. La sortie finale du modèle est un vecteur de cinq dimensions. Une fois la fonction *Softmax* appliquée à ce vecteur, celui-ci contiendra les probabilités d'exécuter chacune des cinq classes établies à la section 3.4.2.

Les sections qui suivent expliquent en détail les particularités des différents modules mentionnés ci-haut.

3.5.1 Traitement d'image

Comme mentionné dans la section précédente, le modèle *FCRN* [37] calcule les images de profondeur une fois le modèle final déployé. Il parvient à convertir l'image RGB en image de profondeur en 0.1 seconde. Le modèle *FCRN* n'est pas utilisé durant l'entraînement puisque l'ensemble de données contient déjà les images de profondeur. Ce choix de méthodologie a été fait afin de limiter les ressources et le temps nécessaire à l'entraînement. Le modèle *FCRN* est donc conservé comme tel et utilise les poids du modèle entraîné par son auteur.

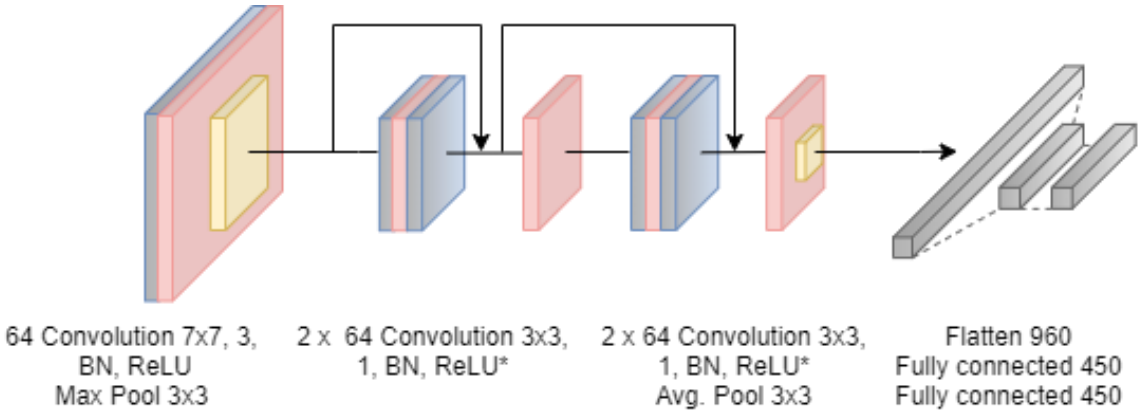


Figure 3.12 Vue détaillée du CNN REP1 de l'*Encodeur*. Le * signifie que les sauts de connexions sont avant l'activation ReLU.

La partie REP1 de l'*Encodeur* a été reprise et adaptée du travail [97]. L'adaptation concerne la taille des filtres de convolution décrits dans la Figure 3.12 . L'architecture du CNN est simple et prend avantage des sauts de connexion («skip connections») [37]. Les *feature maps* obtenus à la fin de la section de convolution sont aplatis et transformés en un vecteur de représentation à 450 dimensions.

L'entraînement du CNN REP1 se fait en deux étapes. La première étape consiste à apprendre une représentation simplifiée de l'image de profondeur minimisant la perte d'information. Pour arriver à cette fin, un autoencodeur a été utilisé. Un autoencodeur appartient à la catégorie des techniques d'apprentissage non supervisé. Il est constitué d'un encodeur et d'un décodeur et a comme objectif de recréer une entrée quelconque. Dans le cas présent, la tâche a été de recréer l'image de profondeur initiale. Les autoencodeurs fonctionnent en apprenant une représentation condensée de l'entrée (l'image) et en décodant cette représentation afin de retrouver l'entrée sous sa forme initiale. L'apprentissage d'une représentation est forcé en diminuant la grandeur du vecteur de représentation par rapport au nombre d'entrées.

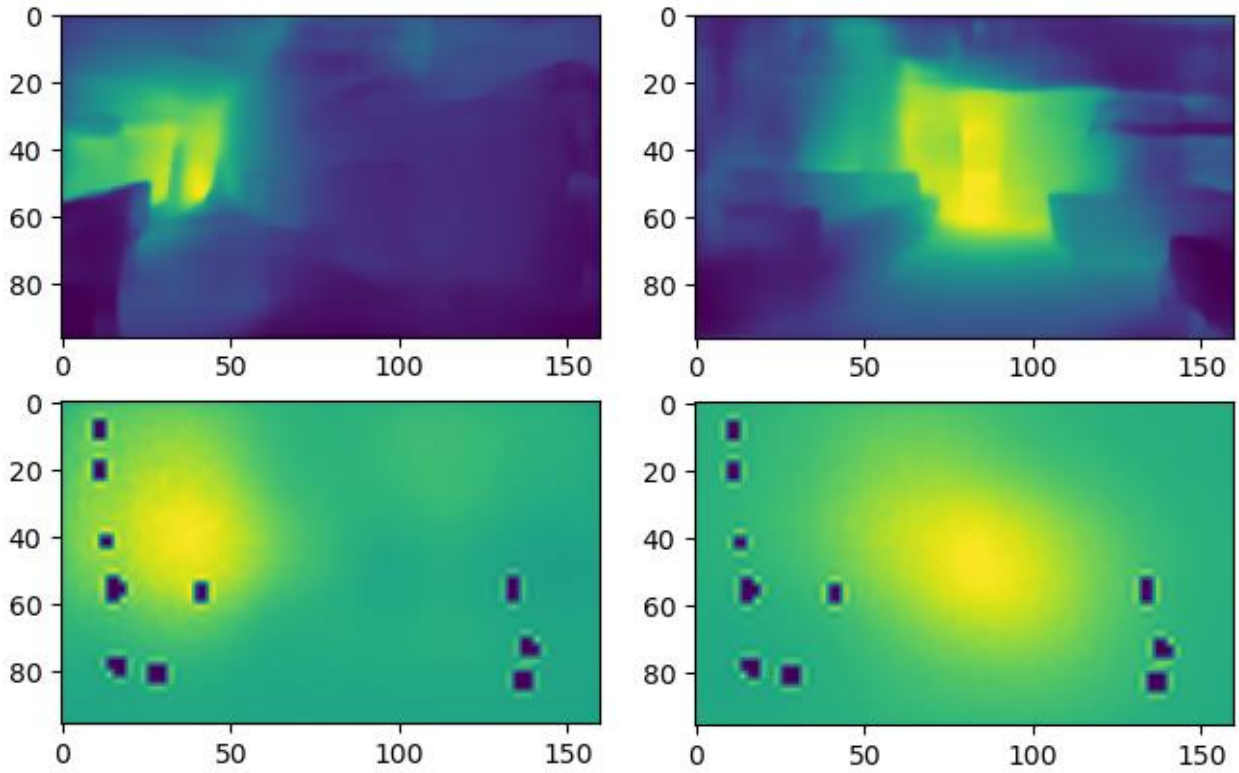


Figure 3.13 Exemple de reconstruction des images de profondeur. Deux prises de vue de profondeur (haut) et leur reconstruction respective (bas). Les axes sont en pixels.

Dans ce cas-ci, l'image 96x160 équivaut à 15360 entrées individuelles et est représentée par un vecteur de 450 dimensions dans la partie encodeur. L'autoencodeur implémenté utilise l'*Encodeur* du modèle principal et un décodeur simple fait sur mesure. Le modèle de l'*Encodeur* est alors préentraîné à la tâche de représentation. La Figure 3.13 montre quelques exemples de reconstruction d'images permettant de caractériser les performances de l'autoencodeur qualitativement. Il est possible de remarquer que quelques régions des images reconstruites apparaissent comme des tâches constantes. Ces tâches sont dues à une modification des déconvolutions (ou «up-convolutions») du décodeur et n'influencent pas la représentation des images initiales. Il est à noter qu'il n'est pas nécessaire d'obtenir une copie conforme de l'image initiale dans le cadre de ce problème. Une image reconstruite affichant l'intensité de la profondeur aux bons endroits tel qu'obtenu est satisfaisante.

La deuxième phase de l'entraînement se déroule en même temps que l'entraînement du modèle ADG. Cependant, un *learning rate* beaucoup plus faible est employé afin de ne permettre qu'une

petite modification du REP1. Autrement, l'encodage de l'image pourrait être réappris de façon différente car rien n'assure que la représentation contient les éléments essentiels comme c'est le cas avec un autoencodeur. Il deviendrait alors difficile pour le LSTM de capturer l'aspect temporel du problème si la représentation change constamment durant l'entraînement.

3.5.2 Représentation des entrées scalaires

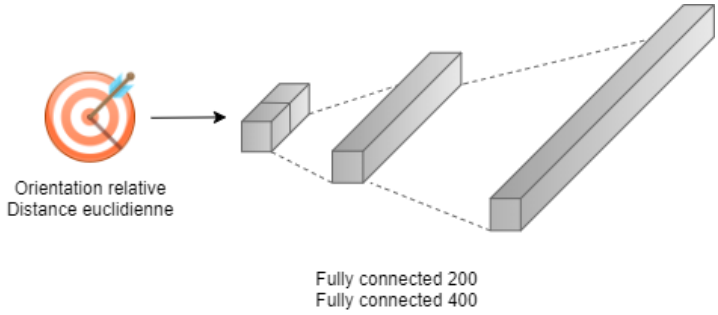


Figure 3.14 Architecture du réseau de guidage autonome pour drones (ADG)

La seconde partie de l'*Encodeur*, REP2, est beaucoup plus simple. La représentation des entrées scalaires est constituée d'un MLP à deux couches. La fonction tangente hyperbolique (*tanh*) a été utilisée comme fonction d'activation entre les deux couches. La raison est simplement que la première des entrées scalaires représente l'orientation relative à l'objectif normalisée entre -1 et +1 tel qu'indiqué dans le Tableau 3.8. Le signe négatif pourra ainsi être conservé dans la représentation des entrées scalaires contrairement au cas où la fonction ReLU avait été utilisée. La fonction *Leaky ReLU* a également été testée,maj mais n'a pas su obtenir de bonnes performances. Le vecteur de représentation, ou *feature map*, passe des deux valeurs scalaires, à une représentation de 200 dimensions puis une représentation de 400 dimensions.

La seconde entrée scalaire utilisée dans le modèle ADG final est la norme de la distance relative à la destination à atteindre. Il a été observé que de fournir la distance relative décomposée en x et y nuisait à la performance du modèle.

Les deux types d'entrées combinés représentées par la cible de la Figure 3.14 constituent l'information nécessaire pour que le drone prenne connaissance de sa position par rapport à sa destination. Le vecteur de représentation est donc de 850 dimensions à la fin de l'*Encodeur*.

3.5.3 Traitement temporel des entrées

Le réseau récurrent employé pour le traitement temporel des données est le LSTM. Comparativement à son compétiteur, le «Gated Reccurent Unit» (GRU), le LSTM est légèrement plus performant à mémoriser les relations à long terme [38]. Le LSTM choisi prend donc en entrée le vecteur de 850 dimensions de représentation des deux entrées concaténées. Le vecteur est transformé en un vecteur de 1000 dimensions et est traité par une cellule classique de LSTM. À chaque instance temporelle, un vecteur de représentation interne («hidden state») est calculé et sera envoyé à la prochaine partie du modèle.

3.5.4 Interprétation de la classe

L'interprétation de la classe s'effectue à la toute fin du modèle et se fait à partir du vecteur de représentation interne du LSTM. Ce dernier est passé à un dernier MLP à deux couches passant de 1000 à 250 dimensions et de 250 à 5 dimensions. Tel que mentionné plus haut, le vecteur de 5 dimensions représente le vecteur de probabilité pour chaque classe une fois que la fonction d'activation *softmax* lui est appliquée.

Afin de faciliter l'entraînement, une fonction *dropout* a été appliquée après la première couche du MLP durant l'entraînement. La fonction *dropout* est une méthode de régularisation connue pour son potentiel à minimiser l'*overfitting* [135]. La méthode agit comme une méthode d'*Ensemble* en désactivant aléatoirement des neurones dans le MLP [136]. En appliquant *dropout* à la couche précédant le calcul des classes, on se trouve à essayer de forcer le modèle à prédire d'autres classes que celle qui aurait normalement été prédite.

Dans un contexte d'apprentissage par imitation pour le guidage, cette méthode s'est avérée cruciale dans l'obtention de bons résultats. L'ensemble de données DGD est énormément déséquilibré tel que montré à la Figure 3.10 en raison du fait que la majorité du temps l'expert s'oriente dans l'espace de façon à aller tout droit le plus longtemps possible. La classe AVANT est donc majoritaire dans DGD, suivi des classes GAUCHE et DROITE puisque ces classes sont plus souvent utilisées lors de changements de direction majeurs. Enfin, les classes AVANT-GAUCHE et AVANT-DROITE se trouvent en faible minorité en raison de leur utilisation seulement en cas de correction de trajectoire. C'est pourquoi forcer le modèle à explorer d'autres options peut inciter à associer des comportements bénéfiques à certaines situations rencontrées.

3.6 Fonction de coût

La fonction de coût employée initialement pour l'entraînement du modèle est la fonction *cross entropy* multi classe conformément à l'équation (3.1). Cette fonction de coût est majoritairement utilisée pour des problèmes de classification multiclasse. Elle a également la capacité d'ajuster les poids attribués au coût de chaque classe tel que démontré par l'équation (3.2), ce qui a pour effet de combattre le problème d'équilibrage du nombre d'éléments de chaque classe. Les poids qui ont été calculés initialement étaient proportionnels à l'inverse du ratio du nombre d'occurrences de chaque classe par rapport au nombre total d'items dans l'ensemble de données. Les poids ont ensuite été ajustés manuellement de façon à améliorer les performances de classification.

$$CrossEntropy = -\sum_{c=1}^{M} y_{o,c} \log(p_{o,c}) \tag{3.1}$$

$$CrossEntropy = -\sum_{c=1}^{M} \beta_c y_{o,c} \log(p_{o,c}) \tag{3.2}$$

M: Nombre de classes

log : Logarithme en base naturelle

y : Indicateur binaire (0 ou 1) si la classe c est celle correspondant à la classe observée o

p : Probabilité prédite que l'observation o appartienne à la classe c

β : Poids pour la pondération de chaque classe c [137]

Désirant améliorer les performances obtenues avec la fonction de coût *cross entropy* et en s'inspirant des fonctions de coût (récompense) utilisées pour l'apprentissage par renforcement, plusieurs éléments enrichissant le *cross entropy* ont été ajoutés. La pondération de contexte, le coût de la distance et de l'orientation ainsi que la pondération de fin de trajectoire seront détaillés dans les sous-sections suivantes.

3.6.1 Pondération du contexte $w_{contexte}$

La pondération du contexte s'inspire du travail de Kahn et al. (2020) qui révèle comment Skydio [138] a entraîné leur drone par apprentissage par imitation. La technique appelée *CEILing* prétend pouvoir accélérer l'apprentissage en apprenant ce qui est important pour l'expert qui a fait les démonstrations plutôt que de simplement imiter ce que l'expert a fait [139]. La façon de procéder est de pondérer chaque exemple afin de pénaliser plus sévèrement ceux qui ont une importance critique.

Une fois transposée dans un problème de guidage, cette façon de procéder prend la forme de poids plus faibles pour des exemples où aucun obstacle ne pose un risque pour le drone, et un poids élevé lorsque le drone navigue dans un environnement restreint dans l'espace. Ainsi, il devient acceptable pour le drone de déroger légèrement de la trajectoire en milieu ouvert, mais obligatoire de suivre une trajectoire stricte en milieu restreint.

Dans le problème de guidage, ci-présent, basé sur la profondeur, il est facile de convertir une image de profondeur en un poids représentant le niveau de liberté du drone à suivre la politique de l'expert. Une observation attentive de la variation de la moyenne et de la médiane dans le temps révèle que la médiane des images en milieux restreints diminue significativement. Il a été possible de moduler une fonction pondérant ces exemples de façon à grandement améliorer l'entraînement. Pour chaque exemple ayant une médiane inférieure à un seuil a fixé empiriquement, le ratio de la médiane avec à la moyenne de l'ensemble de donnée est soustraite à une valeur unitaire. Le résultat de cette opération est additionné avec une valeur unitaire initiale avant d'être mis à la puissance de quatre tel que montré dans l'équation (3.7).

$$w_{contexte}(x) = \begin{cases} \left(1 + \left(1 - \frac{\tilde{x}}{\mu_x}\right)\right)^4, & \tilde{x} \leq a \\ 1, & \tilde{x} > a \end{cases} \tag{3.3}$$

$w_{contexte}$: Pondération de contexte d'une image de profondeur

$\tilde{x}$: Médiane d'une image de profondeur normalisée

μ_x : Moyenne d'une image de profondeur normalisée

a : Seuil empirique

3.6.2 Coût de distance $\boldsymbol{C_d}$

La distance est un des facteurs caractérisant les performances de guidage du drone. Cette distance diminue lorsque la bonne décision est prise lors du guidage. Dans certaines circonstances, comme pour sortir d'un cul-de-sac par exemple, la distance devra augmenter pour pouvoir diminuer par la suite. Le modèle doit apprendre à faire cette distinction avec l'entrée représentant la distance. L'introduction d'un coût pour la distance forcera le modèle à apprendre à minimiser cette distance si pour chaque séquence, la distance finale est plus près de l'objectif que la distance initiale, ce qui est le cas pour la majorité des exemples. Le coût de distance prend la forme du ratio entre la distance relative actuelle et la distance relative initiale du segment analysé similairement à [140] et conformément à l'équation (3.4).

$$\boldsymbol{C}_d = \frac{\boldsymbol{d}}{d_0} \tag{3.4}$$

$\boldsymbol{C}_d$: Vecteur du coût de distance d'une séquence

$\boldsymbol{d}$: Vecteur des distances euclidienne relative d'une séquence (m)

d_0 : Premier élément du vecteur $\boldsymbol{d}$ (m)

3.6.3 Coût d'orientation $\boldsymbol{C_o}$

L'orientation du drone est critique à l'atteinte de l'objectif. Il est donc naturel qu'une composante de coût total d'une séquence soit reliée à l'orientation. L'orientation étant déjà une entrée normalisée entre -1 et +1, il est facile de transformer cette entrée en coût selon l'équation (3.5). Un

multiplicateur trouvé empiriquement multiplie simplement la valeur absolue de l'entrée correspondant à l'orientation. Comme l'orientation est reliée intrinsèquement à la minimisation de la distance, le multiplicateur doit s'assurer que le coût d'orientation est plus important que le coût de distance.

$$\boldsymbol{C_o = 2 \cdot |o|} \tag{3.5}$$

$\boldsymbol{C_o}$: Vecteur du coût d'orientation

$\boldsymbol{o}$: Vecteur d'orientation relative normalisé

3.6.4 Coût des fins de trajectoires $\boldsymbol{C_{ft}}$

La fin d'une trajectoire est aussi très importante, car elle occasionne des changements importants au niveau de l'orientation relative à l'objectif. En effet, lorsque le drone effectue l'approche finale de l'objectif, une légère erreur dans l'orientation grandira de façon exponentielle à mesure que la composante de la vitesse perpendiculaire à l'objectif augmente. Éventuellement, si la distance radiale à l'objectif est trop élevée, il se peut que le drone dépasse l'objectif et doive rebrousser chemin. Dans des situations où l'objectif se trouve dans l'entrée d'une pièce et que le drone doit longer un mur avant d'effectuer un virage, la pondération de la fin de trajectoire permet de capturer plus précisément les intentions de l'expert vis-à-vis le moment opportun du virage. Les exemples faisant partie de l'ensemble de données ayant une distance relative inférieure à 3 mètres sont conséquemment se voient ajouter un coût supplémentaire trouvé empiriquement tel qu'illustré à l'équation (3.6).

$$\boldsymbol{C_{ft}} = \begin{cases} 2.5, & d_i < 3 \\ 1, & d_i \geq 3 \end{cases} \tag{3.6}$$

$\boldsymbol{C_{ft}}$: Vecteur coût de fins de trajectoires

d_i : Élément i du vecteur de distances euclidiennes (m)

3.7 Entraînement

L'entraînement du modèle se fait de façon classique avec les outils de PyTorch. L'optimiseur de choix pour les réseaux de neurones utilisant des modules récurrents est Adam. Adam est initialisé avec les moments betas par défaut de 0.9 et 0.999. Le modèle est entraîné avec des *learning rates* adaptés aux différents modules tel que présenté dans le Tableau 3.9. L'optimiseur Adam est aussi muni d'un paramètre *weight decay* pondérant la normalisation L2 à 0.01.

Tableau 3.9 *Learning rates* des différents modules du modèle ADG

Module	*Learning Rate*
REP1	0.00001
REP2	0.0001
LSTM	0.001
Classificateur	0.001

Un *learning rate scheduler* est aussi implémenté afin de contrôler la diminution des *learning rates* avec le nombre d'*epochs*. Ce dernier divise par 10 les *learning rates* à chaque 20 *epochs*. Le tout est entraîné sur une carte graphique Nvidia 1080Ti. Chaque entraînement est appelé expérience et est enregistrée à l'aide de l'outil Comet ML [141]. Comet permet l'enregistrement des courbes d'apprentissage ainsi que de nombreuses autres données telles que le code principal, des images, des fichiers textes, etc. Pour ce projet, une matrice de confusion ainsi qu'un fichier texte contenant le code de la fonction de coût ont été sauvegardés en plus des courbes d'apprentissage.

Quelques autres particularités de l'entraînement sont expliquées en détail dans les sous-sections suivantes.

3.7.1 Utilisation de l'ensemble de données

Lors de l'entraînement du modèle, l'ensemble de données doit être pris en charge par l'objet *Dataset* appartenant à la librairie PyTorch. Une classe personnalisée héritant de l'objet *Dataset* est créée afin de gérer l'utilisation de l'ensemble de données constitué des fichiers HDF5 de chaque

trajectoire. La classe en question est nommée *DGD* (pour «Drone Guidance Dataset») et crée une table contenant l'information nécessaire pour accéder à tous les segments de chaque trajectoire.

Les trajectoires sont segmentées puisque les réseaux récurrents sont en général en mesure de retenir l'information entre 30 et 50 itérations dépendamment du degré de complexité de la tâche à accomplir. Considérant qu'il est question de traiter une représentation complexe d'une image jumelée à une direction et une distance, il a été nécessaire de limiter la longueur des segments lors de l'entraînement. La table regroupant les informations sur les segments consiste en une liste de tous les segments possibles pour chaque fichier de l'ensemble de donnée. Les segments sont représentés comme un point de départ dans un fichier HDF5 correspondant à un temps t et un nombre d'instants n subséquents au temps de départ.

Grâce aux fonctions *pack_padded_sequence* et *pad_packed_sequence* de PyTorch, il est possible de fournir des séquences de longueur différentes à un réseau récurrent. Cette fonctionnalité est particulièrement utile dans le cas où une trajectoire ou le dernier segment d'une trajectoire a moins de n instants.

La longueur des segments est, à moins de correspondre à l'une des deux situations mentionnées ci-haut, contrôlée par deux hyperparamètres : le nombre d'images n ainsi que le nombre de sauts d'images («skipped frames») s. Le nombre de sauts d'image s signifie qu'une image sur s sera conservée parmi les n images. Le nombre total N d'images utilisées par segment est donc régi par l'équation (3.7).

$$N = floor\left(\frac{n}{s}\right) \tag{3.7}$$

N : Nombre total d'images utilisées par segment

n : Nombre d'images total

s : Nombre de sauts d'images

Au moment de l'entraînement, le *DataLoader* de PyTorch utilisera l'objet *DGD* afin de charger aléatoirement et efficacement les trajectoires segmentées sous forme de tenseur dans une *batch.*

3.7.2 Augmentations de l'ensemble de données

L'ensemble de données peut être augmenté de façon à accroître artificiellement son nombre d'éléments. Les techniques d'augmentation permettent d'augmenter la robustesse du modèle ainsi que sa capacité de généralisation en présentant les données de l'ensemble d'entraînement sous différentes formes (images floutées, recadrées, renversées, etc.) [142]. Dans le cas de l'ensemble de données DGD, il convient de noter que parmi les 624 trajectoires, il existe 624 segments dans lesquels le drone arrive à destination. Une fois segmenté selon la procédure expliquée à la section 3.7.1, l'ensemble de données possède environ 3000 segments, ce qui veut dire que le drone atteint sa destination dans seulement 20% des segments. L'une des augmentations proposées est donc de simuler l'arrivée à la destination artificielle (DA) pour certains segments sélectionnés aléatoirement.

Afin de simuler l'arrivée à la destination, il est nécessaire de réajuster les positions relatives du drone par rapport au dernier instant du segment. L'orientation relative à l'objectif peut également être recalculée à partir des informations contenues dans l'ensemble de données original.

Dans l'éventualité où l'expert a effectué plus de virages d'un côté que de l'autre, il est bénéfique de pouvoir rétablir l'équilibre en renversant aléatoirement une séquence. L'image ainsi que l'orientation sont donc renversées aléatoirement de façon à simuler une séquence miroir (SM). Cela veut donc dire qu'il y a moins de chances que le modèle s'habitue aux différents parcours.

Une analyse des performances des deux méthodes est présentée au Chapitre 4.

3.7.3 Optimisation des hyperparamètres

Les hyperparamètres sont les paramètres influençant le modèle ou la façon dont le modèle apprend, mais qui ne sont pas appris automatiquement dans le processus de *backpropagation*. Afin de bien mesurer les performances du modèle entraîné, il est nécessaire de valider et de tester le modèle sur une partie de l'ensemble de données non utilisée lors de l'entraînement. La raison est que dans certains cas le modèle peut parvenir à apprendre exactement la distribution de l'ensemble de données d'entraînement tout en diminuant ses performances sur de nouveaux exemples.

Cette caractéristique connue peut être prévenue en utilisant un ensemble de données de validation sur lequel on calcule les performances après chaque *epoch* de l'entraînement. Au moment où les performances en coût diminuent sur l'ensemble de validation, l'entraînement est arrêté. Enfin, le

modèle final est testé sur un autre ensemble de données jamais observé jusqu'à présent dans le cycle d'entraînement afin de s'assurer que les performances sont similaires à celles de l'ensemble de validation. Tel que démontré dans [142] [143], une bonne séparation d'un ensemble de données est requise pour des tâches de prédiction. La proportion de l'ensemble de données utilisée pour chaque ensemble est aux alentours de 80%, 10% et 10% pour l'entraînement, la validation et le test respectivement.

Dans le contexte de ce projet, l'ensemble de données est constitué d'une multitude de parcours exécutés par un expert humain dans plusieurs environnements virtuels différents. Le début et la fin de chaque parcours sont déterminés aléatoirement et certains segments d'une trajectoire peuvent se ressembler si le drone passe à des endroits communs. Par souci d'utiliser des ensembles de validation et de test assurément différents de ceux utilisés pour l'entraînement, un environnement complet ayant une quantité significative de trajectoires a été retiré de l'ensemble d'entraînement. L'environnement en question est le *Bar à Billard* du Tableau 3.4. et présenté à la Figure 3.15. Il est simple, mais peut être complexifié considérablement en fonction de l'altitude à laquelle le drone se déplace. Les trajectoires sont séparées en deux ensembles de validation et de test ayant les proportions de 70% et de 30% respectivement pour assurer une meilleure estimation durant la validation.

Une dernière phase de test est mise en place dans l'environnement de simulation utilisé pour générer l'ensemble de données. Cette fois, c'est plutôt le modèle de guidage pour drones ADG qui pilotera le drone aux mêmes destinations aléatoires du *Bar à Billard*. D'autres tests ont été exécutés dans deux autres environnements. Les résultats de ces expérimentations sont présentés au Chapitre 4.



Figure 3.15 Vue isométrique (gauche) et orthogonale supérieure (droite) du *Bar à Billard* [145].

CHAPITRE 4 RÉSULTATS ET DICUSSION

Le modèle de guidage autonome pour drones a été entraîné avec les hyperparamètres présenté à la section 3.7 jusqu'à l'obtention d'un minimum local satisfaisant par les performances en test. Il est reconnu que forcer l'atteinte d'un minimum global est en pratique synonyme d'*overfitting* [146]. Les performances ont été jugées satisfaisantes lorsque la précision sur l'ensemble de test a dépassé la barre des 80%. Elles ont par la suite été confirmées par les performances en simulation. Une recherche d'hyperparamètres du type *grid search* a ensuite été effectuée afin de déterminer quelle proportion des deux méthodes d'augmentation de l'ensemble de données obtiendrait les meilleurs résultats. Les résultats de l'entraînement et de simulation obtenus sont présentés dans les deux sections suivantes. Les résultats de l'entraînement seront évalués afin de déterminer la validité des modèles obtenus en termes d'optimisation alors que les résultats de simulation permettront d'évaluer les performances réelles des modèles.

4.1 Résultats de l'entraînement

4.1.1 Analyse des tendances

L'entraînement s'est déroulé avec succès pour toutes les combinaisons de techniques d'augmentation. Le critère de succès est l'atteinte d'une précision sur l'ensemble de test supérieure à choisir aléatoirement la classe. Dans le cas de l'environnement *Bar à Billard*, il est possible d'obtenir une précision de 68.9% si le modèle choisit toujours la classe AVANT en raison du débalancement. Obtenir une précision supérieure à celle-ci est donc considérée, dans ce travail de recherche, comme un succès. La variation de l'erreur se comporte de façon attendue lors de l'entraînement considérant les pondérations accordées aux modules de la fonction de coût. On remarque de même à la Figure 4.1 que les courbes d'erreur et de précision atteignent un plateau vers la fin de l'entraînement, signifiant qu'un minimum local satisfaisant selon les performances en précision sur l'ensemble de test a été atteint.

Pour ce qui est des courbes d'erreur, l'atteinte d'un plateau pour l'ensemble de validation est signe que l'entraînement s'est déroulé sans *overfitting* [147]. Les courbes d'erreur pour l'ensemble d'entraînement (A) et de validation (B) de la Figure 4.1 se comportent de façon très similaire à l'exception du fait que celle de validation semble décalée vers le bas. Tel que résumé au Tableau

4.1, l'erreur d'entraînement pour les différents modèles converge en moyenne à 3.96 sans unités (s.u.) alors que l'erreur de validation converge en moyenne à 3.71. Cette différence peut s'expliquer facilement en raison de l'utilisation de masques *dropout* dans le MLP effectuant la classification. La technique *dropout* est seulement appliquée lors de l'entraînement. Les masques ont un effet négatif sur l'erreur puisqu'ils inhibent aléatoirement des neurones, ce qui explique que l'erreur sur l'ensemble de validation converge en moyenne plus basse que celle d'entraînement. En raison de la taille de l'ensemble de test par rapport à l'ensemble de validation, il est normal que la moyenne de l'erreur en test possède un écart type plus élevé englobant la moyenne de validation.

Tableau 4.1 Statistiques sur les erreurs obtenues pour la recherche d'hyperparamètres (s.u.)

	Moyenne	Écart type	Intervalle
Entraînement	3.96	0.16	3.74-4.24
Validation	3.71	0.07	3.62-3.84
Test	3.66	0.20	3.35-4.03

De plus, l'ensemble de validation peut être caractérisé comme étant plus simple que plusieurs des environnements utilisés dans l'ensemble de données d'entraînement. Comme mentionné précédemment à la section 3.7.3, l'ensemble de données a dû être divisé afin de construire un ensemble de données d'entraînement, de validation et de test. Par souci d'éviter le phénomène d'*overfitting*, un des environnements possédant 14.11% des trajectoires a été retiré de l'ensemble de données et destiné à valider et tester le modèle. N'ayant pas accès à une grande quantité d'environnements, il a été nécessaire de choisir entre un environnement ayant plus de trajectoires et étant plus complexe ou un environnement ayant moins de trajectoires et moins complexe.

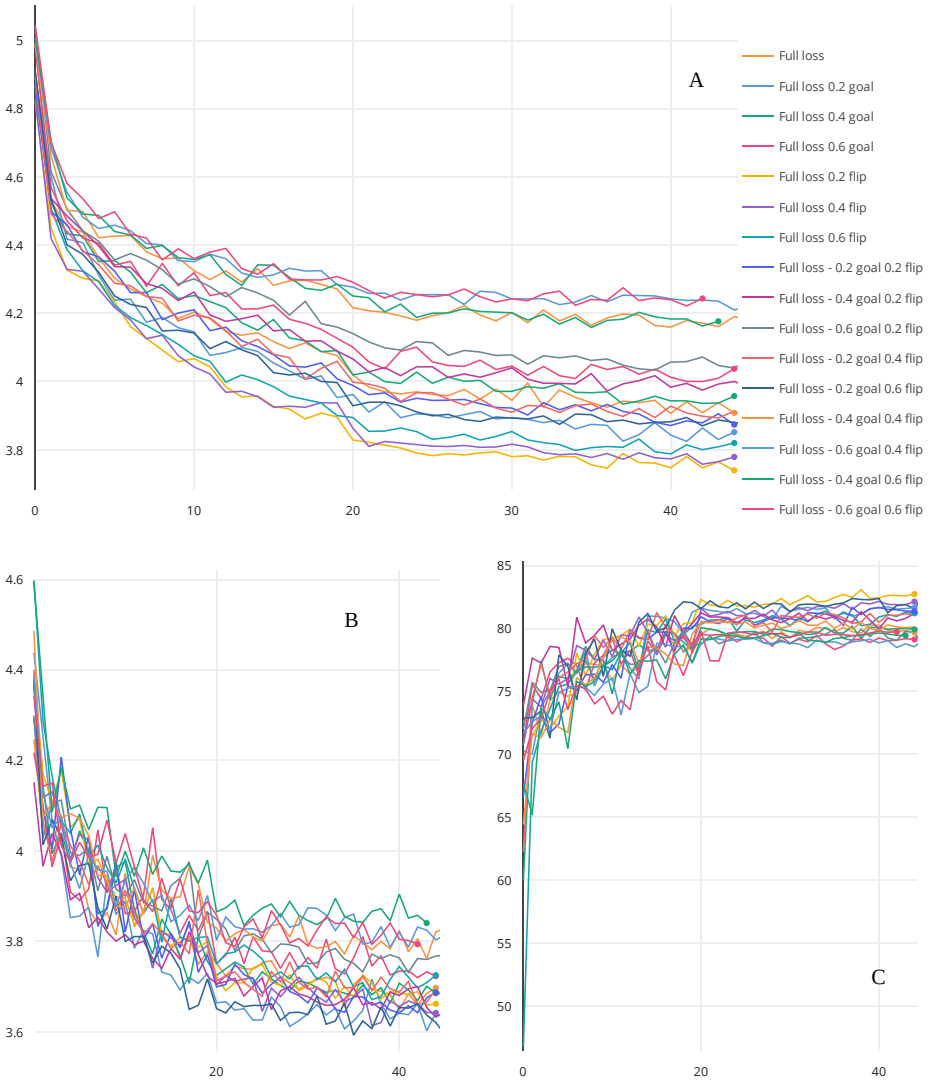


Figure 4.1 Courbes d'apprentissage de la recherche d'hyperparamètres. L'axe des ordonnées correspond à l'erreur d'entraînement, l'erreur de validation et la précision de validation pour les graphiques A, B, C respectivement alors que l'axe des abscisses correspond au nombre d'*epochs*.

Puisqu'un environnement possédant plus de trajectoires que nécessaire aurait dégradé les performances du modèle final, celui ayant une plus petite taille et étant plus simple a été sélectionné. La complexité de l'environnement fait ici référence à l'arrangement spatial et donc au niveau de l'encombrement et la difficulté de guidage. Cette méthodologie permet d'optimiser l'utilisation d'un ensemble de données lorsque sa taille est limitée. Une procédure de validation croisée à *k*-blocs permettra de valider les performances obtenues.

Dans les meilleurs cas, la précision de classification atteint 83% lors de la validation. Contrairement à 78% dans les pires scénarios. Ces résultats de classification sont considérés comme étant excellents dans cette étude malgré leur faible précision comparativement aux modèles de classification connus tels que FixEfficientNet-L2 [148] sur ImageNet [36]. Trois raisons principales expliquent pourquoi cette précision est atteinte et pourquoi elle est acceptable

En premier lieu, l'ensemble de données qui a été créé n'est probablement pas optimal. La structure et le lien entre les données doit être impeccable et il presque impossible qu'un ensemble de données optimal ait été créé malgré les précautions employées lors de la réalisation de DGD. En comparaison, les ensembles de données tels que ImageNet sont le fruit d'énormes travaux de recherche.

La seconde raison découle du fait que le problème de guidage orienté est résolu à l'aide d'un modèle de classification. Bien que l'aspect temporel soit géré par un module récurrent dans le modèle, il est tout de même possible que le modèle décide d'effectuer un virage prématurément comparativement à l'expert. Quelques fractions de seconde de décalage entre l'action de l'agent et celle de l'expert peuvent expliquer une baisse de performance en précision lors de l'entraînement, la validation ou le test du modèle. Par exemple, les cercles rouges de la Figure 4.3 montrent deux moments dans les trajectoires qui sont marqués par les mêmes actions mais, qui ont été exécutés après avoir parcouru une distance différente. Dans ces circonstances, la commande envoyée par l'agent autonome est probablement tout autant justifiée que celle de l'expert. Tant et aussi longtemps que les commandes de l'agent n'occasionnent pas de collisions et ne dévient pas significativement de la trajectoire sous-optimale de l'expert, l'agent ne devrait pas être pénalisé en termes d'erreur d'entraînement ou de précision.

La troisième et dernière raison est en lien avec le choix des classes du problème de classification. La section 3.4.2 explique le choix des cinq classes utilisées dans ce problème. Les cinq classes sont : présentées au Tableau 3.5. Les classes AVANT-GAUCHE et AVANT-DROITE avaient été introduites par souci d'efficacité du guidage sachant qu'il pouvait être difficile de les distinguer des classes GAUCHE et DROITE. Afin de quantifier le niveau auquel les deux classes supplémentaires sont confondues avec leur classe adverse, une matrice de confusion a été construite pour chaque phase de test des modèles. La Figure 4.2 montre un exemple typique de matrice de confusion pour les modèles de guidage autonomes obtenus lors de la recherche d'hyperparamètres.

On remarque immédiatement le débalancement des classes mentionné à la section 3.5.4. Dans le cas où la distribution des classes aurait été égale, la matrice de confusion devrait posséder une diagonale colorée en foncé. L'axe des ordonnées correspond à la vraie classe alors que l'axe des abscisses correspond à la classe prédite. Ainsi, aucune prédiction ne devrait se trouver ailleurs que sur la diagonale dans le cas d'une classification parfaite. Or, on remarque la diagonale est

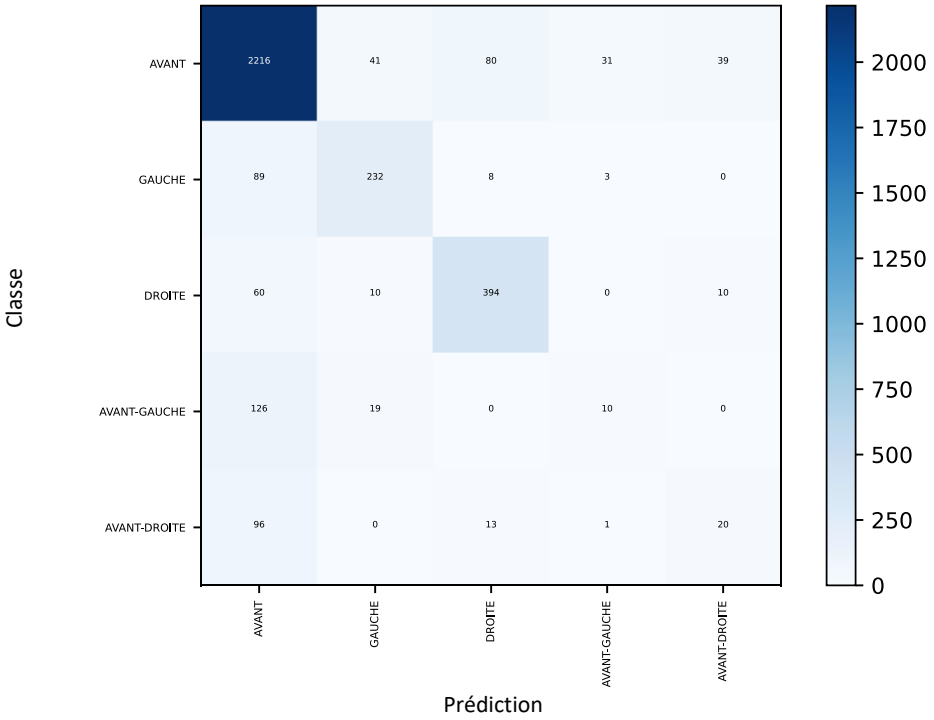


Figure 4.2 Exemple de matrice de confusion typique

seulement plus prononcée pour les trois classes les plus présentes dans l'ensemble de données : AVANT, GAUCHE et DROITE. Lorsque la classe à prédire est AVANT-GAUCHE ou AVANT-DROITE, le modèle semble avoir beaucoup de difficulté à atteindre ce résultat.

Dans le cas de la ligne représentant la classe AVANT-GAUCHE, la classe AVANT est plutôt prédite 81% du temps, la classe GAUCHE 12% et la classe AVANT-GAUCHE seulement 6%. Similairement pour la ligne représentant la classe AVANT-DROITE, la classe AVANT est prédite 74% du temps, la classe DROITE 10% et la classe AVANT-DROITE 15%.

Les autres classes sont majoritairement bien classifiées par le modèle, ce qui confirme que le débalancement de l'ensemble de données rend difficile, pour le modèle, de distinguer la bonne

classe des autres plus fréquentes. Par exemple, il semble extrêmement difficile de distinguer quand l'expert effectue un virage léger avec les classes combinées (AVANT-GAUCHE ou AVANT-DROIT) de quand il va tout droit. Ceci est probablement dû au fait que l'utilisation de ces classes est en plus d'être peu fréquente (environ 4% chacune), est souvent très brève et n'occasionne pas de mouvements majeurs dans l'orientation du drone. De plus, lorsque l'expert utilise les classes combinées, le modèle est aussi tenté comme prévu de prédire la classe adverse (GAUCHE ou DROITE). Ce qui peut être signe que l'utilisation d'une classe combinée de façon prolongée s'apparente à l'utilisation d'une classe adverse.

Somme toute, il convient d'affirmer que le débalancement des classes dans l'ensemble de données nuit significativement à la résolution du problème de guidage autonome étudié. Il faut cependant noter que le débalancement est un mal nécessaire en approchant la situation avec une solution impliquant l'apprentissage par imitation. Tel que mentionné précédemment, la majorité des déplacements effectués dans des environnements quotidiens (en voiture, à la maison, dans un édifice, etc.) impliquent une minorité de changements de direction par rapport au temps passé à suivre une trajectoire linéaire tel qu'un corridor ou une route automobile.

4.1.2 Mitigation du décalage du modèle avec l'expert

Deux autres approches ont été abordées dans ce mémoire afin d'évaluer adéquatement les performances du modèle en prenant en compte cette nuance. Initialement, une nouvelle fonction coût a été développée afin de mieux visualiser les parcours générés par l'agent autonome et de les comparer avec ceux générés par l'expert. L'exemple de parcours présenté à la Figure 4.3 montre les deux parcours semblant similaires à première vue, mais qui se terminent à plus de 10 mètres de distance l'un de l'autre. Après une analyse plus poussée des trajectoires, on remarque que le modèle a décidé de tourner légèrement plus tôt que l'expert. Il est aussi possible de remarquer que l'agent effectue tout de même les mêmes changements de direction que l'expert tel qu'indiqué par les cercles rouges de même taille sur la Figure 4.3.

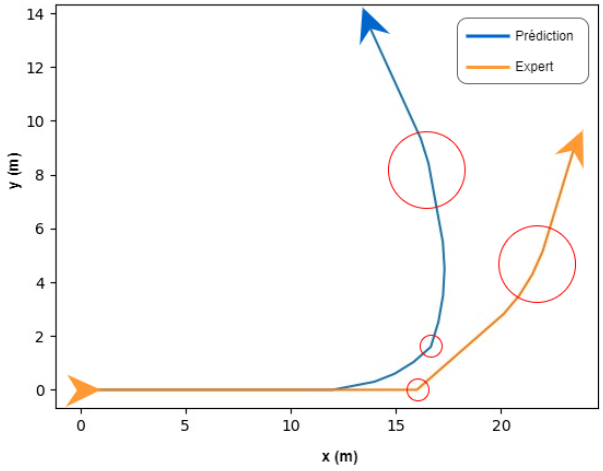


Figure 4.3 Tracé d'une trajectoire de l'expert comparée à celle de l'agent autonome

Ce comportement est explicable du fait que lors de la prédiction, la même séquence d'images est montrée au modèle et ce dernier doit prédire la bonne commande pour imiter l'expert. Il n'y a donc pas de rétroaction visuelle sur la commande qui a été prédite et l'erreur pour cette fonction de coût s'accumule avec le temps. C'est aussi pourquoi le modèle réplique aussi le comportement de l'expert. Dans une validation par simulation, il est fort probable que l'agent intelligent n'ait pas tourné aussi brusquement, tel que mis en évidence par les cercles rouges de la figure, puisque la perception visuelle du drone aura également changé en concordance avec les commandes envoyées. On en conclut que la fonction de coût est erronée puisqu'elle ne respecte pas la dynamique Markovienne de la tâche de guidage selon laquelle l'effet d'une action observable au prochain état.

Même si elle permet d'évaluer qualitativement les performances de l'agent autonome, il est difficile d'implémenter une mesure quantitative de ce comportement autre que l'erreur moyenne de déplacement. L'idéal serait d'effectuer des simulations dans Gazebo pour les mêmes parcours que l'expert afin de quantifier l'erreur. Cette option demanderait beaucoup plus d'organisation et de ressources puisque les modèles sont entraînés sur plusieurs machines et seulement l'une d'entre elles s'exécute sous le système d'exploitation Linux nécessaire à l'utilisation de ROS et Gazebo. De plus, même si les entraînements s'effectuaient sur la machine Linux utilisée dans le cadre de ce mémoire, il n'y aurait pas assez de mémoire graphique pour entraîner un modèle et simuler ce dernier de façon automatique. Le tout supposerait aussi que l'exécution et la communication entre

le programme d'entraînement et le programme de simulation est assez fiable pour gérer les étapes de validation et de test sans mettre en péril les nombreuses heures d'entraînement nécessaires à l'obtention d'un modèle.

La seconde approche destinée à tenir compte du possible décalage entre la prédiction et l'action de l'expert transforme le problème de classification en un problème de prédiction de distribution de probabilité. Une fonction coût exploitant la *Kullback Leibler Divergence* (KLD) a donc été implémentée dans le but d'assouplir la sévérité de la fonction *cross entropy*. L'idée avait comme objectif de moins pénaliser le modèle lorsque la bonne classe est prédite un peu trop tôt ou un peu trop tard dans la séquence temporelle. Ainsi, il serait acceptable pour le modèle de tourner un peu plus tôt que l'expert afin d'éviter un obstacle par exemple. L'implémentation consiste à calculer une distribution de probabilité représentant celle de l'expert dans une fenêtre temporelle à l'aide de la classe y attitrée à chaque exemple de la séquence. La fonction coût n'a malheureusement pas permis d'entraîner un modèle fonctionnel.

4.1.3 Analyse des modèles obtenus

Les tendances relatives aux performances d'entraînement des modèles étant maintenant couvertes, il convient d'analyser les résultats de la recherche d'hyperparamètres. Tel que mentionné précédemment, une stratégie d'optimisation des hyperparamètres du type *grid search* a été employée afin de déterminer quelle combinaison d'augmentation de l'ensemble de données permet d'obtenir le meilleur modèle. La recherche d'hyperparamètres s'est effectuée une fois que les autres hyperparamètres ont été ajustés manuellement. L'effet de l'utilisation des techniques d'augmentation sur les résultats de précision et d'erreur seront donc expliqués dans la présente section.

Les techniques d'augmentation analysées dans ce travail de maîtrise sont la simulation de l'arrivée à une destination artificielle (DA) ainsi que le renversement d'une séquence en séquence miroir (SM). La première méthode consiste à simuler l'arrivée prématurée à une destination à la fin d'une séquence tel qu'expliqué à la section 3.7.2. La seconde méthode renverse l'image et les directions empruntées par l'expert dans une séquence pour que la classe GAUCHE devienne DROITE et vice versa.

Les Tableau 4.2 et Tableau 4.3 présentent les précisions de classification obtenues sur les ensembles de validation et de test par les modèles issus de la recherche d'hyperparamètres. Comme les résultats de précision sont similaires sur les ensembles de validation et de test, on peut conclure que le modèle possède une certaine constance. Les modèles n'ont également pas mieux performé sur l'ensemble de validation, ce qui confirme que la combinaison de tous les hyperparamètres n'a pas engendré d'*overfitting* sur l'ensemble de validation. Il est important de valider qu'aucun *overfitting* n'est observable dans les tendances des courbes de la Figure 4.1 puisqu'il invaliderait l'analyse qui suit.

Tableau 4.2 Précision de classification sur l'ensemble de validation

		Destination artificielle (%)			
		0	20	40	60
Miroir (%)	0	80.3	81.6	80.2	80.5
	20	83.1	81.7	81.7	80.5
	40	82.1	82.4	80.6	79.1
	60	81.2	83.3	79.5	79.7

Tableau 4.3 Précision de classification sur l'ensemble de test

		Destination artificielle (%)			
		0	20	40	60
Miroir (%)	0	80.9	80.1	79.4	80.0
	20	82.1	79.4	76.8	81.3
	40	82.6	82.1	77.1	80.1
	60	82.6	82.1	80.4	77.2

Les Tableau 4.2 et Tableau 4.3 expriment la précision en validation et en test pour différentes combinaisons des techniques d'augmentation. Les lignes correspondent à la probabilité de transformer une séquence en SM alors que les colonnes représentent la probabilité de simuler une DA. Un code de couleur est également utilisé pour mettre en évidence les effets de la combinaison de chaque technique. La couleur verte indique une bonne performance alors que la couleur rouge indique une mauvaise performance

L'analyse portera plus spécifiquement sur les résultats du Tableau 4.3 puisque ce dernier représente mieux les performances du modèle et que l'effet de la variation des hyperparamètres est plus marqué par la variation des teintes de couleurs.

D'abord, il semble que l'augmentation de la probabilité de simuler l'arrivée à une destination artificielle ait un impact négatif sur la précision de classification. Le gradient horizontal de couleur montre que dépassé 20% de probabilité de simuler une arrivée à destination précoce, les performances du modèle en souffrent significativement. Le caractère non linéaire de l'effet de la

combinaison des hyperparamètres est aussi visible dans la colonne correspondant à 40% de DA, spécifiquement pour 20% et 40% de SM. En raison de la taille plus faible de l'ensemble de test, il est normal que le modèle ait moins bien performé sur l'ensemble de test que sur celui de validation.. Les tests en simulation permettront de clarifier la cause de cette différence.

Pour ce qui est de l'effet du renversement des séquences en SM, l'augmentation du taux auquel l'augmentation est appliquée semble corréler avec une augmentation des performances dans les cas où l'augmentation simulant l'arrivée artificielle est faible. Cet effet n'est cependant plus détectable lorsque la probabilité de DA dépasse les 20%.

Les deux modèles les plus performants sur l'ensemble de test sont donc ceux qui ont été entraînés avec une probabilité de 0% de DA, puis 40% et 60% de SM. Les deux modèles ont obtenu respectivement 82.59% et 82.56% de précision de classification sur l'ensemble de test. Excepté les modèles plus faibles entraînés avec les combinaisons mentionnées ci-haut, il est important de souligner que les performances obtenues sont très similaires. Les performances en simulation permettront cependant de différencier certains modèles des autres.

4.2 Résultats de simulation

Contrairement aux tests effectués lors de l'entraînement, une simulation virtuelle dans l'environnement 3D qui a servi à réaliser l'ensemble de données DGD permettra au drone piloté par l'agent autonome d'influencer et d'interagir avec l'environnement en observant l'effet de ses actions. Les expérimentations en simulations ont donc pour objectif d'évaluer les performances de façon plus réaliste en respectant le processus de Markov.

Les trajectoires obtenues par simulation virtuelle devraient ainsi ressembler visuellement à celles de l'expert en plus d'avoir une distance parcourue similaire. Une série de 9 destinations consécutives a été élaborée afin d'évaluer différentes trajectoires de plusieurs niveaux de difficulté. L'atteinte d'une destination est caractérisée par une position du drone inférieure à un rayon de 0,65 mètre. Le rayon a été fixé en moyennant la distance finale de l'expert aux destinations. La vitesse de déplacement utilisée est celle obtenue par l'étude de l'effet de la vitesse présentée dans cette section. Cette vitesse a été fixée à 0.3 mètre par seconde. Les vitesses explorées au cours des expériences en simulation sont limitées par la fréquence d'exécution du script de guidage contenant

le modèle ADG. L'objectif du mémoire de maîtrise n'étant pas de guider un drone à haute vitesse, l'optimisation du code n'a pas été priorisé et son exécution à 8 Hz a été jugée acceptable compte tenu des performances obtenues.

Pour les mêmes raisons qui sont mentionnées à la section 3.7.3, l'environnement 3D qui a été utilisé pour les simulations est le *Bar à Billard*. Les trajectoires obtenues par simulation virtuelle étant différentes que celles de l'expert pour le même objectif, il devient impossible de comparer directement les commandes envoyées comme lors de l'entraînement. Il s'avère que la distance parcourue pour atteindre l'objectif est un bon indice de performance pour cette tâche. Il s'agit de comparer la différence de distance entre l'agent autonome et l'expert afin d'avoir une idée de l'efficacité de l'agent autonome en imitation. Il faut rappeler que l'objectif ultime est d'atteindre les destinations le plus rapidement possible, soit en parcourant la plus petite distance possible. Le ratio de la différence de distance (l'erreur) et de la distance parcourue par l'expert permet ensuite de relativiser par rapport à la longueur du trajet. Cet indice de performance jumelé à une analyse visuelle des parcours a donc été employé afin de caractériser les différents modèles.

Les modèles ont été testés dans diverses expérimentations afin d'établir leurs limitations. La section qui suit traitera initialement des performances des modèles obtenus dans la section 4.1, suivi de l'effet de la vitesse, de l'altitude et du rafraîchissement de la mémoire sur les performances. Ensuite, une analyse des performances dans différents environnements 3D sera présentée avant d'évaluer l'importance d'un réseau récurrent dans le modèle et de terminer par l'analyse d'importance de l'utilisation de la profondeur d'image.

4.2.1 Comparaison des modèles obtenus

Tableau 4.4 Performances moyennes par trajectoire par rapport à l'expert

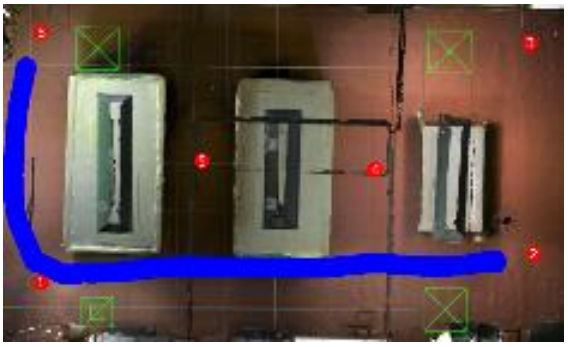
Trajectoire	Erreur moyenne (m)	Erreur moyenne relative (%)
1 à 2	5.6	56
2 à 6	0.9	6
6 à 10	4.0	58
10 à 4	2.6	18
4 à 10	0.6	4
10 à 1	6.5	70
1 à 6	12.6	254
6 à 9	4.2	37
9 à 1	3.1	23
Moyenne	4.5	58

Les modèles obtenus par la recherche d'hyperparamètres de la section 4.1 ont été testés dans l'environnement de simulation virtuel décrit ci-haut. Le Tableau 4.4 résume les performances moyennes recueillies pour chacune des trajectoires de la simulation du *Bar à Billard*. La Figure 4.4 présente trois trajectoires parcourues par trois modèles différents en comparaison avec le trajet effectué par l'expert. Les trois trajectoires sont classées comme étant mauvaise, moyenne ou bonne en fonction de la distance totale parcourue et de l'allure de leurs trajectoires. Le meilleur des trois modèles a surpassé l'expert en choisissant un chemin plus rapide alors que les deux autres ont connu un parcours saccadé et parsemé d'indécisions quant à la direction à prendre. Le pire des trois modèles de la Figure 4.4 n'a pas atteint sa cible.

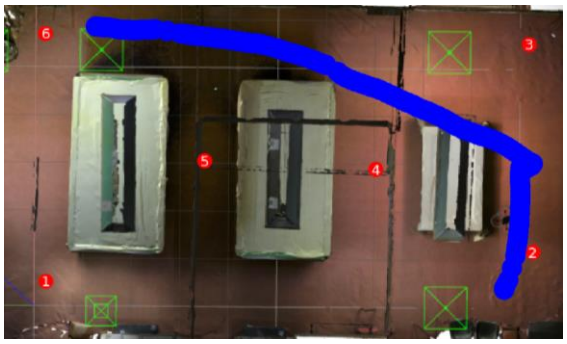
En moyenne, les modèles obtenus ont parcouru 4.5 mètres de plus pour atteindre leur objectif, soit 58% plus de distance que l'expert. Ces performances relatives montrent que les modèles sont loin d'être aussi efficaces que l'expert. Cependant, il faut tout de même remarquer que les modèles ont tous été en mesure d'atteindre leur objectif et que ces statistiques tiennent aussi compte des pires modèles obtenus. Il est aussi important de mentionner que parmi les 144 trajectoires parcourues pour obtenir ce tableau, moins de 5% d'entre elles ont nécessité une intervention manuelle pour corriger la trajectoire leur permettant d'atteindre leur objectif. Ces interventions étaient autorisées

seulement si le drone s'était écrasé ou si ce dernier était pris depuis trop longtemps au même endroit.

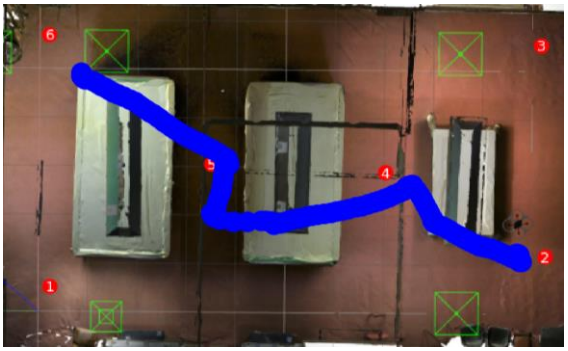
Expert (14.0 m.)



Bon (12.9 m.)



Moyen (13.1 m.)



Mauvais (21.1 m.)

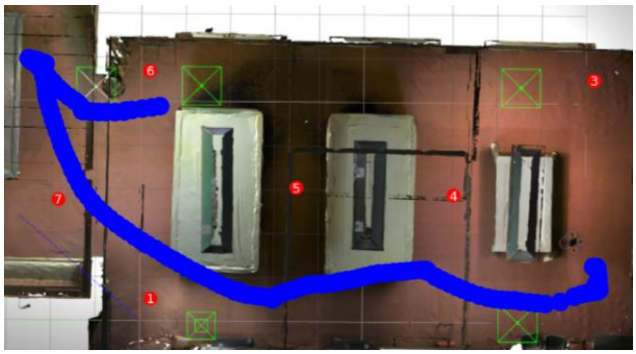


Figure 4.4 Comparaison et évaluation de la qualité des trajectoires (2 à 6) de 3 modèles avec l'expert

Certaines trajectoires ont évidemment été beaucoup plus difficiles à imiter que d'autres. Plusieurs trajectoires débutent dans une orientation défavorable à l'atteinte du prochain objectif en raison de la façon dont l'objectif précédent a été approché. Par exemple, la trajectoire 1 à 6 se démarque par son erreur relative de 254%.

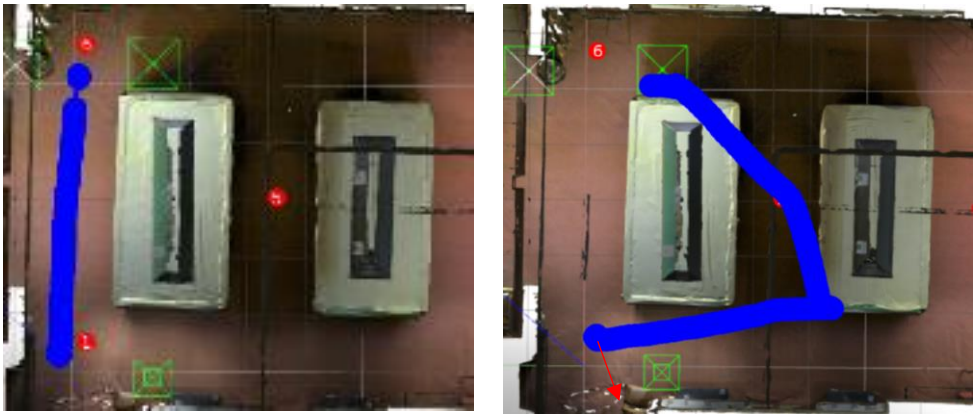


Figure 4.5 Comparaison des trajectoires 1 à 6 par l'expert (gauche) et le modèle 60% DA et 40% SM (droite)

Cette trajectoire illustrée à la Figure 4.5 est dans ce cas-ci très courte et débute la majorité du temps en faisant face au coin inférieur gauche tout en ayant un angle relatif à l'objectif plus faible vers la gauche (orientation initiale illustrée par la flèche rouge). Le modèle débute donc la nouvelle trajectoire avec un virage sur soi-même vers la gauche durant lequel un long corridor de profondeur apparaît. Plutôt que de continuer sa rotation pour faire face à l'objectif 6, bon nombre de modèles entreprennent d'explorer le premier corridor de profondeur observable. Il en résulte qu'un grand détour est entrepris avant d'atteindre l'objectif.

Dans le même ordre d'idées, les trajectoires 10 à 4 et 4 à 10 du Tableau 4.4 auraient dû obtenir en moyenne une erreur similaire. Or, la trajectoire 10 à 4 s'est avérée plus difficile en raison de la différence de profondeur perçue par l'algorithme de profondeur entre l'objectif 4 et le mur qui lui fait face. En effet, en provenance de l'objectif 10, il semble rester une plus grande distance à parcourir vers le mur lors de l'arrivée à la hauteur de l'objectif 4, ce qui encourage le modèle à continuer tout droit plutôt que de corriger son orientation comme il le fait dans la trajectoire 4 à 10. La plupart des autres trajectoires du Tableau 4.4 ayant une erreur élevée subissent le même problème qui peut être résumé par une envie du modèle de suivre un corridor de profondeur plus longtemps que nécessaire avant de corriger son orientation. Ce dernier corrige ainsi souvent son orientation lorsqu'une diminution de profondeur signifiant l'approche d'un mur ou d'un obstacle se fait sentir. La mesure de la profondeur est cependant dans le cas des trajectoires 4 à 10 et 10 à 4 à blâmer dans ce comportement.

Certains modèles ont tout de même su se démarquer par rapport aux autres. Le Tableau 4.5 regroupe l'erreur moyenne des 9 trajectoires pour chacun des couples de probabilités d'augmentation qui ont permis d'obtenir les 16 modèles présentés. Le modèle ayant le mieux performé est celui du Tableau 4.5 ayant été entraîné avec 20% de DA et 40% de SM. Le modèle a parcouru en moyenne sur les 9 trajectoires 1,92 mètre de plus que l'expert, soit seulement 16,9% plus de distance. Le meilleur modèle selon les résultats de l'entraînement du Tableau 4.3, ayant une précision de 82,59%, aurait dû être celui entraîné avec 0% de DA et 40% de SM. Or, ce modèle a obtenu la pire performance par rapport aux autres modèles, avec une moyenne de 9,40 mètres de plus par trajectoire selon le Tableau 4.5.

Tableau 4.5 Erreur moyenne en mètres entre les agents autonomes et l'expert

		Destination artificielle (%)				
		0	20	40	60	Moyenne
Miroir (%)	0	7.3	8.5	2.6	6.3	6.2
	20	4.9	2.6	4.4	4.1	4.0
	40	9.4	1.9	3.5	4.2	4.8
	60	2.6	2.9	3.2	2.8	2.9
	Moyenne	6.0	4.0	3.5	4.4	4.5

Le modèle ayant les meilleures performances sera celui jugé optimal dans le reste de ce mémoire de maîtrise. Ces performances en imitation sont présentées à la Figure 4.6, où il est possible de comparer le parcours de l'expert et celui du modèle optimal. Cette allure habituelle de l'expert montre qu'il priorise le suivi de couloirs sans obstacle au sol (dans le cas présent représenté par la table de billard). L'agent dicté par le modèle optimal préfère typiquement explorer le présent couloir de profondeur le plus longtemps possible, jusqu'à tolérer un léger dépassement de son objectif. Au contraire, l'expert a choisi de corriger lentement son orientation tout en approchant la destination.

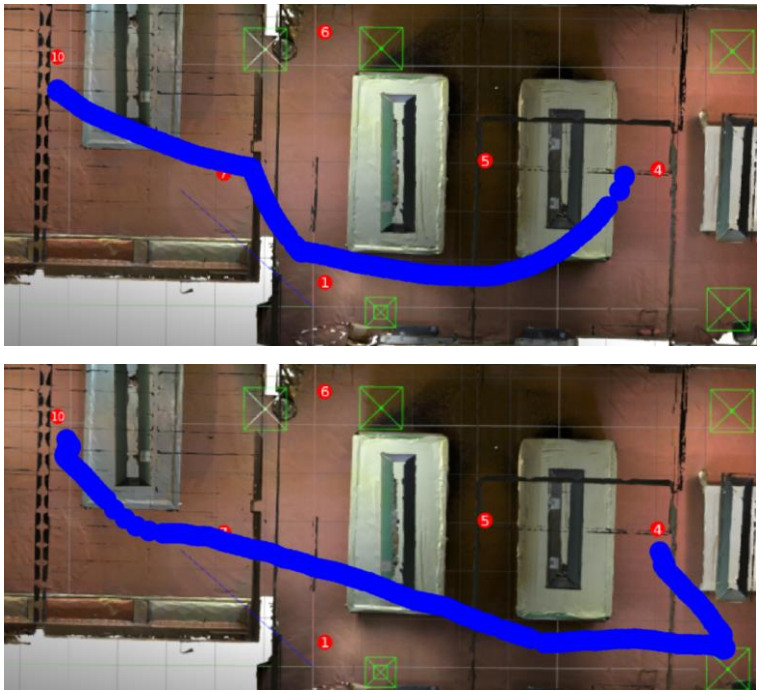


Figure 4.6 Comparaison de la trajectoire 10 à 4 de l'expert (haut) et de l'agent autonome (bas)

On remarque aussi que les modèles les plus performants le sont beaucoup moins en simulation. L'effet de ne pas utiliser de séquences miroir est, comme dans le Tableau 4.3, aussi visible dans le Tableau 4.5 avec une moyenne de 6.19 mètres. Cependant, les modèles ayant performé légèrement moins dans les tests d'entraînement sont maintenant ceux ayant le mieux performé en simulation. De plus, les modèles ayant eu une précision significativement moindre lors des tests d'entraînement du Tableau 4.3 semblent pour leur part avoir des performances de simulation à mi-chemin entre les deux extrêmes du Tableau 4.5, et être moins variables par rapport à leur précision d'entraînement.

Il est important de noter que les modèles les plus précis en tests d'entraînement n'utilisaient pas d'augmentation de DA. Les résultats de simulation semblent donc montrer la nécessité d'augmenter de façon raisonnable, l'ensemble de données avec cette technique.

De façon générale, les résultats du Tableau 4.5 confirment ce que Ross et al. (2010) [25], [26] ont observé dans les problèmes d'apprentissage par imitation. Les problèmes de prédiction séquentielle dans lesquels les observations dépendent des actions précédentes violent les principes d'échantillonnage i.i.d. Malgré les mesures prises pour mitiger l'échantillonnage, l'entraînement, la validation et les tests sont tout de même effectués dans un contexte ne respectant pas le principe

de Markov, ce qui entraîne une fausse perception de la performance des modèles obtenus. Par ailleurs, les erreurs effectuées par le modèle en simulation peuvent s'accumuler et pousser l'agent dans une situation jamais rencontrée par l'expert. Il est dans ce cas très difficile pour le modèle de choisir la bonne action à poser.

Les modèles les plus précis selon la phase de test n'ont en fait pas appris à naviguer logiquement, mais plutôt appris une série d'actions rigides à appliquer dans différents contextes. Au contraire, les autres modèles semblent avoir appris une formulation moins rigide des actions à exécuter, ce qui les place au premier rang lors de l'évaluation de leur performance en simulation. Il semble donc que les techniques d'augmentation ont un effet bénéfique sur les performances de simulation au détriment des performances d'entraînement.

4.2.2 Étude d'un modèle performant

Une étude approfondie a été conduite afin d'estimer l'enveloppe optimale des paramètres de vol du drone autonome. Tel qu'observé au Tableau 4.6, l'étude se base sur trois principaux caractères soit la vitesse du vol, l'altitude ainsi que la fréquence de rafraîchissement de la mémoire du module récurrent. Les altitudes munies d'un astérisque signifient qu'un cloisonnement des obstacles a été effectué tel qu'expliqué dans l'étude d'altitude. Le modèle utilisé par l'agent autonome est celui qui a le mieux performé dans les simulations de 4.2.1. Ce modèle a obtenu une précision de 82.1% lors de l'entraînement et une erreur moyenne en simulation de seulement 1.9 mètre. Il fera donc office de modèle de référence dans cette section et dans les sections qui suivent.

Tableau 4.6 Résumé des facteurs étudiés et leurs objectifs

Facteur	Intervalle	Objectif
Vitesse (m/s)	[0.3, 0.5, 0.7, 1]	Tester la généralisation à différentes vitesses
Altitude (m)	[0.475, 0.475*, 1.250*]	Tester la généralisation en altitude
Fréquence de rafraîchissement	N/A	Vérifier la capacité de la mémoire artificielle

L'effet de la vitesse à laquelle le drone se déplace sur les performances du drone en simulation est le premier facteur étudié dans cette section. Puisque le modèle est entraîné avec un saut de trois images («skip frames»), le modèle sera habitué à une vitesse trois fois plus grande que celle utilisée

pour la collecte de l'ensemble de données, soit une fréquence de 3 Hz. Le temps d'exécution de la boucle de commande du drone en simulation virtuelle est aussi différent de celui utilisé lors de la collecte et s'estime à 8 Hz. Ces facteurs font en sorte qu'il a été nécessaire d'implémenter un délai d'exécution pour la boucle de commande afin de faire croire au modèle que les séquences d'images reçues sont similaires à celles reçues lors de l'entraînement en termes de distance parcourue par pas de temps. Le délai implémenté occasionnant un comportement optimal change cette fréquence à 1.54 Hz. Les deux fréquences sont ainsi plus près l'une de l'autre tel qu'escompté. Ces faibles fréquences de mise à jour de la commande sont tout de même fonctionnelles dans le cadre de cette étude considérant la vitesse de déplacement.

Tableau 4.7 Erreur entre l'agent autonome et l'expert en mètres pour chaque trajectoire en fonction de la vitesse du drone

Trajectoire	Vitesse (m/s)				Moyenne	Moyenne
	0.3	0.5	0.7	1		(%)
1 à 2	4.4	4.8	5.7	6.1	5.3	52
2 à 6	-1.4	2.1	-1.6	0.4	-0.1	-1
6 à 10	-0.2	2.1	7.8	5.0	3.7	52
10 à 4	3.7	10.8	-2.0	8.3	5.2	36
4 à 10	1.5	1.0	1.6	0.1	1.0	6
10 à 1	1.4	2.6	4.3	3.0	2.8	30
1 à 6	0.7	22.6	13.1	7.9	11.1	224
6 à 9	4.5	2.1	0.5	2.5	2.4	21
9 à 1	2.7	1.0	0.1	16.9	5.2	37
Moyenne	1.9	5.5	3.3	5.6	4.1	51

Une fois ce délai bien ajusté en fonction de la vitesse, le modèle a été en mesure d'atteindre les performances présentées dans la section 4.2.1. Un balayage de la vitesse du drone a été effectué afin d'observer l'effet de la vitesse sur les performances. Le Tableau 4.7 montre que la vitesse de 0.3 mètre par seconde est la plus optimale dans le contexte de simulation. Il est à mentionner que les erreurs négatives représentent des cas où l'agent autonome a parcouru moins de distance que l'expert.

Les modèles ayant été exécutés avec une vitesse plus rapide ont cependant obtenu des performances moins constantes et moins compétitives que celles obtenues à 0.3 mètre par seconde en se fiant à aux quatre premières moyennes de la dernière ligne du Tableau 4.7. La constance des performances concerne encore les mêmes trajectoires que celles mentionnées dans l'analyse de 4.2.1. Les performances sur les parcours 10 à 4 ainsi que 1 à 6 ont cette fois été détériorées de plus belle en raison de la vitesse plus élevée de déplacement. Le scénario à 0.7 mètre par seconde fait par contre exception dans le cas du trajet 10 à 4, où la chance semble l'avoir récompensé. Les erreurs d'orientation commises ont par ailleurs été amplifiées telles que montrées sur la Figure 4.7, et ce, même si le guidage est effectué par le même modèle.

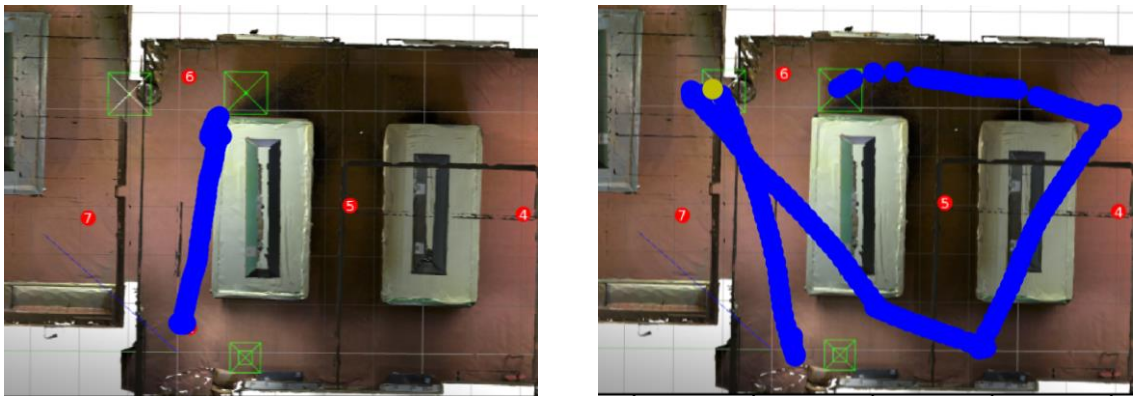


Figure 4.7 Comparaison des trajectoires 1 à 6 pour une vitesse de 0.3 m/s (gauche) et 0.5 m/s (droite)

Les performances sont cependant de moins en moins bonnes pour l'ensemble des trajectoires à mesure que la vitesse augmente. L'observation du comportement du drone commandé par l'agent autonome a aussi su montrer que le modèle était plus lent à réagir et à effectuer plusieurs virages cruciaux dans l'atteinte des objectifs lorsque la vitesse augmente, ce qui concorde avec les résultats observés dans le Tableau 4.7. Même si la fréquence d'exécution du script du modèle ADG était augmentée en fonction de la vitesse, il est justifié de croire que les performances resteraient similaires puisque le modèle est entraîné avec un ratio de mémoire versus distance parcourue fixe.

Le deuxième facteur étudié concerne l'altitude de guidage du drone. Deux éléments justifient l'étude de ce facteur. La première découle du fait que l'environnement 3D du *Bar à Billard* correspond à une salle à aire ouverte lorsque le drone navigue avec son altitude initiale. Il est libre de survoler les tables de billard comme bon lui semble dans les tests effectués jusqu'à présent. La

deuxième raison est que la variation d'altitude permettra d'évaluer la capacité de généralisation du modèle à différentes altitudes. Les tables de billard ont dû être remplies à l'intérieur dans le modèle pour éviter que l'agent autonome ne soit tenté de passer en dessous de celles-ci. La qualité du maillage 3D en dessous des tables est faible et aurait occasionné plusieurs collisions.

Les deux premières expériences du Tableau 4.8 se sont effectuées à une altitude nominale de 0.475 mètre comparativement à une altitude normale de 1.25 mètre nominalement. Dans le premier scénario d'altitude, le dessous des tables de billard a simplement été cloisonné tel qu'expliqué ci-haut.

Tableau 4.8 Écart entre l'agent autonome et l'expert en mètres pour chaque trajectoire en fonction du scénario d'altitude

Trajectoire	Scénario d'altitude			Moyenne	Moyenne relative
	1	2	3		(%)
1 à 2	9.6	23.6	23.7	19.0	187
2 à 6	2.1	17.4	6.7	8.7	62
6 à 10	16.4	13.5	4.3	11.4	162
10 à 4	39.7	13.5	12.3	21.8	151
4 à 10	25.1	25.0	49.1	33.1	197
10 à 1	0.7	8.3	46.8	18.6	200
1 à 6	20.6	18.7	-0.1	13.1	264
6 à 9	10.4	4.3	9.9	8.2	72
9 à 1	6.7	2.8	7.0	5.5	40
Moyenne	14.6	14.1	17.7	15.5	148

Il a été remarqué que le drone piloté par l'agent autonome dans le scénario 1 avait souvent tendance à s'approcher dangereusement des tables de billard. La raison qui explique ce comportement est la représentation qui est faite de l'image de profondeur dans le modèle de guidage. Il semble que le module REP1 du modèle présenté à la section 3.5.1 apprend à représenter l'orientation horizontale de la ou des directions ayant une forte profondeur indépendamment de la position verticale de ces zones dans l'image. Cette caractéristique a pour effet de faire croire que le drone se dirige vers un couloir profond même si un obstacle l'attend à son altitude. Dans la scène du bar à billard, les tables

de billard de la Figure 4.8 ont cet effet : à une certaine distance de celles-ci, une ouverture au-dessus de la table encourage le drone à s'y diriger, lorsque le drone est suffisamment proche de la table et que l'ouverture n'est plus visible, l'agent autonome arrête le drone et change de direction.



Figure 4.8 Scénario encourageant l'agent à s'approcher dangereusement d'un obstacle. L'encadré noir met en évidence la profondeur résiduelle incitant à ignorer la table.

Dans les deuxième et troisième scénarios, le cloisonnement des tables a été prolongé jusqu'au plafond tel qu'illustré à la Figure 4.9 afin de tenir compte de cette faiblesse (représenté par un astérisque au Tableau 4.6). Le scénario 2 se déroule également à basse altitude et obtient une erreur moyenne similaire à l'altitude du scénario 1. De façon générale, l'agent autonome atteint des résultats similaires dans les deux premières situations. Les performances obtenues dans le scénario 3 diffèrent cependant alors que l'altitude est remise à la normale. Excepté pour le cas de la trajectoire 10 à 1, l'agent performe en général de façon égale ou meilleure que ceux des scénarios précédents.

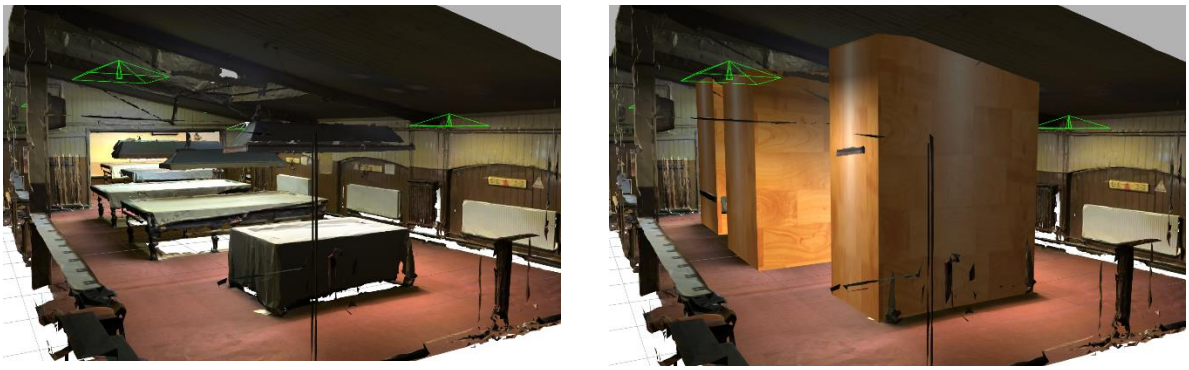


Figure 4.9 Avant et après le cloisonnement complet des tables de billard.

Comme les erreurs importantes du Tableau 4.8 en témoignent, l'agent autonome a parfois atteint un minimum local dans lequel il tourne en rond avant de choisir par hasard une direction à

emprunter. Ces minimums locaux sont souvent frappés alors que l'agent autonome longe un mur perpendiculaire à la direction de son objectif et que l'agent se trouve au milieu du mur. Peu de temps après que l'orientation relative dépasse les ±90°, le drone se tourne jusqu'à la détection du prochain corridor de profondeur situé à ±180° par rapport à lui-même et l'histoire se répète de nouveau, tel qu'illustré à la Figure 4.10. L'effet peut varier en fonction de la fréquence de rafraîchissement de la mémoire du modèle récurrent et de l'efficacité de la prédiction de profondeur. Le premier élément sera étudié dans la prochaine expérience. La précision de la profondeur s'est par contre souvent montrée moins efficace lors du guidage à basse altitude, ce qui a pu également nuire aux performances obtenues.

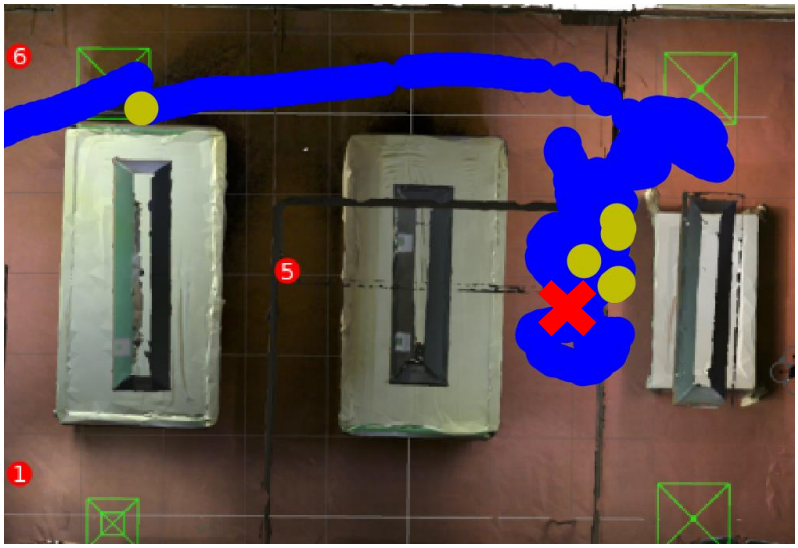


Figure 4.10 Trajectoire typique lorsque le modèle est incertain.
Le X rouge est le point de départ de la trajectoire.

Admettant que la performance obtenue pour le trajet 10 à 1 dans le troisième scénario est aberrante comparativement aux autres erreurs obtenues, la moyenne de l'erreur pour le troisième scénario chuterait drastiquement et cette altitude deviendrait probablement optimale. Ceci prouverait que l'altitude optimale est celle à laquelle le modèle a été entraîné. Il a cependant été remarqué tel que mentionné précédemment, que le modèle est peu influencé par l'altitude, mais plutôt par le problème de minimums locaux. Effectuer cette expérience à plusieurs reprises pourrait également permettre de minimiser l'incertitude face à ces résultats.

Le dernier facteur étudié concerne la fréquence de rafraîchissement de la mémoire interne du module récurrent. Lorsque ces modules récurrents sont en phase d'entraînement, leur mémoire

interne est rafraîchie après chaque séquence afin de ne pas réutiliser de l'information du contexte de la séquence précédente. Ce rafraîchissement est justifié par le fait que les séquences ne se suivent pas. Il est habituel de rafraîchir la mémoire du modèle déployé à la même fréquence que lors de l'entraînement. Contrairement à la phase d'entraînement, la séquence de simulation sera beaucoup plus longue et il est justifié de se demander si le rafraîchissement de la mémoire est nécessaire. Le gain de conserver l'information sur toute la durée d'une trajectoire versus le rafraîchissement conforme à l'entraînement est présenté dans le Tableau 4.9. Le scénario 1 correspond au scénario du modèle de référence dans lequel le rafraîchissement s'effectue à toutes les 33 itérations alors que le rafraîchissement s'effectue à chaque nouvelle trajectoire dans le scénario 2.

Tableau 4.9 Effet de la fréquence de rafraîchissement de la mémoire sur l'erreur en mètres entre l'agent autonome et l'expert pour chaque trajectoire

Trajectoire	Scénario de rafraîchissement de fréquence	
	1	2
1 à 2	4.4	5.1
2 à 6	-1.4	2.1
6 à 10	-0.2	4.7
10 à 4	3.7	4.4
4 à 10	1.5	-3.7
10 à 1	1.4	2.0
1 à 6	0.7	4
6 à 9	4.5	10.4
9 à 1	2.7	2.1
Moyenne	1.9	3.5

Il semble que le rafraîchissement à chaque nouvelle trajectoire est légèrement moins efficace à imiter l'expert que le modèle le plus performant. L'observation des simulations a révélé que le rafraîchissement à toutes les 33 itérations comme lors de l'entraînement permet souvent d'agir comme un incitatif à essayer une nouvelle stratégie. Dans le cas où le drone se dirige dans la mauvaise direction ou que ce dernier est dans un cul-de-sac tel que celui de la Figure 4.11, les

rafraîchissements se sont par exemple avérés utiles pour corriger la situation. Comparativement avec le deuxième scénario, le drone s'est vu pris dans ces situations plus longtemps, car il n'a pas été entraîné à conserver autant d'informations temporelles. Ceci constitue une limitation de ce type de modèles puisque la plupart des modèles récurrents ont une mémoire très limitée (30 à 50 pas de temps) dépendamment de la complexité de la tâche à effectuer. Somme toute, cette limitation est probablement fortement responsable du fait que l'agent autonome ait performé près de deux fois moins bien que l'agent original.

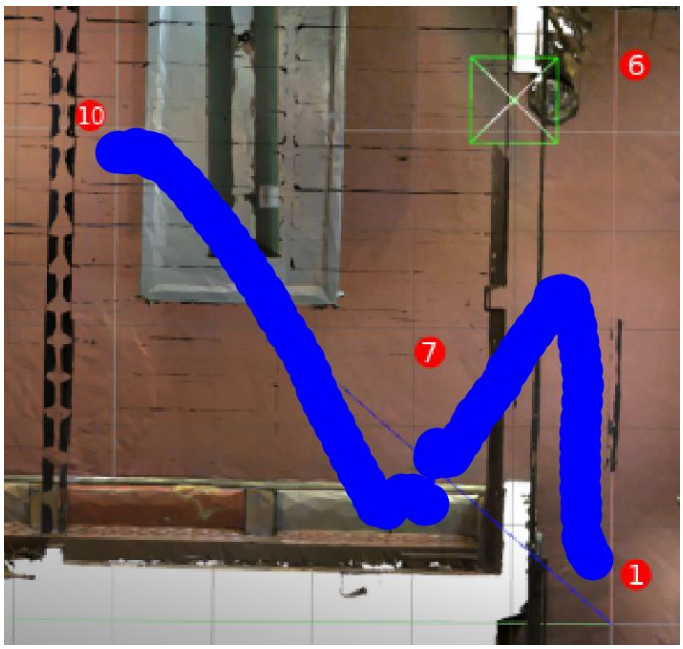


Figure 4.11 Trajectoire (10 à 1) où le drone reste pris dans le coin en raison du non-rafraîchissement de la mémoire.

4.2.3 Impact de l'ensemble de validation et de test

Le choix des données du bar à billard comme ensemble de validation et de test a été comme mentionné à la section 3.7.3 basé sur la nécessité de diviser l'ensemble de données de façon représentative et non biaisée. Il était à ce moment reconnu que l'ensemble de données de validation et de test était plus simple, car il était constitué de moins d'obstacles et de passages restreints. Une étude de l'effet du choix de l'ensemble de validation et de test a donc été effectuée. Par souci de

conformité, l'entraînement été complété en utilisant les mêmes hyperparamètres que pour le modèle optimal.

Pour ce faire, l'étude proposée consiste à reproduire partiellement la technique de validation croisée à *k* blocs («k-fold cross-validation»). Plutôt que de naïvement séparer l'ensemble de données en *k* blocs de tailles identiques, ce qui amalgamerait des trajectoires issues d'un même environnement 3D entre les blocs d'entraînement et de validation, considérons trois environnements se distinguant des autres par leur apparence et leur complexité. Le premier environnement est celui du *Bar à Billard*, faisant office d'un environnement ouvert et simple. Le second correspond à celui du *Bureau,* représentant un environnement de difficulté moyenne. Le dernier environnement est celui d'une *Salle de machines* avec une multitude de passages étroits complexes. Le Tableau 4.10 résume les ensembles qui ont été utilisés pour la pseudo-validation croisée à *k* blocs.

Tableau 4.10 Ensembles utilisés pour la pseudo-validation croisée à *k* blocs. Les environnements 3D en gras sont ceux utilisés pour la validation et les tests

Numéro des ensembles		
1	2	3
Bar à Billard	*Bar à Billard*	*Bar à Billard*
Maison 1	*Maison 1*	*Maison 1*
Salle de machines	*Salle de machines*	***Salle de machines***
Salle de serveurs	*Salle de serveurs*	*Salle de serveurs*
Bureau	***Bureau***	*Bureau*
Resto-Bar	*Resto-Bar*	*Resto-Bar*
Maison 2	*Maison 2*	*Maison 2*

Tableau 4.11 Performances obtenues en fonction de l'ensemble de données de validation et de test

Ensemble de validation et test	Précision en test (%)	Erreur de simulation (%)
Bar à billard	82.10	17
Bureau	76.76	46
Salle de machine	73.33	78
Moyenne	77.39	47

La conclusion qui peut être tirée des résultats présentés au Tableau 4.11 est évidemment que le choix de l'ensemble de données de validation et de test a une grande influence sur les performances en simulation ainsi qu'en validation et en test d'entraînement. Les résultats montrent d'une part que la complexité de l'ensemble de test est inversement proportionnelle à la précision de test. L'explication logique de ce phénomène est qu'il est simplement normal de ne pas obtenir de bonnes performances sur un ensemble de test si l'entraînement ne s'effectue pas sur des trajectoires complexes. Le nombre de trajectoires dans chacun des ensembles de validation et de test choisi a également son rôle à jouer dans l'obtention de ces résultats. L'environnement 3D du *Bureau* et de la *Salle de machines* sont beaucoup plus volumineux que celui du *Bar à Billard*, ce qui peut avoir un impact sur la capacité de généralisation des modèles.

La Figure 4.12 montre des trajectoires typiques pour les deux derniers environnements de plus grande taille. Les points jaunes démarquent les endroits où il y a eu collision. La coque en mousse du AR Drone 2.0 ainsi que la basse vitesse de déplacement permet de tolérer les impacts mineurs sans causer l'écrasement. Certaines collisions sont sans doute dues au faible champ de vision du drone et à la présence de défauts difficilement détectables dans les environnements 3D. Il est difficile à dire si l'agent autonome corrige automatiquement la trajectoire post-impact en raison d'un retard dans l'exécution de la commande pour éviter l'obstacle ou en raison de la présence

d'impacts dans l'ensemble de données. Les situations d'impacts ont effectivement été conservées pour permettre au modèle d'apprendre comment réagir suite à de faibles collisions.

Ces deux phénomènes expliquent donc pourquoi les modèles entraînés avec le *Bar à billard* comme ensemble de validation et de test ont obtenu la meilleure précision ainsi qu'une faible erreur. Somme toute, les agents autonomes qui ont utilisé ces modèles de guidage ont tout de même été en mesure de compléter la majorité des trajectoires sans collision immanquables avec les obstacles des environnements plus complexes. La précision moyenne des modèles de 77.39% ainsi que l'erreur moyenne de 47.20% représente donc bien les performances observées au cours des simulations.



Figure 4.12 Exemples de trajectoires dans des environnements plus grands (bureau : droite) et plus complexes (salle de machine : gauche).

4.2.4 Nécessité du module récurrent

L'introduction d'un module récurrent à l'intérieur du modèle de guidage autonome avait pour objectif de gérer l'aspect temporel de la prise de décision. Nous avons cru intéressant d'évaluer l'effet d'un tel module sur les performances de guidage. Un modèle identique a donc été généré ayant comme particularité que le module LSTM a complètement été remplacé par un MLP de trois couches de la même dimension que la cellule de mémoire du LSTM (1000 unités par couche).

L’entraînement a également été complété en utilisant les mêmes hyperparamètres que pour le modèle optimal.

Le modèle sans capacité de mémoire temporelle a durant l’entraînement atteint une précision de test de 72.67%, en comparaison avec 82.1% pour le meilleur modèle. Ce résultat d’entraînement démontre la capacité à réutiliser l’information observée et les actions effectuées dans le temps que procurent les modèles récurrents. La simulation virtuelle du modèle muni du MLP confirme cette théorie.

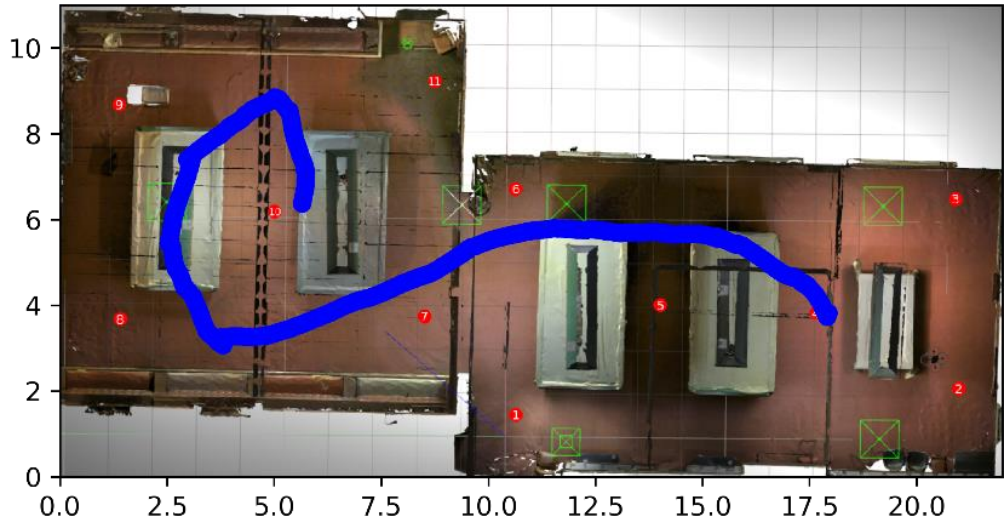


Figure 4.13 Exemple de trajectoire exécutée par le modèle dépourvu de module récurrent (trajet 4 à 10)

La Figure 4.13 montre une des trajectoires typiques ayant obtenu une erreur relative élevée dans le Tableau 4.12. Ces trajectoires sont marquées par un guidage beaucoup moins cohérent et moins précis. L’agent autonome semble avoir de la difficulté à effectuer des virages précis pour atteindre son objectif par exemple. Plutôt que de se déplacer en ayant une perception de ses déplacements antérieurs, l’agent autonome sans modèle récurrent ne fait que suivre le corridor de profondeur le plus important sans tenter d’explorer des directions plus favorables à l’atteinte de la destination.

Un nombre plus élevé de collisions a été enregistré dans le *Bar à Billard* lors des simulations comparativement au meilleur modèle obtenu. De façon générale, le modèle a réussi malgré l’utilisation de trajectoires sinueuses parsemées une moyenne de 11 collisions sur les 9 trajectoires, à atteindre les destinations de la simulation avec en moyenne deux interventions. La moyenne de l’erreur relative du Tableau 4.12 de 104% du modèle utilisant le MLP étant environ six fois plus

élevée que celle du meilleur modèle permet sans équivoque d'attester la nécessité d'utiliser un module récurrent pour le guidage autonome.

Tableau 4.12 Effet du module récurrent sur l'erreur relative entre l'agent autonome et l'expert pour chaque trajectoire

Trajectoire	Modèle de référence (%)	MLP (%)
1 à 2	43	198
2 à 6	-10	92
6 à 10	-3	152
10 à 4	26	37
4 à 10	9	174
10 à 1	15	54
1 à 6	14	199
6 à 9	40	2
9 à 1	19	26
Moyenne	17	104

4.2.5 Étude de l'effet de la profondeur

L'une des hypothèses principales sur lequel se base ce mémoire stipule que l'efficacité du guidage autonome est grandement influencée par la perception de la profondeur de l'environnement. Le modèle de guidage créé a donc été muni d'un module transformant l'image vidéo du drone en image de profondeur. Les caractéristiques 3D de l'environnement telles que les corridors et les obstacles ont donc été plus facilement identifiables par les modèles présentés ci-haut. Il importe finalement de comparer comment se comporte un modèle entraîné avec les images brutes RGB. Quelques modifications ont été apportées au modèle d'apprentissage par imitation ainsi qu'à la procédure d'entraînement avant d'obtenir un modèle digne de comparaison.

Il a été surprenant de constater que le modèle opérant avec une vision RGB a beaucoup de potentiel malgré le manque d'information sur la profondeur. Le Tableau 4.13 fait état des prouesses qui ont

été effectuées par le modèle et montrent que si ce n'était pas de la trajectoire 10 à 1, le modèle RGB aurait été comparable sinon meilleur que le modèle original.

Tableau 4.13 Effet du type de vision sur l'erreur relative entre l'agent autonome et l'expert pour chaque trajectoire

Trajectoire	Modèle de référence (%)	RGB (%)
1 à 2	43	5
2 à 6	-10	-8
6 à 10	-3	-5
10 à 4	26	-3
4 à 10	9	4
10 à 1	15	276
1 à 6	14	14
6 à 9	40	19
9 à 1	19	88
Moyenne	17	43

En entraînement, le modèle RGB a atteint une précision de test de 76.62%, ce qui est relativement proche de celle de 82.1% obtenu par son modèle de référence. Une fois en simulation virtuelle dans le *Bar à Billard*, le modèle RGB obtient une moyenne de 4.43 mètres de plus par trajectoire, ce qui lui vaut une erreur relative de 43.30% par rapport à l'expert.

De façon générale, il semble que le modèle adopte la stratégie de guidage favorisant les trajectoires rectilignes entre sa position initiale et son objectif. La nature de l'environnement Bar à billard est favorable à ce genre de stratégie puisqu'il s'agit d'un environnement majoritairement ouvert avec peu d'obstacles. Les erreurs relatives négatives obtenues au Tableau 4.13 pour plusieurs trajectoires sont alors explicables par le fait que les trajectoires de l'expert sont dictées par la profondeur, qui favorise des trajectoires souvent non rectilignes. Il y a de très fortes chances que les performances ne soient pas aussi excellentes dans un environnement plus complexe.

D'ailleurs, les simulations avec le modèle RGB ont montré qu'il avait de la difficulté à éviter des obstacles, ce qui soutient cette hypothèse. Bien qu'il semble longer et éviter les murs, il y a eu plusieurs collisions avec les coins de murs et les autres obstacles du *Bar à Billard*. Des collisions à répétition étaient par exemple inévitables si la destination se trouvait derrière un mur. C'est d'ailleurs ce qui s'est produit dans le cas de la trajectoire 10 à 1. Ne pas avoir de perception de la profondeur comme celle du modèle original semble aussi contribuer à ce que le modèle RGB ait un comportement hésitant lorsque vient le temps de choisir une direction. La Figure 4.14 montre ces deux comportements.



Figure 4.14 Comportements typiques du modèle RGB
Collisions avec un obstacle entre le drone et la destination (gauche) et hésitation dans la trajectoire (droite)

Il est impossible de savoir comment l'image RGB est représentée à l'intérieur du modèle et quel genre d'information cette représentation contient. Une supposition serait qu'il parvient à extraire une indication moins précise de la profondeur dans l'image de la même façon dont les algorithmes de prédiction de profondeur s'y prennent. En raison des résultats obtenus, il est justifié de conclure que la perception de profondeur est hautement bénéfique en ce qui a trait au guidage autonome.

4.3 Discussion et pistes de travaux futurs

Les résultats des sections précédentes soutiennent l'hypothèse qu'il est possible d'entraîner un modèle de guidage autonome intelligent pour drone en utilisant une approche d'apprentissage par imitation. L'apprentissage basé sur la vision de profondeur ainsi qu'une estimation de la position relative d'une destination s'est avéré un succès selon les résultats de simulation obtenus. Il est justifié d'affirmer, considérant la performance des modèles obtenus, que l'agent de guidage arrive à répliquer de façon significative la logique de guidage d'un humain.

Le succès obtenu dans cette tâche d'imitation implique que l'ensemble de données qui a été créé contient une logique observable suffisamment solide pour être extraite par un algorithme

d'apprentissage, confirmant l'atteinte des SO1 et SO3. Comme mentionné plus tôt, les performances d'entraînement rapportent que la formulation du problème pourrait être améliorée afin de simplifier la tâche d'apprentissage.

Le nombre de classes choisies a définitivement eu un impact sur la précision de classification lors des phases d'entraînement. Les classes adverses ont notamment posé problème comme prévu lors de l'élaboration de la méthodologie. Les modèles les plus performants ont cependant réussi dans une certaine mesure à détecter le moment opportun pour utiliser ces commandes en simulation malgré leurs performances non satisfaisantes sur l'ensemble de validation et de test.

Le débalancement des classes est aussi un problème à revoir concernant l'ensemble de données. Les ajustements des méthodes d'entraînement, de la fonction coût et d'augmentation de l'ensemble de données se sont avérés laborieux à plusieurs reprises. Comme démontré dans [85], il pourrait être avantageux de reformuler le problème de classification en problème de régression. Cela aurait pour effet d'atténuer le débalancement ainsi que le problème de classes adverses. Impliquant alors de recueillir des données en continu, il pourrait être intéressant d'envisager le remplacement de l'expert humain par un algorithme de planification optimale pour éviter d'obtenir des donnés contaminées par du bruit et des erreurs de guidage.

Toujours concernant l'ensemble de données, il est difficile d'évaluer la qualité des données développées sans la disponibilité d'autres ensembles de données pour comparer. Peu de problèmes abordés dans la littérature incorporent autant de complexité en termes de réalisme des simulations et de durée des parcours à effectuer. Les ensembles de données disponibles sont soit trop simplistes ou indisponibles, ce qui a justifié la nécessité d'en créer un répondant à nos besoins. Plusieurs scénarios complexes rendant la prise de décision difficile sont nécessairement présents dans les environnements utilisés pour la création de l'ensemble de données et des simulations. La trajectoire de l'expert peut par exemple passer à un endroit où deux chemins possibles sont visibles sur l'image de profondeur dont l'un est un cul-de-sac. Il est raisonnable de croire que l'expert tentera une fois ou deux d'explorer le cul-de-sac sachant qu'il devra rebrousser chemin, mais ignorera probablement cette option à l'avenir afin de ne pas perdre trop de temps et minimiser la ressemblance des segments dans l'ensemble de données. L'aspect fastidieux de la réalisation d'un ensemble de données aura donc fort probablement contribué à emprunter le chemin alternatif au

cul-de-sac même si ce dernier pouvait avoir l'air plus logique à emprunter selon l'image de profondeur.

L'exemple décrit ici une situation partiellement observable puisque de l'information connue par l'expert n'est pas détectable dans les données recueillies pour l'ensemble de données. Il devient extrêmement difficile d'optimiser la politique de guidage dans des situations comme celles-ci et peut même au contraire déstabiliser l'apprentissage.

Un agent de guidage autonome intelligent étant capable de fonctionner à différentes altitudes et vitesses en simulation a été obtenu en dépit des problèmes mentionnés ci-haut. L'utilisation de la vision de profondeur et d'un module récurrent a été prouvée efficace dans la résolution du problème. La généralisation du modèle obtenu a également été prouvée par sa capacité à fonctionner même lorsqu'il est dépourvu d'éléments importants de son ensemble de données. L'importance de la présence de ces environnements dans l'ensemble de données a alors pu être mesurée en fonction de leur taille et de leur complexité.

Aussi, la précision de l'algorithme d'estimation de la profondeur a eu plusieurs impacts sur la performance des modèles, notamment en lien avec la consistance des prédictions. Un algorithme plus précis et plus performant pourrait améliorer les résultats obtenus.

Le choix du AR Drone 2.0 permet de facilement passer des essais en simulation aux essais expérimentaux. Un environnement de type volière équipée d'un système Vicon serait un bon point de départ pour valider l'implémentation du module de guidage et le comparer aux simulations. Il est facile d'entrevoir des essais expérimentaux à plus grande échelle à condition d'avoir un système pouvant approximer la position du drone. L'incertitude sur la position du drone devra simplement être prise en compte dans la taille de la zone de destination, ce qui confirme partiellement l'atteinte du SO3. En effet le SO3 n'aurait été complété que si de vrais tests expérimentaux avaient eu lieu. Cependant, il est prudent d'affirmer que les performances seraient similaires en raison de la haute qualité des environnements de simulation et de l'abstraction de ceux-ci obtenus par l'algorithme de prédiction de profondeur FCRN.

Le modèle de guidage autonome ne se limite pas seulement au AR Drone 2.0. Bien qu'il ait été développé sur ce dernier pour des raisons de commodité, le modèle permettrait de guider tout drone ou même véhicule possédant une capacité de communication via Wi-Fi une caméra frontale ayant un champ de vision à 90° et un mode de localisation. Il est raisonnable de croire que les drones

plus récents pourraient apporter un grand avantage si l'on considère le nombre de caméras installées et leur champ de vision respectif. Par exemple, deux caméras (avant et arrière) avec un champ de vision de 180° pourraient grandement améliorer la représentation de l'état du drone. Cependant, ceci requerrait un nouvel ensemble de données de guidage incorporant ces modifications. Les caméras haute résolution auraient dans ce cas peu d'influence sur les performances obtenues puisqu'elles sont limitées par les algorithmes de prédiction de profondeur, qui utilisent des images de résolution réduite pour augmenter leur efficacité.

Dans le cas où les commandes de guidage sont fournies via communication Wi-Fi au drone, il est possible de l'imaginer naviguer dans un édifice à grande superficie utilisant un système GPS. En effet, il est parfois possible d'établir la position approximative d'un objet si le signal GPS n'est pas trop affaibli même à l'intérieur. La taille de la zone de destination devra dans ce cas-ci être prise en compte par ce facteur. Plusieurs autres alternatives permettant d'obtenir la position sont aussi envisageables telles que la triangulation par Wi-Fi, l'odométrie visuelle, etc.

Il sera éventuellement désiré d'effectuer les calculs pour le guidage à même le drone. Un nouveau drone ayant de meilleures capacités de calcul et de localisation sera alors probablement nécessaire. Il pourrait être intéressant d'installer plusieurs dispositifs de localisation différents adaptés à différentes situations. La complexité des missions de guidage pourrait ainsi être grandement augmentée si les systèmes de localisation prenaient le relais automatiquement en fonction de l'environnement.

CHAPITRE 5 CONCLUSION

L'objectif de ce travail de recherche était de concevoir un modèle de guidage intelligent basé sur la vision capable d'imiter la logique de guidage d'un humain dans un environnement inconnu complexe en se basant sur la vision et une estimation de sa destination. La stratégie choisie pour la synthèse du modèle se base sur l'apprentissage par imitation. Ce type d'apprentissage supervisé a pour objectif d'extraire la logique derrière une série de démonstrations par un expert. Dans le cas de cette étude sur le guidage automne, l'expert est un humain pilotant un drone vers une destination inconnue. Le modèle obtenu a donc extrait la politique de guidage de l'expert à partir d'une grande quantité de démonstrations. Les démonstrations représentent ici un ensemble de données d'entraînement pour le modèle.

La contribution du travail décrite dans ce mémoire inclut donc un modèle de guidage autonome basé sur la vision, un ensemble de données d'entraînement pour un problème de guidage par imitation ainsi qu'une méthodologie permettant de répliquer et comparer l'expérience.

Ce mémoire fait aussi état d'une preuve de concept pour le guidage autonome de drones intelligents. Les travaux effectués constituent un point de départ pour l'exploration de techniques de guidage plus sophistiquées pour les véhicules aériens miniatures. Le module de guidage est prêt à passer à l'étape expérimentale afin de valider les résultats de simulation. Une fois terminé, l'ajout d'une commande pour l'altitude de même que l'intégration d'autres modules de détection spécifiques permettant l'accomplissement de tâches plus complexes pourront être envisagés.

Une étude approfondie de ce type d'approche a été en mesure d'évaluer la robustesse du modèle face à différents facteurs incluant l'altitude, la vitesse et la variation de la taille de l'ensemble de données par validation croisée. L'importance de la représentation temporelle fournie par le module récurrent ainsi que l'importance de l'utilisation de la profondeur d'image ont également été démontrées.

Le lien entre les performances d'entraînement avec la qualité de l'ensemble de données a été mis en évidence et pourra servir à l'amélioration de celui-ci. Le problème de débalancement du nombre d'éléments dans chaque classe pourrait par exemple être compensé par une utilisation plus sélective de l'ensemble de données en éliminant les segments contenant seulement la classe AVANT. Le passage du contexte de simulation virtuelle au contexte d'expérimentation pourra se faire à

condition qu'un système de localisation et de communication avec le drone soit disponible. Considérant les performances obtenues par le modèle entraîné avec la vision RGB, il serait intéressant de combiner la méthode utilisant la profondeur ainsi que l'image en couleur.

Enfin, l'utilisation de ce modèle pour le guidage de drones est limitée par l'accès à un système de positionnement et de communication Wi-Fi. Aucun contrôle en altitude permettant de contourner des obstacles n'est implémenté. La taille des obstacles pouvant être évités est d'ailleurs limitée par la précision de l'algorithme d'estimation de profondeur. Le guidage d'un drone à une vitesse supérieur à la vitesse maximale testée dans ce mémoire, soit 1 mètre par seconde, est déconseillé en raison de la fréquence d'exécution du script de 8 Hz.

RÉFÉRENCES

[1] M. A. Ma'sum *et al.*, « Simulation of intelligent Unmanned Aerial Vehicle (UAV) For military surveillance », dans *2013 International Conference on Advanced Computer Science and Information Systems (ICACSIS)*, sept. 2013, p. 161-166, doi: 10.1109/ICACSIS.2013.6761569.

[2] S. M. Shavarani, M. G. Nejad, F. Rismanchian, et G. Izbirak, « Application of hierarchical facility location problem for optimization of a drone delivery system: a case study of Amazon prime air in the city of San Francisco », *Int J Adv Manuf Technol*, vol. 95, nº 9-12, p. 3141-3153, avr. 2018, doi: 10.1007/s00170-017-1363-1.

[3] Y. Naidoo, R. Stopforth, et G. Bright, « Development of an UAV for search rescue applications », dans *IEEE Africon '11*, sept. 2011, p. 1-6, doi: 10.1109/AFRCON.2011.6072032.

[4] S. Tang et V. Kumar, « Autonomous Flight », *Annu. Rev. Control Robot. Auton. Syst.*, vol. 1, nº 1, p. 29-52, mai 2018, doi: 10.1146/annurev-control-060117-105149.

[5] A. Bry, C. Richter, A. Bachrach, et N. Roy, « Aggressive flight of fixed-wing and quadrotor aircraft in dense indoor environments », *The International Journal of Robotics Research*, vol. 34, nº 7, p. 969-1002, juin 2015, doi: 10.1177/0278364914558129.

[6] K. Tateno, F. Tombari, I. Laina, et N. Navab, « CNN-SLAM: Real-Time Dense Monocular SLAM with Learned Depth Prediction », dans *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Honolulu, HI, juill. 2017, p. 6565-6574, doi: 10.1109/CVPR.2017.695.

[7] I. Goodfellow, Y. Bengio, et A. Courville, « Convolutional networks », dans *Deep learning*, Cambridge, Massachusetts: The MIT Press, 2016, p. 326-366.

[8] D. A. Pomerleau, « ALVINN: An Autonomous Land Vehicle in a Neural Network », dans *Advances in Neural Information Processing Systems 1*, D. S. Touretzky, Éd. Morgan-Kaufmann, 1989, p. 305–313.

[9] J. Harmer *et al.*, « Imitation Learning with Concurrent Actions in 3D Games », *arXiv:1803.05402 [cs, stat]*, sept. 2018, Consulté le: août 31, 2020. [En ligne]. Disponible à: http://arxiv.org/abs/1803.05402.

[10] C. Smith, B. McGuire, T. Huang, et G. Yang, « The History of Artificial Intelligence », p. 27, 2017.

[11] R. Anyoha, « The History of Artificial Intelligence », *Science in the News*, août 28, 2017. http://sitn.hms.harvard.edu/flash/2017/history-artificial-intelligence/ (consulté le juin 18, 2020).

[12] « Welcome — Theano 1.0.0 documentation ». http://deeplearning.net/software/theano/ (consulté le juin 18, 2020).

[13] « Caffe | Deep Learning Framework ». https://caffe.berkeleyvision.org/ (consulté le juin 18, 2020).

[14] « Keras: the Python deep learning API ». https://keras.io/ (consulté le juin 18, 2020).

[15] « TensorFlow ». https://www.tensorflow.org/ (consulté le juin 18, 2020).

[16] « PyTorch ». https://www.pytorch.org (consulté le juin 18, 2020).

[17] I. Goodfellow, Y. Bengio, et A. Courville, *Deep Learning*. MIT Press, 2016.

[18] « MNIST handwritten digit database, Yann LeCun, Corinna Cortes and Chris Burges ». http://yann.lecun.com/exdb/mnist/ (consulté le juin 18, 2020).

[19] « New York Stock Exchange ». https://kaggle.com/dgawlik/nyse (consulté le juin 18, 2020).

[20] L. P. Kaelbling, M. L. Littman, et A. W. Moore, « Reinforcement Learning: A Survey », *Journal of Artificial Intelligence Research*, vol. 4, p. 237-285, mai 1996, doi: 10.1613/jair.301.

[21] L. Ma, Y. Liu, J. Chen, et D. Jin, « Learning to Navigate in Indoor Environments: from Memorizing to Reasoning », p. 6, 2019.

[22] V. Mnih *et al.*, « Human-level control through deep reinforcement learning », *Nature*, vol. 518, n° 7540, Art. n° 7540, févr. 2015, doi: 10.1038/nature14236.

[23] Y. Tang *et al.*, « An Overview of Perception and Decision-Making in Autonomous Systems in the Era of Learning », *arXiv:2001.02319 [cs]*, févr. 2020, Consulté le: juin 11, 2020. [En ligne]. Disponible à: http://arxiv.org/abs/2001.02319.

[24] S. Ross et J. A. Bagnell, « Efficient Reductions for Imitation Learning », p. 8.

[25] S. Ross, G. J. Gordon, et J. A. Bagnell, « A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning », *arXiv:1011.0686 [cs, stat]*, nov. 2010, Consulté le: avr. 29, 2019. [En ligne]. Disponible à: http://arxiv.org/abs/1011.0686.

[26] S. Ross et J. A. Bagnell, « Efficient Reductions for Imitation Learning », p. 8, 2010.

[27] H. He, J. Eisner, et H. Daume, « Imitation Learning by Coaching », dans *Advances in Neural Information Processing Systems 25*, F. Pereira, C. J. C. Burges, L. Bottou, et K. Q. Weinberger, Éd. Curran Associates, Inc., 2012, p. 3149–3157.

[28] S. Duell, S. Udluft, et V. Sterzing, « Solving Partially Observable Reinforcement Learning Problems with Recurrent Neural Networks », dans *Neural Networks: Tricks of the Trade*, vol. 7700, G. Montavon, G. B. Orr, et K.-R. Müller, Éd. Berlin, Heidelberg: Springer Berlin Heidelberg, 2012, p. 709-733.

[29] P. de Haan, D. Jayaraman, et S. Levine, « Causal Confusion in Imitation Learning », dans *Advances in Neural Information Processing Systems 32*, H. Wallach, H. Larochelle, A. Beygelzimer, F. d\textquotesingle Alché-Buc, E. Fox, et R. Garnett, Éd. Curran Associates, Inc., 2019, p. 11698–11709.

[30] M. Bansal, A. Krizhevsky, et A. Ogale, « ChauffeurNet: Learning to Drive by Imitating the Best and Synthesizing the Worst », *arXiv:1812.03079 [cs]*, déc. 2018, Consulté le: juin 19, 2020. [En ligne]. Disponible à: http://arxiv.org/abs/1812.03079.

[31] C. Nwankpa, W. Ijomah, A. Gachagan, et S. Marshall, « Activation Functions: Comparison of trends in Practice and Research for Deep Learning », *arXiv:1811.03378 [cs]*, nov. 2018, Consulté le: juin 11, 2020. [En ligne]. Disponible à: http://arxiv.org/abs/1811.03378.

[32] S. I. Gallant, « Perceptron-based learning algorithms », *IEEE Transactions on Neural Networks*, vol. 1, nº 2, p. 179-191, juin 1990, doi: 10.1109/72.80230.

[33] D. Beaini, S. Achiche, et M. Raison, « Improving Convolutional Neural Networks Via Conservative Field Regularisation and Integration », *arXiv:2003.05182 [cs]*, mars 2020, Consulté le: juill. 22, 2020. [En ligne]. Disponible à: http://arxiv.org/abs/2003.05182.

[34] D. Beaini, S. Achiche, A. Duperré, et M. Raison, « Deep green function convolution for improving saliency in convolutional neural networks », *The Visual Computer*, janv. 2020, doi: 10.1007/s00371-020-01795-8.

[35] D. Beaini, S. Achiche, A. Duperre, et M. Raison, « Saliency Enhancement using Gradient Domain Edges Merging », *arXiv:2002.04380 [cs]*, févr. 2020, Consulté le: juill. 22, 2020. [En ligne]. Disponible à: http://arxiv.org/abs/2002.04380.

[36] J. Deng, W. Dong, R. Socher, L.-J. Li, Kai Li, et Li Fei-Fei, « ImageNet: A large-scale hierarchical image database », dans *2009 IEEE Conference on Computer Vision and Pattern Recognition*, juin 2009, p. 248-255, doi: 10.1109/CVPR.2009.5206848.

[37] C. Szegedy, S. Ioffe, V. Vanhoucke, et A. A. Alemi, « Inception-v4, Inception-ResNet and the Impact of Residual Connections on Learning », présenté à Thirty-First AAAI Conference on Artificial Intelligence, févr. 2017, Consulté le: juin 11, 2020. [En ligne]. Disponible à: https://www.aaai.org/ocs/index.php/AAAI/AAAI17/paper/view/14806.

[38] J. Chung, C. Gulcehre, K. Cho, et Y. Bengio, « Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling », *arXiv:1412.3555 [cs]*, déc. 2014, Consulté le: févr. 17, 2020. [En ligne]. Disponible à: http://arxiv.org/abs/1412.3555.

[39] B. McCann, J. Bradbury, C. Xiong, et R. Socher, « Learned in Translation: Contextualized Word Vectors », dans *Advances in Neural Information Processing Systems 30*, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, et R. Garnett, Éd. Curran Associates, Inc., 2017, p. 6294–6305.

[40] S. W. Chen, N. Atanasov, A. Khan, K. Karydis, D. D. Lee, et V. Kumar, « Neural Network Memory Architectures for Autonomous Robot Navigation », *arXiv:1705.08049 [cs]*, mai 2017, Consulté le: avr. 26, 2019. [En ligne]. Disponible à: http://arxiv.org/abs/1705.08049.

[41] C. Bousquet-Jette, S. Achiche, D. Beaini, Y. S. Law-Kam Cio, C. Leblond-Ménard, et M. Raison, « Fast scene analysis using vision and artificial intelligence for object prehension by an assistive robot », *Engineering Applications of Artificial Intelligence*, vol. 63, p. 33-44, août 2017, doi: 10.1016/j.engappai.2017.04.015.

[42] A. Hornung, K. M. Wurm, M. Bennewitz, C. Stachniss, et W. Burgard, « OctoMap: an efficient probabilistic 3D mapping framework based on octrees », *Autonomous Robots*, vol. 34, nº 3, p. 189-206, avr. 2013, doi: 10.1007/s10514-012-9321-0.

[43] « Review of Stereo Vision Algorithms: From Software to Hardware », 2008. https://www.tandfonline.com/doi/full/10.1080/15599610802438680 (consulté le juin 17, 2020).

[44] M. Sarkis et K. Diepold, « Sparse stereo matching using belief propagation », dans *2008 15th IEEE International Conference on Image Processing*, oct. 2008, p. 1780-1783, doi: 10.1109/ICIP.2008.4712121.

[45] M. H. Ouali, H. Lange, et C. I. Laurgeau, « An energy minimization approach to dense stereovision », dans *Proceedings of 3rd IEEE International Conference on Image Processing*, sept. 1996, vol. 2, p. 841-845 vol.2, doi: 10.1109/ICIP.1996.561036.

[46] H. H. Ler, P. R. Innocent, et J. Garibaldi, « Real-Time Correlation-Based Stereo Vision with Reduced Border Errors », p. 18.

[47] S. Khamis, S. Fanello, C. Rhemann, A. Kowdle, J. Valentin, et S. Izadi, « StereoNet: Guided Hierarchical Refinement for Real-Time Edge-Aware Depth Prediction », dans *Computer Vision – ECCV 2018*, vol. 11219, V. Ferrari, M. Hebert, C. Sminchisescu, et Y. Weiss, Éd. Cham: Springer International Publishing, 2018, p. 596-613.

[48] A. Bhoi, « Monocular Depth Estimation: A Survey », *arXiv:1901.09402 [cs]*, janv. 2019, Consulté le: mai 22, 2020. [En ligne]. Disponible à: http://arxiv.org/abs/1901.09402.

[49] J. Michels, A. Saxena, et A. Y. Ng, « High speed obstacle avoidance using monocular vision and reinforcement learning », dans *Proceedings of the 22nd international conference on Machine learning - ICML '05*, Bonn, Germany, 2005, p. 593-600, doi: 10.1145/1102351.1102426.

[50] A. Saxena, M. Sun, et A. Y. Ng, « Learning 3-D Scene Structure from a Single Still Image », dans *2007 IEEE 11th International Conference on Computer Vision*, Rio de Janeiro, Brazil, 2007, p. 1-8, doi: 10.1109/ICCV.2007.4408828.

[51] J. Hu, M. Ozay, Y. Zhang, et T. Okatani, « Revisiting Single Image Depth Estimation: Toward Higher Resolution Maps with Accurate Object Boundaries », *arXiv:1803.08673 [cs]*, sept. 2018, Consulté le: mai 25, 2020. [En ligne]. Disponible à: http://arxiv.org/abs/1803.08673.

[52] I. Laina, C. Rupprecht, V. Belagiannis, F. Tombari, et N. Navab, « Deeper Depth Prediction with Fully Convolutional Residual Networks », *arXiv:1606.00373 [cs]*, juin 2016, Consulté le: janv. 22, 2019. [En ligne]. Disponible à: http://arxiv.org/abs/1606.00373.

[53] K. He, X. Zhang, S. Ren, et J. Sun, « Deep Residual Learning for Image Recognition », *arXiv:1512.03385 [cs]*, déc. 2015, Consulté le: juin 16, 2020. [En ligne]. Disponible à: http://arxiv.org/abs/1512.03385.

[54] D. Eigen, C. Puhrsch, et R. Fergus, « Depth Map Prediction from a Single Image using a Multi-Scale Deep Network », dans *Advances in Neural Information Processing Systems 27*, Z. Ghahramani, M. Welling, C. Cortes, N. D. Lawrence, et K. Q. Weinberger, Éd. Curran Associates, Inc., 2014, p. 2366–2374.

[55] F. Liu, C. Shen, G. Lin, et I. Reid, « Learning Depth from Single Monocular Images Using Deep Convolutional Neural Fields », *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 38, n° 10, p. 2024-2039, oct. 2016, doi: 10.1109/TPAMI.2015.2505283.

[56] Bo Li, Chunhua Shen, Yuchao Dai, A. van den Hengel, et Mingyi He, « Depth and surface normal estimation from monocular images using regression on deep features and hierarchical CRFs », dans *2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, juin 2015, p. 1119-1127, doi: 10.1109/CVPR.2015.7298715.

[57] D. Xu, W. Wang, H. Tang, H. Liu, N. Sebe, et E. Ricci, « Structured Attention Guided Convolutional Neural Fields for Monocular Depth Estimation », *arXiv:1803.11029 [cs]*, mars 2018, Consulté le: juin 17, 2020. [En ligne]. Disponible à: http://arxiv.org/abs/1803.11029.

[58] H. Fu, M. Gong, C. Wang, K. Batmanghelich, et D. Tao, « Deep Ordinal Regression Network for Monocular Depth Estimation », *arXiv:1806.02446 [cs]*, juin 2018, Consulté le: juin 17, 2020. [En ligne]. Disponible à: http://arxiv.org/abs/1806.02446.

[59] C. Godard, O. Mac Aodha, et G. J. Brostow, « Unsupervised Monocular Depth Estimation with Left-Right Consistency », *arXiv:1609.03677 [cs, stat]*, avr. 2017, Consulté le: juin 17, 2020. [En ligne]. Disponible à: http://arxiv.org/abs/1609.03677.

[60] J. Engel, V. Koltun, et D. Cremers, « Direct Sparse Odometry », *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 40, nº 3, p. 611-625, mars 2018, doi: 10.1109/TPAMI.2017.2658577.

[61] M. Pizzoli, C. Forster, et D. Scaramuzza, « REMODE: Probabilistic, monocular dense reconstruction in real time », dans *2014 IEEE International Conference on Robotics and Automation (ICRA)*, Hong Kong, China, mai 2014, p. 2609-2616, doi: 10.1109/ICRA.2014.6907233.

[62] B. Ummenhofer *et al.*, « DeMoN: Depth and Motion Network for Learning Monocular Stereo », *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, p. 5622-5631, juill. 2017, doi: 10.1109/CVPR.2017.596.

[63] T. Zhou, M. Brown, N. Snavely, et D. G. Lowe, « Unsupervised Learning of Depth and Ego-Motion from Video », dans *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, juill. 2017, p. 6612-6619, doi: 10.1109/CVPR.2017.700.

[64] N. Yang, L. von Stumberg, R. Wang, et D. Cremers, « D3VO: Deep Depth, Deep Pose and Deep Uncertainty for Monocular Visual Odometry », *arXiv:2003.01060 [cs]*, mars 2020, Consulté le: août 31, 2020. [En ligne]. Disponible à: http://arxiv.org/abs/2003.01060.

[65] H. Schneiderman et M. Nashman, « A discriminating feature tracker for vision-based autonomous driving », *IEEE Transactions on Robotics and Automation*, vol. 10, nº 6, p. 769-775, déc. 1994, doi: 10.1109/70.338531.

[66] U. Franke, D. Gavrila, S. Gorzig, F. Lindner, F. Puetzold, et C. Wohler, « Autonomous driving goes downtown », *IEEE Intelligent Systems and their Applications*, vol. 13, nº 6, p. 40-48, nov. 1998, doi: 10.1109/5254.736001.

[67] M. Nieuwenhuisen et S. Behnke, « Hierarchical Planning with 3D Local Multiresolution Obstacle Avoidance for Micro Aerial Vehicles », p. 7.

[68] H. Chao, Y. Cao, et Y. Chen, « Autopilots for small unmanned aerial vehicles: A survey », *International Journal of Control, Automation and Systems*, vol. 8, nº 1, p. 36-44, févr. 2010, doi: 10.1007/s12555-010-0105-z.

[69] L. Heng *et al.*, « Autonomous Visual Mapping and Exploration With a Micro Aerial Vehicle », *Journal of Field Robotics*, vol. 31, nº 4, p. 654-675, 2014, doi: 10.1002/rob.21520.

[70] L. von Stumberg, V. Usenko, J. Engel, J. Stückler, et D. Cremers, « From Monocular SLAM to Autonomous Drone Exploration », *2017 European Conference on Mobile Robots (ECMR)*, p. 1-8, sept. 2017, doi: 10.1109/ECMR.2017.8098709.

[71] M. Blösch, S. Weiss, D. Scaramuzza, et R. Siegwart, « Vision based MAV navigation in unknown and unstructured environments », dans *2010 IEEE International Conference on Robotics and Automation*, mai 2010, p. 21-28, doi: 10.1109/ROBOT.2010.5509920.

[72] S. Kamburugamuve, H. He, G. Fox, et D. Crandall, « Cloud-based Parallel Implementation of SLAM for Mobile Robots », dans *Proceedings of the International Conference on Internet of things and Cloud Computing - ICC '16*, Cambridge, United Kingdom, 2016, p. 1-7, doi: 10.1145/2896387.2896433.

[73] T. Osa, J. Pajarinen, G. Neumann, J. A. Bagnell, P. Abbeel, et J. Peters, « An Algorithmic Perspective on Imitation Learning », *Foundations and Trends in Robotics*, vol. 7, nº 1-2, p. 1-179, 2018, doi: 10.1561/2300000053.

[74] R. Wang, C. Ciliberto, P. Amadori, et Y. Demiris, « Random Expert Distillation: Imitation Learning via Expert Policy Support Estimation », *arXiv:1905.06750 [cs, stat]*, juin 2019, Consulté le: mars 09, 2020. [En ligne]. Disponible à: http://arxiv.org/abs/1905.06750.

[75] A. Loquercio, A. I. Maqueda, C. R. del-Blanco, et D. Scaramuzza, « DroNet: Learning to Fly by Driving », *IEEE Robotics and Automation Letters*, vol. 3, nº 2, p. 1088-1095, avr. 2018, doi: 10.1109/LRA.2018.2795643.

[76] M. Bojarski *et al.*, « End to End Learning for Self-Driving Cars », *arXiv:1604.07316 [cs]*, avr. 2016, Consulté le: janv. 10, 2019. [En ligne]. Disponible à: http://arxiv.org/abs/1604.07316.

[77] N. Dinh Van et G.-W. Kim, « Fuzzy Logic and Deep Steering Control based Recommendation System for Self-Driving Car », dans *2018 18th International Conference on Control, Automation and Systems (ICCAS)*, oct. 2018, p. 1107-1110.

[78] « udacity/self-driving-car », *GitHub*. https://github.com/udacity/self-driving-car (consulté le juin 15, 2020).

[79] « Become a Self-Driving Car Engineer ». https://www.udacity.com/course/self-driving-car-engineer-nanodegree--nd013 (consulté le juin 15, 2020).

[80] D. Palossi, A. Loquercio, F. Conti, E. Flamand, D. Scaramuzza, et L. Benini, « A 64mW DNN-based Visual Navigation Engine for Autonomous Nano-Drones », *IEEE Internet Things J.*, vol. 6, nº 5, p. 8357-8371, oct. 2019, doi: 10.1109/JIOT.2019.2917066.

[81] « Home | Bitcraze ». https://www.bitcraze.io/ (consulté le juin 15, 2020).

[82] S. Ross *et al.*, « Learning Monocular Reactive UAV Control in Cluttered Natural Environments », *arXiv:1211.1690 [cs]*, nov. 2012, Consulté le: mai 19, 2020. [En ligne]. Disponible à: http://arxiv.org/abs/1211.1690.

[83] N. Smolyanskiy, A. Kamenev, J. Smith, et S. Birchfield, « Toward low-flying autonomous MAV trail navigation using deep neural networks for environmental awareness », dans *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, Vancouver, BC, sept. 2017, p. 4241-4247, doi: 10.1109/IROS.2017.8206285.

[84] R. P. Padhy, S. Verma, S. Ahmad, S. K. Choudhury, et P. K. Sa, « Deep Neural Network for Autonomous UAV Navigation in Indoor Corridor Environments », *Procedia Computer Science*, vol. 133, p. 643-650, janv. 2018, doi: 10.1016/j.procs.2018.07.099.

[85] A. Kouris et C. Bouganis, « Learning to Fly by MySelf: A Self-Supervised CNN-Based Approach for Autonomous Navigation », dans *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, oct. 2018, p. 1-9, doi: 10.1109/IROS.2018.8594204.

[86] « Drones », *Parrot Store Official*. https://www.parrot.com/ca/drones (consulté le juin 15, 2020).

[87] J. Xu, S. Zhu, H. Guo, et S. Wu, « Automated Labeling for Robotic Autonomous Navigation Through Multi-Sensory Semi-Supervised Learning on Big Data », *IEEE Transactions on Big Data*, p. 1-1, 2019, doi: 10.1109/TBDATA.2019.2892462.

[88] R. Glasgow, « Minimal Selfhood and the Origins of Consciousness », Consulté le: avr. 29, 2019. [En ligne]. Disponible à: https://opus.bibliothek.uni-wuerzburg.de/frontdoor/index/index/docId/15747.

[89] S. Yang, S. Konam, C. Ma, S. Rosenthal, M. Veloso, et S. Scherer, « Obstacle Avoidance through Deep Networks based Intermediate Perception », *arXiv:1704.08759 [cs]*, avr. 2017, Consulté le: févr. 07, 2019. [En ligne]. Disponible à: http://arxiv.org/abs/1704.08759.

[90] C. Greatwood et A. G. Richards, « Reinforcement learning and model predictive control for robust embedded quadrotor guidance and control », *Autonomous Robots*, vol. 43, nº 7, p. 1681-1693, oct. 2019, doi: 10.1007/s10514-019-09829-4.

[91] Y. Wang et S. Boyd, « Fast Model Predictive Control Using Online Optimization », *IEEE Transactions on Control Systems Technology*, vol. 18, nº 2, p. 267-278, mars 2010, doi: 10.1109/TCST.2009.2017934.

[92] G. Kahn, A. Villaflor, B. Ding, P. Abbeel, et S. Levine, « Self-Supervised Deep Reinforcement Learning with Generalized Computation Graphs for Robot Navigation », dans *2018 IEEE International Conference on Robotics and Automation (ICRA)*, mai 2018, p. 5129-5136, doi: 10.1109/ICRA.2018.8460655.

[93] A. Ghosh, B. Bhattacharya, et S. B. R. Chowdhury, « SAD-GAN: Synthetic Autonomous Driving using Generative Adversarial Networks », *arXiv:1611.08788 [cs]*, nov. 2016, Consulté le: févr. 19, 2019. [En ligne]. Disponible à: http://arxiv.org/abs/1611.08788.

[94] P. Chakravarty, K. Kelchtermans, T. Roussel, S. Wellens, T. Tuytelaars, et L. V. Eycken, « CNN-based single image obstacle avoidance on a quadrotor », dans *2017 IEEE International Conference on Robotics and Automation (ICRA)*, mai 2017, p. 6369-6374, doi: 10.1109/ICRA.2017.7989752.

[95] U. Farooq, M. Amar, E. ul Haq, M. U. Asad, et H. M. Atiq, « Microcontroller Based Neural Network Controlled Low Cost Autonomous Vehicle », dans *2010 Second International Conference on Machine Learning and Computing*, févr. 2010, p. 96-100, doi: 10.1109/ICMLC.2010.71.

[96] U. Farooq, M. Amar, M. U. Asad, A. Hanif, et S. O. Saleh, « Design and Implementation of Neural Network Based Controller for Mobile Robot Navigation in Unknown Environments », *International Journal of Computer and Electrical Engineering*, vol. 6, nº 2, p. 83-89, 2014, doi: 10.7763/IJCEE.2014.V6.799.

[97] M. Pfeiffer, M. Schaeuble, J. Nieto, R. Siegwart, et C. Cadena, « From Perception to Decision: A Data-driven Approach to End-to-end Motion Planning for Autonomous Ground

Robots », *2017 IEEE International Conference on Robotics and Automation (ICRA)*, p. 1527-1533, mai 2017, doi: 10.1109/ICRA.2017.7989182.

[98] S. Bansal, V. Tolani, S. Gupta, J. Malik, et C. Tomlin, « Combining Optimal Control and Learning for Visual Navigation in Novel Environments », *arXiv:1903.02531 [cs]*, juill. 2019, Consulté le: juin 10, 2020. [En ligne]. Disponible à: http://arxiv.org/abs/1903.02531.

[99] J. Kim et S. Jeong, « Target-Driven Navigation with Imitation Learning », p. 7, 2017.

[100] A. Hussein, E. Elyan, M. M. Gaber, et C. Jayne, « Deep imitation learning for 3D navigation tasks », *Neural Computing and Applications*, vol. 29, nº 7, p. 389-404, avr. 2018, doi: 10.1007/s00521-017-3241-z.

[101] S. Gupta, D. Fouhey, S. Levine, et J. Malik, « Unifying Map and Landmark Based Representations for Visual Navigation », *arXiv:1712.08125 [cs]*, déc. 2017, Consulté le: juin 11, 2020. [En ligne]. Disponible à: http://arxiv.org/abs/1712.08125.

[102] S. Gupta, J. Davidson, S. Levine, R. Sukthankar, et J. Malik, « Cognitive Mapping and Planning for Visual Navigation », 2017, p. 2616-2625, Consulté le: juin 10, 2020. [En ligne]. Disponible à: http://openaccess.thecvf.com/content_cvpr_2017/html/Gupta_Cognitive_Mapping_and_CVPR_2017_paper.html.

[103] D. S. Chaplot, D. Gandhi, S. Gupta, A. Gupta, et R. Salakhutdinov, « Learning to Explore using Active Neural SLAM », *arXiv:2004.05155 [cs]*, avr. 2020, Consulté le: août 31, 2020. [En ligne]. Disponible à: http://arxiv.org/abs/2004.05155.

[104] « Facebook ». https://www.facebook.com/ (consulté le sept. 01, 2020).

[105] X. Zhou, Y. Gao, et L. Guan, « Towards Goal-Directed Navigation Through Combining Learning Based Global and Local Planners », *Sensors*, vol. 19, nº 1, p. 176, janv. 2019, doi: 10.3390/s19010176.

[106] O. Zhelo, J. Zhang, L. Tai, M. Liu, et W. Burgard, « Curiosity-driven Exploration for Mapless Navigation with Deep Reinforcement Learning », *arXiv:1804.00456 [cs]*, mai 2018, Consulté le: juin 11, 2020. [En ligne]. Disponible à: http://arxiv.org/abs/1804.00456.

[107] A. Khan, V. Kumar, et A. Ribeiro, « Learning Sample-Efficient Target Reaching for Mobile Robots », dans *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, oct. 2018, p. 3080-3087, doi: 10.1109/IROS.2018.8594168.

[108] A. Khan, C. Zhang, N. Atanasov, K. Karydis, V. Kumar, et D. D. Lee, « Memory Augmented Control Networks », *arXiv:1709.05706 [cs]*, févr. 2018, Consulté le: juin 10, 2020. [En ligne]. Disponible à: http://arxiv.org/abs/1709.05706.

[109] L. Chen, X. Hu, W. Tian, H. Wang, D. Cao, et F. Wang, « Parallel planning: a new motion planning framework for autonomous driving », *IEEE/CAA Journal of Automatica Sinica*, vol. 6, nº 1, p. 236-246, janv. 2019, doi: 10.1109/JAS.2018.7511186.

[110] P. Mirowski *et al.*, « Learning to Navigate in Cities Without a Map », dans *Advances in Neural Information Processing Systems 31*, S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, et R. Garnett, Éd. Curran Associates, Inc., 2018, p. 2419–2430.

[111] S. Paschall et J. Rose, « Fast, lightweight autonomy through an unknown cluttered environment: Distribution statement: A — Approved for public release; distribution

unlimited », dans *2017 IEEE Aerospace Conference*, mars 2017, p. 1-8, doi: 10.1109/AERO.2017.7943617.

[112] J. Tordesillas, B. T. Lopez, M. Everett, et J. P. How, « FASTER: Fast and Safe Trajectory Planner for Flights in Unknown Environments », *arXiv:2001.04420 [cs]*, janv. 2020, Consulté le: sept. 01, 2020. [En ligne]. Disponible à: http://arxiv.org/abs/2001.04420.

[113] A. Borowczyk, D.-T. Nguyen, A. Phu-Van Nguyen, D. Q. Nguyen, D. Saussié, et J. L. Ny, « Autonomous Landing of a Multirotor Micro Air Vehicle on a High Velocity Ground Vehicle**This work was partially supported by CFI JELF Award 32848 and a hardware donation from DJI. », *IFAC-PapersOnLine*, vol. 50, nº 1, p. 10488-10494, juill. 2017, doi: 10.1016/j.ifacol.2017.08.1980.

[114] A. G. Perera, Y. W. Law, et J. Chahl, « Human Pose and Path Estimation from Aerial Video Using Dynamic Classifier Selection », *Cognitive Computation*, vol. 10, nº 6, p. 1019-1041, déc. 2018, doi: 10.1007/s12559-018-9577-6.

[115] A. Mohebbi, S. Achiche, et L. Baron, « Multi-criteria fuzzy decision support for conceptual evaluation in design of mechatronic systems: a quadrotor design case study », *Research in Engineering Design*, vol. 29, nº 3, p. 329-349, juill. 2018, doi: 10.1007/s00163-018-0287-6.

[116] A. Mohebbi, S. Achiche, et L. Baron, « Integrated and concurrent detailed design of a mechatronic quadrotor system using a fuzzy-based particle swarm optimization », *Engineering Applications of Artificial Intelligence*, vol. 82, p. 192-206, juin 2019, doi: 10.1016/j.engappai.2019.03.025.

[117] P. E. Hart, N. J. Nilsson, et B. Raphael, « A Formal Basis for the Heuristic Determination of Minimum Cost Paths », *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, nº 2, p. 100-107, juill. 1968, doi: 10.1109/TSSC.1968.300136.

[118] S. Karaman et E. Frazzoli, « Sampling-based Algorithms for Optimal Motion Planning », *arXiv:1105.1186 [cs]*, mai 2011, Consulté le: sept. 01, 2020. [En ligne]. Disponible à: http://arxiv.org/abs/1105.1186.

[119] M. J. Bency, A. H. Qureshi, et M. C. Yip, « Neural Path Planning: Fixed Time, Near-Optimal Path Generation via Oracle Imitation », *arXiv:1904.11102 [cs]*, avr. 2019, Consulté le: août 31, 2020. [En ligne]. Disponible à: http://arxiv.org/abs/1904.11102.

[120] « Fast Depth Prediction and Obstacle Avoidance on a Monocular Drone Using Probabilistic Convolutional Neural Network - IEEE Journals & Magazine ». https://ieeexplore.ieee.org/abstract/document/8924655?casa_token=EITRPIyNt7UAAAAA:sx_Hf-zO4cDANXtxb2MhijVjzaoCLdotLeE29ZMntmkaSuJkPRhsn94T-UcGO_CywkKIn05OpigT (consulté le juill. 30, 2020).

[121] I. Laina, *Deeper Depth Prediction with Fully Convolutional Residual Networks (FCRN): iro-cp/FCRN-DepthPrediction*. 2019.

[122] N. Silberman, D. Hoiem, P. Kohli, et R. Fergus, « Indoor Segmentation and Support Inference from RGBD Images », dans *Computer Vision – ECCV 2012*, vol. 7576, A. Fitzgibbon, S. Lazebnik, P. Perona, Y. Sato, et C. Schmid, Éd. Berlin, Heidelberg: Springer Berlin Heidelberg, 2012, p. 746-760.

[123] S. Zennaro *et al.*, « Performance evaluation of the 1st and 2nd generation Kinect for multimedia applications », dans *2015 IEEE International Conference on Multimedia and Expo (ICME)*, juin 2015, p. 1-6, doi: 10.1109/ICME.2015.7177380.

[124] « Vicon | Award Winning Motion Capture Systems », *Vicon*. https://www.vicon.com/ (consulté le juill. 15, 2020).

[125] « Iphone Background - Unlimited Download. cleanpng.com. », *cleanpng.com*. https://www.cleanpng.com/png-parrot-ar-drone-2-0-parrot-bebop-drone-ar-freeflig-2224708/ (consulté le juill. 30, 2020).

[126] « ROS.org | Powering the world's robots ». https://www.ros.org/ (consulté le juill. 24, 2020).

[127] « Gazebo ». http://gazebosim.org/ (consulté le juill. 24, 2020).

[128] *AutonomyLab/ardrone_autonomy*. Autonomy Lab at SFU, 2020.

[129] « tum_simulator - ROS Wiki ». http://wiki.ros.org/tum_simulator (consulté le avr. 15, 2020).

[130] G. Kozhevnikov, *Barahlush/ardrone_autopilot*. 2020.

[131] SPACESCAN, « Abandoned Bar #1 ». https://www.turbosquid.com/3d-models/free-abandoned-bar-1-3d-model/1098424 (consulté le juill. 24, 2020).

[132] « SPACESCAN 3D Models for Download | TurboSquid ». https://www.turbosquid.com/Search/Artists/SPACESCAN (consulté le juill. 24, 2020).

[133] « The HDF5® Library & File Format », *The HDF Group*. https://www.hdfgroup.org/solutions/hdf5/ (consulté le juill. 24, 2020).

[134] A. Paszke *et al.*, « PyTorch: An Imperative Style, High-Performance Deep Learning Library », dans *Advances in Neural Information Processing Systems 32*, H. Wallach, H. Larochelle, A. Beygelzimer, F. d\textquotesingle Alché-Buc, E. Fox, et R. Garnett, Éd. Curran Associates, Inc., 2019, p. 8026–8037.

[135] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, et R. Salakhutdinov, « Dropout: A Simple Way to Prevent Neural Networks from Overfitting », p. 30.

[136] J. M. Johnson et T. M. Khoshgoftaar, « Survey on deep learning with class imbalance », *J Big Data*, vol. 6, nº 1, p. 27, mars 2019, doi: 10.1186/s40537-019-0192-5.

[137] « torch.nn — PyTorch master documentation ». https://pytorch.org/docs/stable/nn.html?highlight=cross%20entropy#torch.nn.CrossEntropyLoss (consulté le juill. 25, 2020).

[138] « Skydio 2: The drone you've been waiting for. », *Skydio, Inc.* https://www.skydio.com/ (consulté le juin 28, 2020).

[139] G. Kahn, A. Bachrach, et H. Martiros, « How We Trained a Deep Neural Pilot to Autonomously Fly the Skydio Drone - IEEE Spectrum », *IEEE Spectrum: Technology, Engineering, and Science News*, févr. 13, 2020. https://spectrum.ieee.org/automaton/robotics/drones/deep-neural-pilot-skydio-2 (consulté le juin 28, 2020).

[140] P. Sermanet *et al.*, « Time-Contrastive Networks: Self-Supervised Learning from Video », dans *2018 IEEE International Conference on Robotics and Automation (ICRA)*, mai 2018, p. 1134-1141, doi: 10.1109/ICRA.2018.8462891.

[141] Comet.ml, « Comet.ml - Supercharging Machine Learning », *Comet.ml - Supercharging Machine Learning*. https://www.comet.ml/ (consulté le juin 29, 2020).

[142] L. Perez et J. Wang, « The Effectiveness of Data Augmentation in Image Classification using Deep Learning », *arXiv:1712.04621 [cs]*, déc. 2017, Consulté le: févr. 28, 2019. [En ligne]. Disponible à: http://arxiv.org/abs/1712.04621.

[143] M. Balazinski, S. Achiche, et L. Baron, « Influence of Optimization and Selection Criteria on Genetically-Generated Fuzzy Knowledge Bases », août 2000.

[144] S. Achiche, M. Balazinski, et L. Baron, « Multi-combinative strategy to avoid premature convergence in genetically-generated fuzzy knowledge bases », *Journal of Theoretical and Applied Mechanics*, vol. 42, Art. n° 3, 2004.

[145] SPACESCAN, « Old Bar ». https://www.turbosquid.com/3d-models/free-3ds-mode-gangster-bar-pool-room/1114542 (consulté le juill. 24, 2020).

[146] A. Choromanska, M. Henaff, M. Mathieu, G. B. Arous, et Y. LeCun, « The Loss Surfaces of Multilayer Networks », *arXiv:1412.0233 [cs]*, janv. 2015, Consulté le: juill. 25, 2020. [En ligne]. Disponible à: http://arxiv.org/abs/1412.0233.

[147] T. Poggio *et al.*, « Theory of Deep Learning III: explaining the non-overfitting puzzle », *arXiv:1801.00173 [cs]*, janv. 2018, Consulté le: juill. 25, 2020. [En ligne]. Disponible à: http://arxiv.org/abs/1801.00173.

[148] H. Touvron, A. Vedaldi, M. Douze, et H. Jégou, « Fixing the train-test resolution discrepancy: FixEfficientNet », *arXiv:2003.08237 [cs]*, avr. 2020, Consulté le: juill. 01, 2020. [En ligne]. Disponible à: http://arxiv.org/abs/2003.08237.

[149] SPACESCAN, « 3D luxury home 4 model - TurboSquid 1367805 ». https://www.turbosquid.com/3d-models/3d-luxury-home-4-model-1367805 (consulté le juill. 24, 2020).

[150] SPACESCAN, « Abandoned office interior 3D model - TurboSquid 1366534 ». https://www.turbosquid.com/3d-models/abandoned-office-interior-3d-model-1366534 (consulté le juill. 24, 2020).

[151] SPACESCAN, « Scanned home 2 house 3D model - TurboSquid 1189324 ». https://www.turbosquid.com/3d-models/scanned-home-2-house-3d-model-1189324 (consulté le juill. 24, 2020).

ANNEXE A : CARACTÉRISTIQUES DES MODÈLES 3D DES ENVIRONNEMENTS

L'Annexe A est dédiée à présenter visuellement les modèles 3D des environnements sélectionnés pour la génération de l'ensemble de données. Chaque modèle présent dans le Tableau 3.4 aura au minimum une prise de vue et affichera la numérotation utilisée pour marquer les lieux d'intérêts nécessaires à la création des trajectoires.

En premier lieu, voici l'environnement *Bar à Billard*.



Figure A.1 Vue intérieur d'une des pièces du *Bar à Billard* [145].



Figure A.2 Vue isométrique (gauche) et orthogonale supérieure (droite) du *Bar à Billard* [145].



Figure A.3 Numérotation des points d'intérêts du B*ar à Billard*

Voici ensuite le modèle de l'environnement *Maison 1*.



Figure A.4 Vue isométrique du premier étage de la *Maison 1* [149].



Figure A.5 Vue orthogonale du premier étage de la *Maison 1* [149].



Figure A.6 Vue isométrique du deuxième étage de la *Maison 1* [149].



Figure A.7 Vues intérieures du salon, d'un couloir et de la cuisine de la *Maison 1* [149].

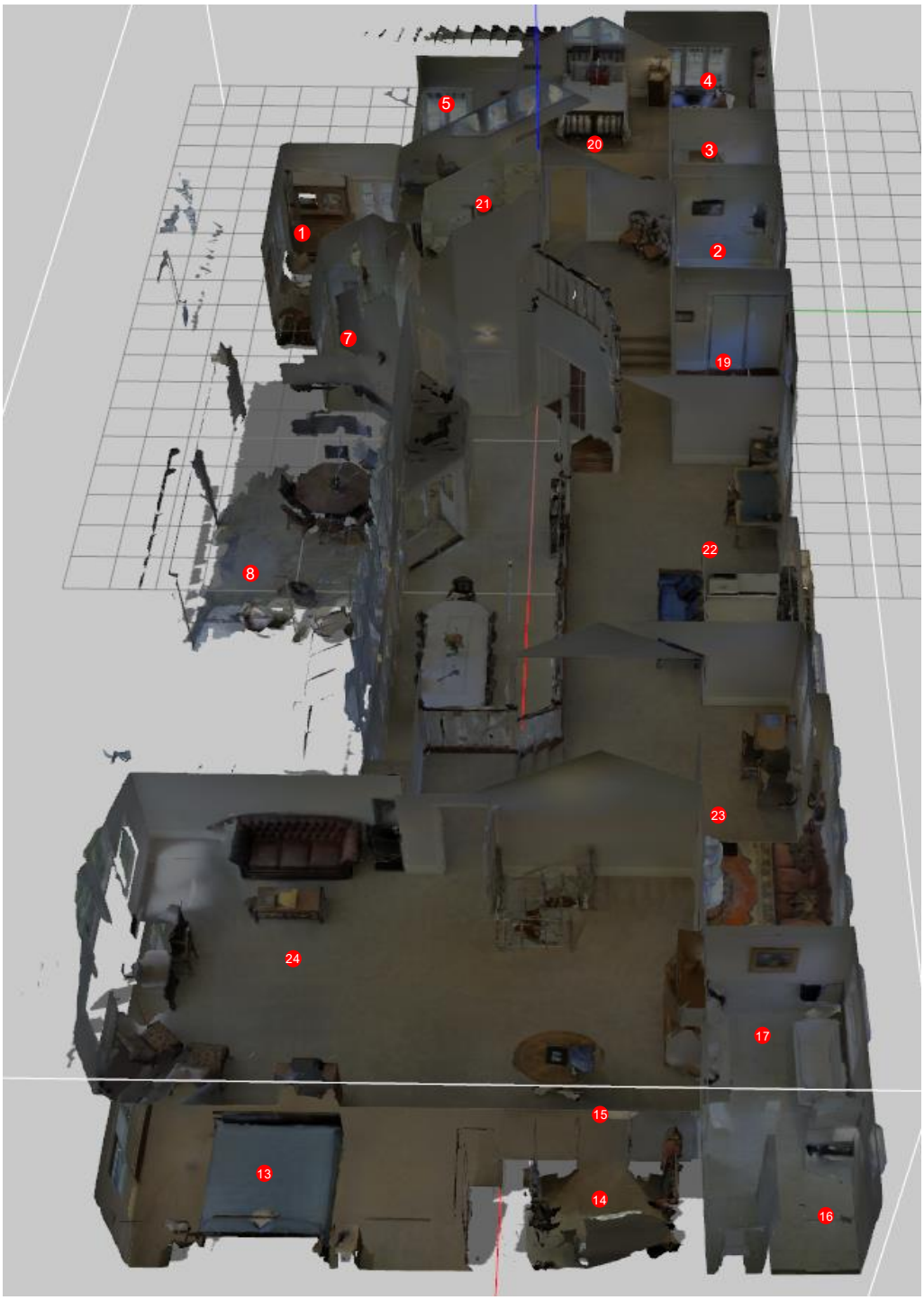


Figure A.8 Numérotation des points d'intérêt de la *Maison 1*, vue de côté en perspective.

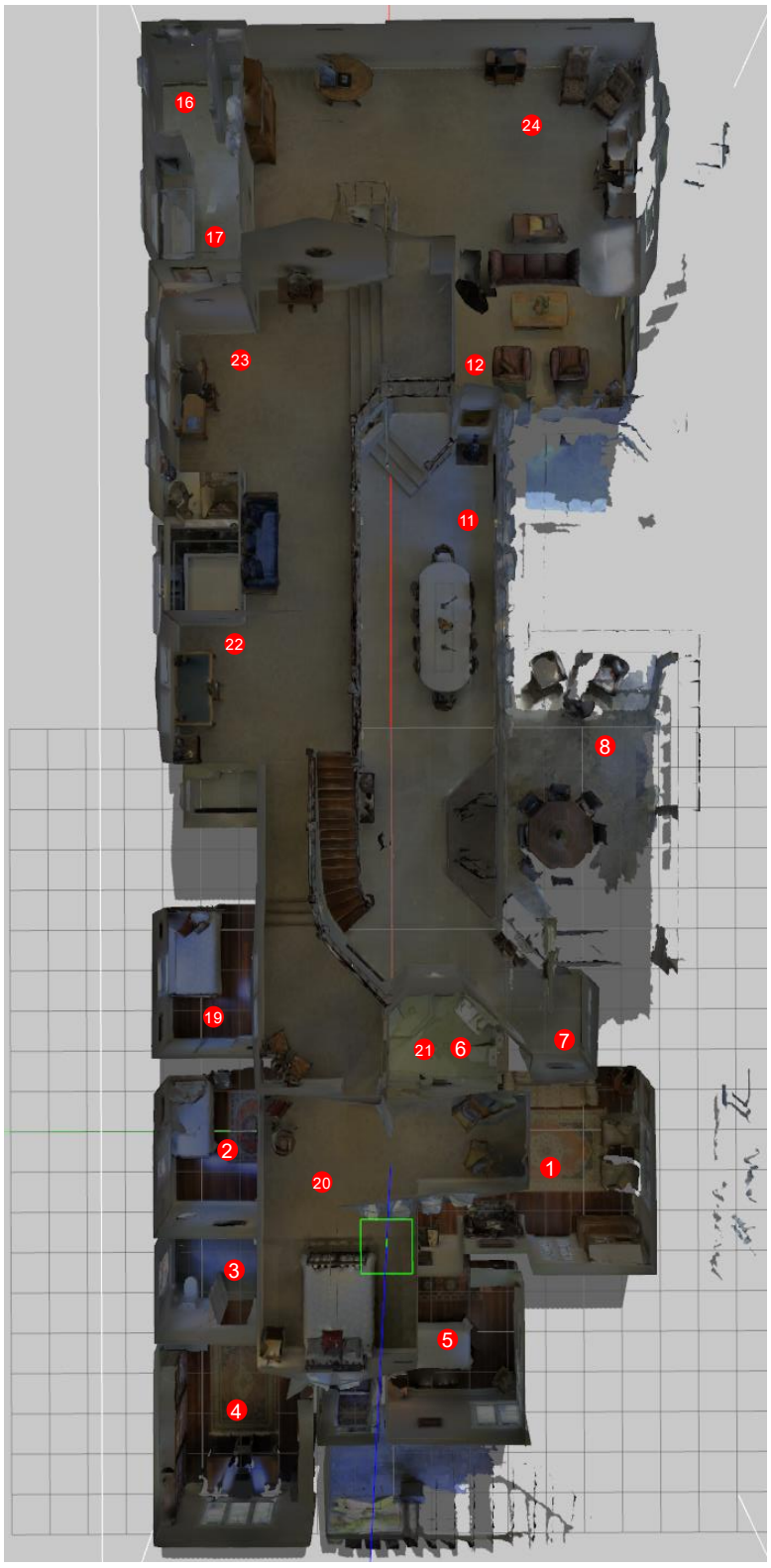


Figure A.9 Numérotation des points d'intérêt de la *Maison 1*, vue orthogonale.



Figure A.10 Numérotation des points d'intérêt de la *Maison 1*, vue isométrique.

Ensuite suit le modèle 3D de l'environnement *Salle de machines*.



Figure A.11 Vue intérieure 1 de la *Salle de machines*

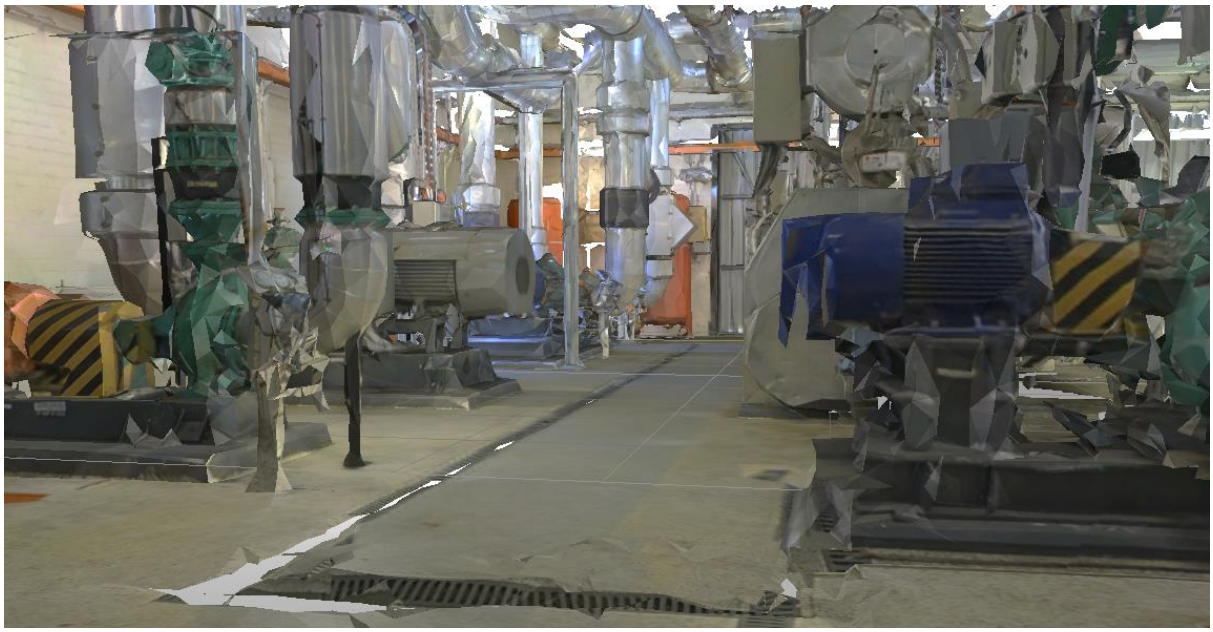


Figure A.12 Vue intérieure 2 de la *Salle de machines*

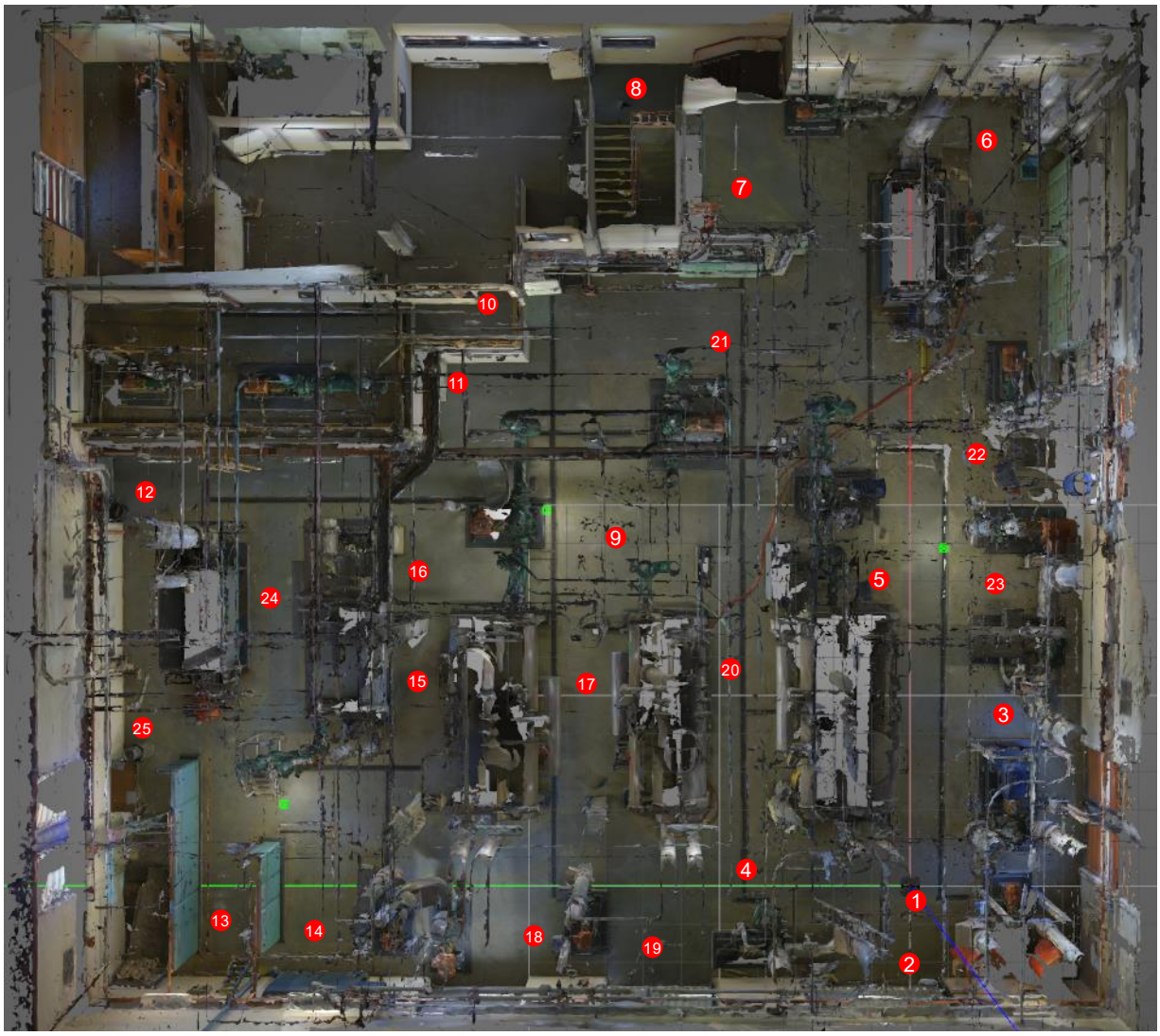


Figure A.13 Numérotation des points d'intérêt de la *Salle de machines.*

Le modèle 3D de la *Salle de serveurs*.

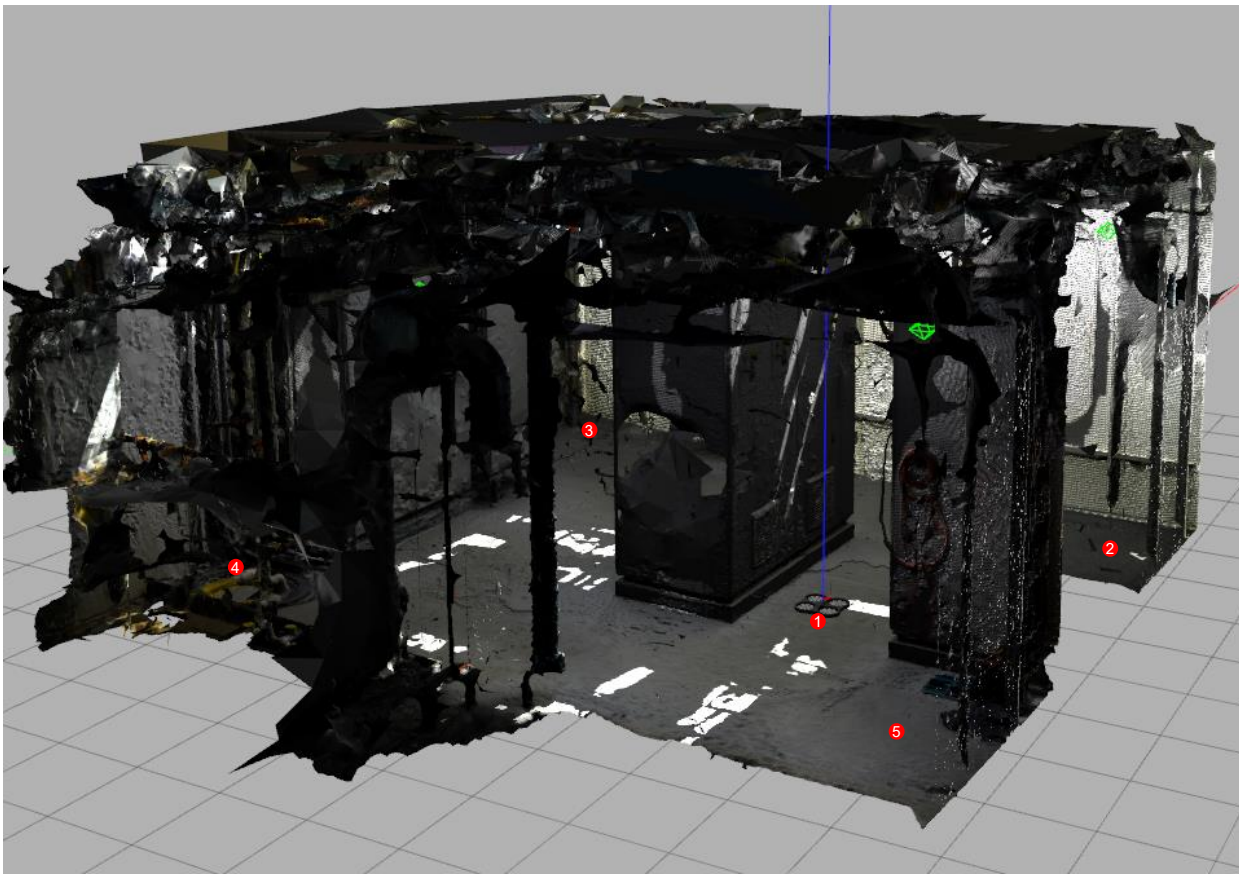


Figure A.14 Numérotation des points d'intérêt de la *Salle de serveurs.*

Le modèle 3D de l'environnement *Bureau.*



Figure A.15 Vue isométrique du *Bureau* [150].





Figure A.16 Vues intérieure du *Bureau*

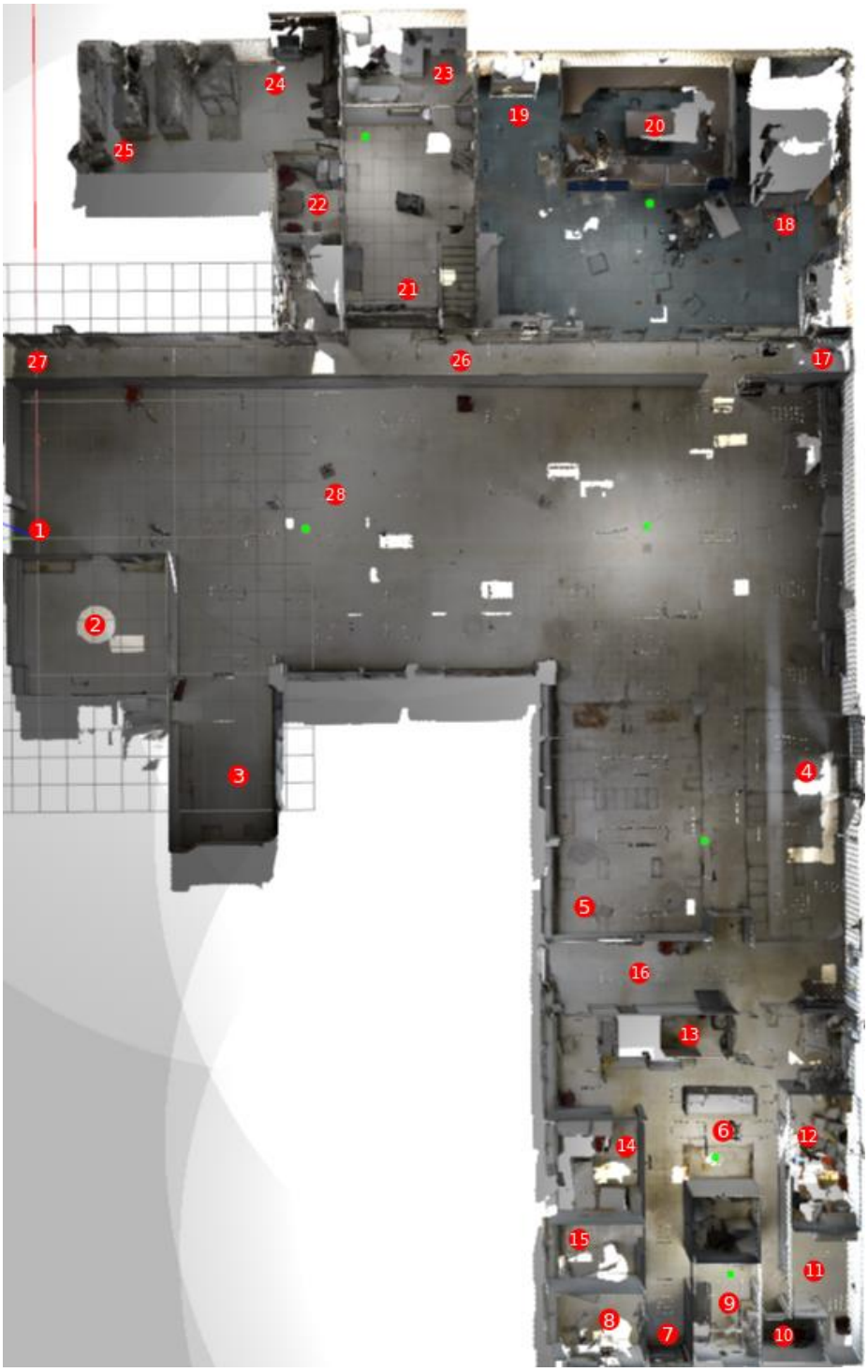


Figure A.17 Numérotation des points d'intérêt du *Bureau*.

Le modèle 3D de l'environnement *Resto-Bar*.

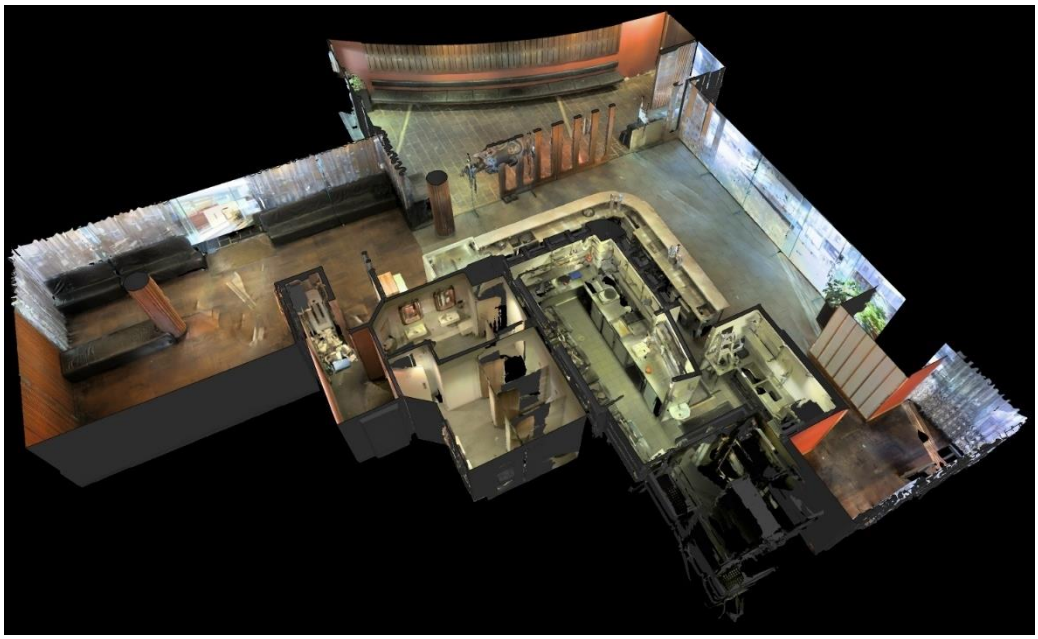


Figure A.18 Vue isométrique du *Resto-Bar* [131].



Figure A.19 Vue intérieure du *Resto-Bar* [131]..

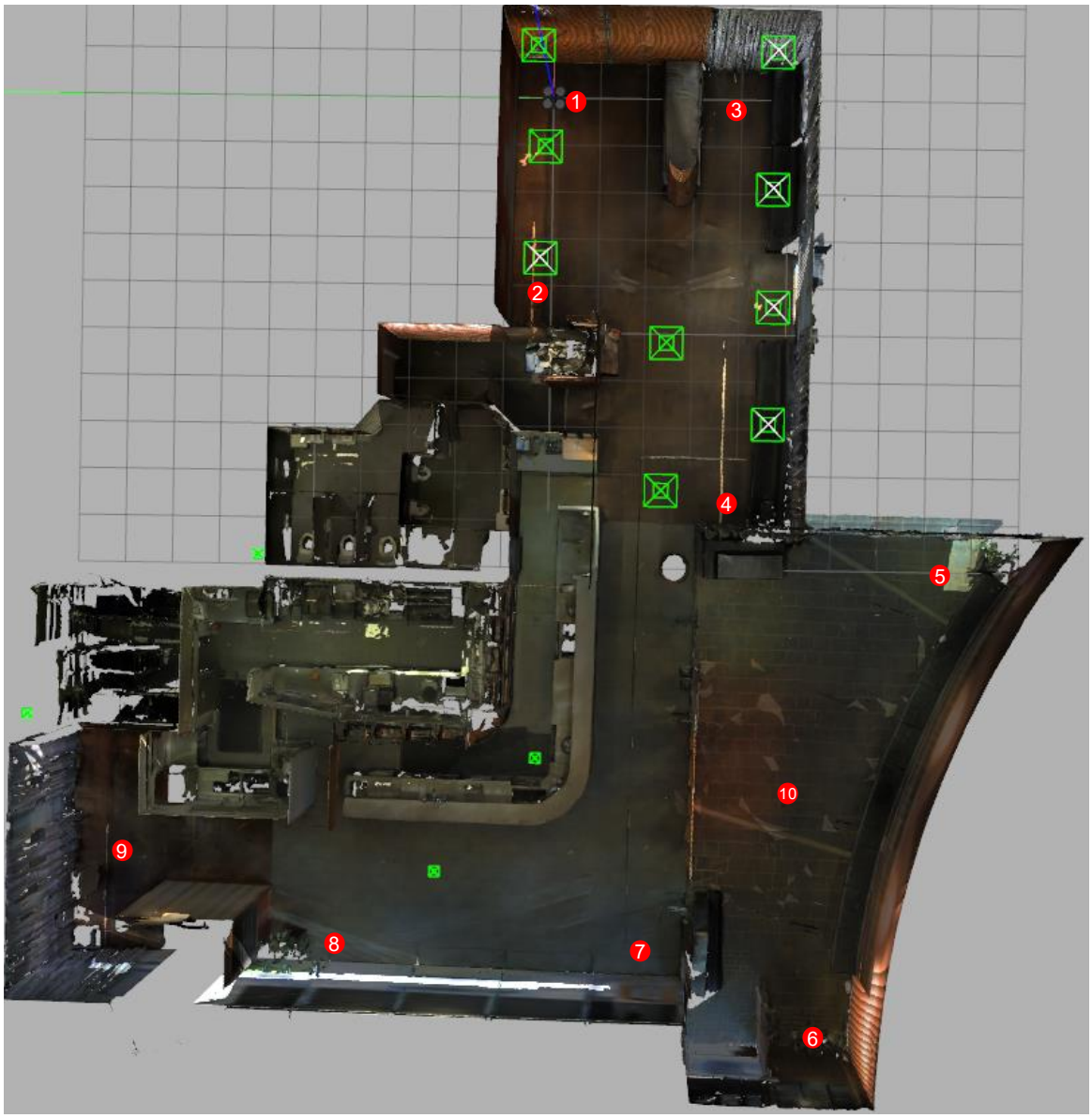


Figure A.20 Numérotation des points d'intérêt du *Resto-Bar*.

Enfin, le modèle 3D de l'environnement *Maison 2*.



Figure A.21 Vue orthogonale du premier étage de la *Maison 2* [151].





Figure A.22 Vues intérieur du salon et de la cuisine de la *Maison 2* [151].

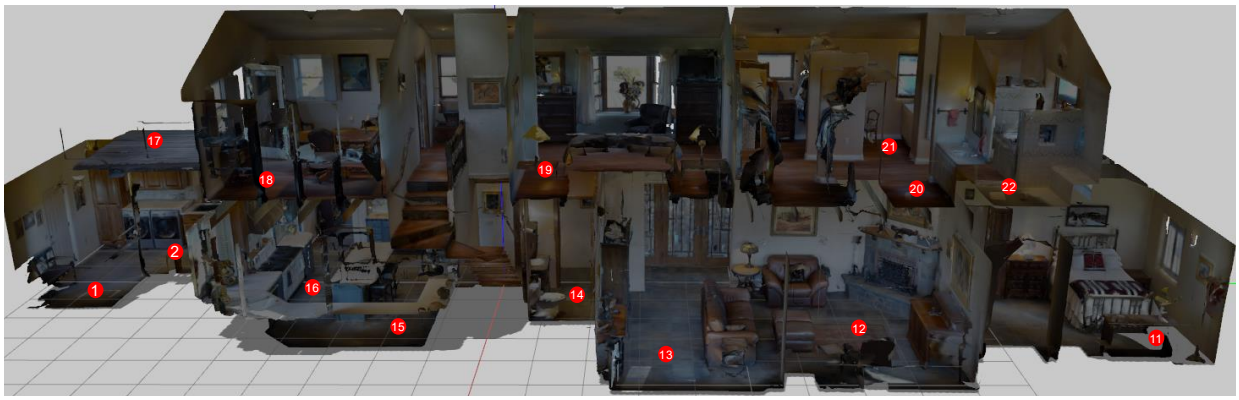


Figure A.23 Numérotation des points d'intérêt de la *Maison 2*, vue avant.

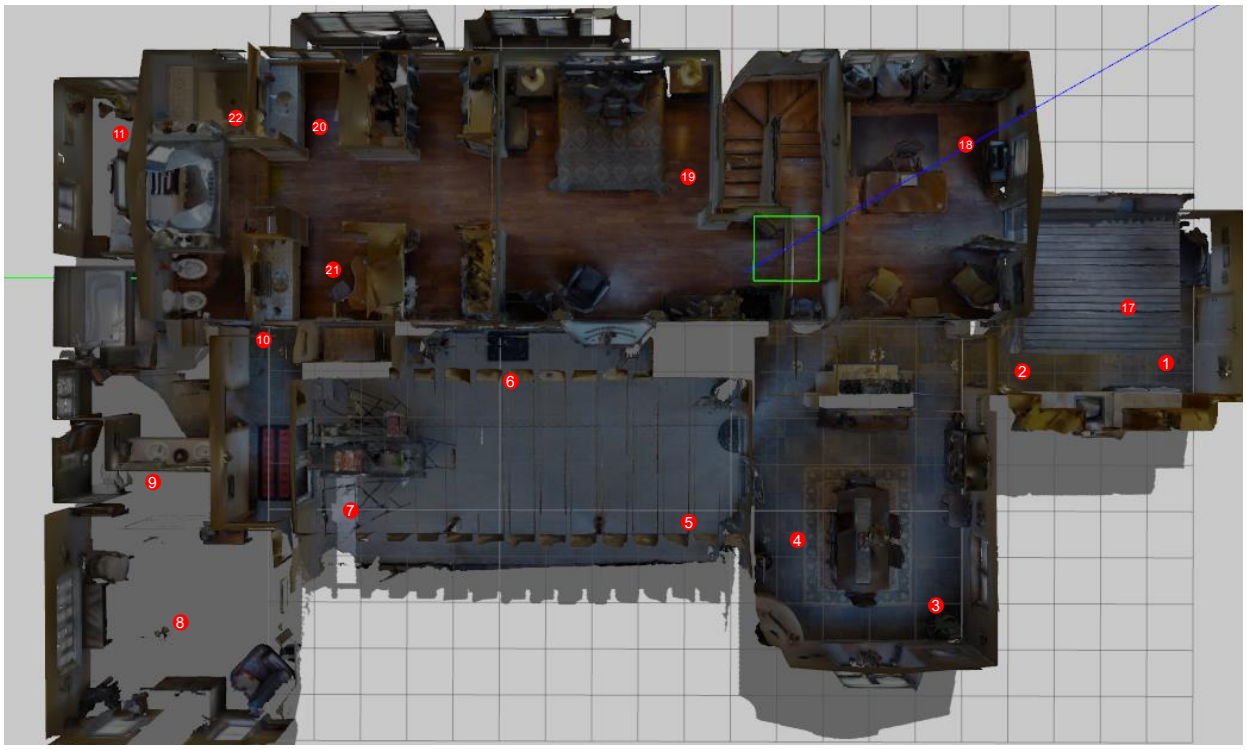


Figure A.24 Numérotation des points d'intérêt de la *Maison 2*, vue orthogonale supérieure.

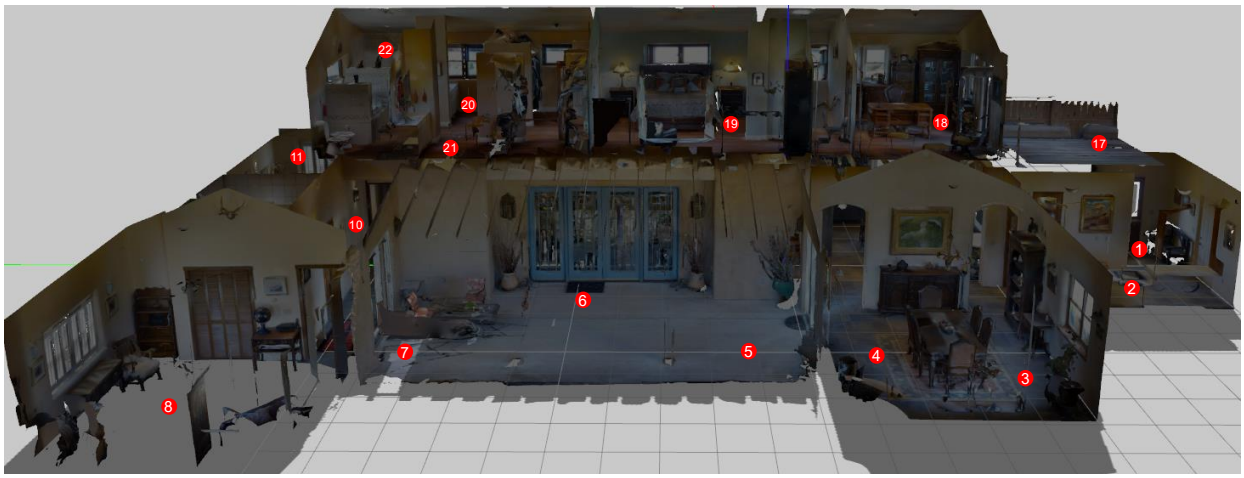


Figure A.25 Numérotation des points d'intérêt de la *Maison 2*, vue arrière.